

INFORMÁTICA Y COMUNICACIONES
G.S. Desarrollo de Aplicaciones
Multiplataforma



Acceso a datos

Carlos Alberto Cortijo Bon



EDITORIAL
SÍNTESIS

A large, solid blue curved shape on the left side of the slide, resembling a stylized 'C' or a partial circle.

Acceso a datos

ASESOR EDITORIAL:

Juan Carlos Moreno Pérez

© Carlos Alberto Cortijo Bon

© EDITORIAL SÍNTESIS, S. A.
Vallehermoso, 34. 28015 Madrid
Teléfono 91 593 90 98
<http://www.sintesis.com>

ISBN: 978-84-917188-1-9

Impreso en España - Printed in Spain

Reservados todos los derechos. Está prohibido, bajo las sanciones penales y el resarcimiento civil previstos en las leyes, reproducir, registrar o transmitir esta publicación, íntegra o parcialmente, por cualquier sistema de recuperación y por cualquier medio, sea mecánico, electrónico, magnético, electroóptico, por fotocopia o por cualquier otro, sin la autorización previa por escrito de Editorial Síntesis, S. A.

Índice

PRESENTACIÓN	11
1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS	13
Objetivos	13
Mapa conceptual	14
Glosario	14
1.1. Programas y datos	15
1.2. Persistencia de datos	15
1.3. Sistemas de persistencia de datos	16
1.4. Almacenamiento de datos	17
1.4.1. Ficheros	17
1.4.2. Bases de datos relacionales	18
1.4.3. Documentos de XML	18
1.4.4. Bases de datos de objetos	19
1.4.5. Bases de datos NoSQL	20
1.5. Restricciones de integridad	20
1.6. Acceso a los datos con iteradores	20
1.7. Control de accesos concurrentes y transacciones	20
1.8. Persistencia de datos en ficheros	22
1.9. Persistencia de datos en bases de datos relacionales	22
1.9.1. Persistencia de datos de XML en bases de datos relacionales	23
1.9.2. Persistencia de objetos en bases de datos relacionales	23
1.10. Persistencia de datos en bases de datos de objetos	24
1.11. Persistencia de datos en bases de datos de XML nativas	25
1.12. Persistencia de datos en bases de datos NoSQL	26
Resumen	27
Actividades de autoevaluación	29

2. FICHEROS	31
Objetivos	31
Mapa conceptual	32
Glosario	32
2.1. Persistencia de datos en ficheros	33
2.2. Tipos de ficheros según su contenido	34
2.3. Codificaciones para texto	34
2.4. La clase File de Java	35
2.5. Gestión de excepciones en Java	38
2.5.1. Captura y gestión de excepciones	38
2.5.2. Gestión diferenciada de distintos tipos de excepciones	40
2.5.3. Declaración de excepciones lanzadas por un método de clase	42
2.5.4. Excepciones, inicialización y liberación de recursos: bloque <i>finally</i> y <i>try</i> con recursos	43
2.6. Formas de acceso a los ficheros	44
2.7. Operaciones sobre ficheros con Java	45
2.7.1. Operaciones de lectura	45
2.7.2. Operaciones de escritura	46
2.8. Acceso secuencial a ficheros en Java	46
2.8.1. Clases relacionadas con flujos de datos	46
2.8.2. Clases para recodificación	49
2.8.3. Clases para <i>buffering</i>	50
2.8.4. Operaciones de lectura para flujos de entrada	51
2.8.5. Operaciones de escritura para flujos de salida	54
2.9. Operaciones con ficheros de acceso aleatorio en Java	57
2.10. Organizaciones de ficheros	61
2.10.1. Organización secuencial	61
2.10.2. Organización secuencial Indexada	62
Resumen	63
Ejercicios propuestos	64
Actividades de autoevaluación	65
 3. BASES DE DATOS RELACIONALES	 69
Objetivos	69
Mapa conceptual	70
Glosario	70
3.1. Conectores	71
3.2. Conectores para bases de datos relacionales	71
3.3. Acceso a resultados de consultas sobre bases de datos relacionales mediante conectores	73
3.4. Desfase objeto-relacional	74
3.5. Java Database Connectivity	74
3.6. Operaciones básicas con JDBC	75
3.6.1. Apertura y cierre de conexiones	75
3.6.2. La interfaz <i>Statement</i>	77
3.6.3. Ejecución de sentencias de DDL	78
3.6.4. Ejecución de sentencias para modificar contenidos de la base de datos	79
3.6.5. Ejecución de consultas y manejo de <i>ResultSet</i>	80
3.7. Sentencias preparadas	83

3.8. Transacciones.....	85
3.9. Valores de claves autogeneradas.....	88
3.10. Llamadas a procedimientos y funciones almacenados.....	91
3.11. Actualizaciones sobre los resultados de una consulta.....	94
3.12. Ejecución de <i>scripts</i>	96
3.13. Ejecución de sentencias por lotes.....	96
Resumen.....	98
Ejercicios propuestos.....	99
Actividades de autoevaluación.....	102
 4. CORRESPONDENCIA OBJETO-RELACIONAL.....	 105
Objetivos.....	105
Mapa conceptual.....	106
Glosario.....	106
4.1. Correspondencia objeto-relacional.....	107
4.2. Hibernate.....	108
4.3. Iniciación a la correspondencia objeto-relacional con Hibernate.....	109
4.4. Correspondencia objeto-relacional a partir de las tablas.....	111
4.4.1. Creación de la conexión con la base de datos.....	113
4.4.2. Creación del proyecto.....	114
4.4.3. Fichero de configuración de Hibernate: <code>hibernate.cfg.xml</code>	115
4.4.4. Fichero de ingeniería inversa <code>hibernate.reveng.xml</code>	116
4.4.5. POJO (clases) y ficheros de correspondencia.....	117
4.4.6. <code>HibernateUtil.java</code>	118
4.5. Descarga y uso de una versión reciente de Hibernate.....	118
4.6. Programa de ejemplo para persistencia de objetos con Hibernate.....	119
4.7. Ficheros hbm o de correspondencia de Hibernate.....	121
4.7.1. Correspondencia para las clases y atributos de clase.....	124
4.7.2. Correspondencia para las relaciones.....	125
4.8. Manejo de relaciones de uno a muchos entre objetos persistentes.....	130
4.9. Manejo de relaciones de uno a uno entre objetos persistentes.....	132
4.10. Sesiones y estados de los objetos persistentes.....	133
4.10.1. De transitorio a persistente con <code>save()</code> , <code>saveOrUpdate()</code> y <code>persist()</code>	135
4.10.2. Obtención de un objeto persistente con <code>get()</code> y <code>load()</code>	136
4.10.3. De persistente a eliminado con <code>delete()</code>	137
4.10.4. De persistente a separado con <code>evict()</code> , <code>close()</code> y <code>clear()</code>	137
4.10.5. De separado a persistente con <code>update()</code> , <code>saveOrUpdate()</code> , <code>lock()</code> y <code>merge()</code>	137
4.11. Lenguajes de consulta HQL y JPQL.....	138
4.11.1. La interfaz <code>Query</code>	139
4.12. Correspondencia de la herencia.....	144
4.12.1. Eliminación de subtipos (una tabla para la jerarquía).....	148
4.12.2. Una tabla por subclase (eliminación de la jerarquía).....	151
4.13. Consultas con SQL.....	152
Resumen.....	153
Ejercicios propuestos.....	154
Actividades de autoevaluación.....	158

5. BASES DE DATOS DE OBJETOS Y OBJETO-RELACIONALES	161
Objetivos	161
Mapa conceptual	162
Glosario	162
5.1. Bases de datos de objetos y objeto-relacionales, y correspondencia objeto-relacional	163
5.2. Características de las bases de datos de objetos	163
5.3. El estándar ODMG	164
5.4. ODL	165
5.4.1. Modelo de objetos de ODL	165
5.4.2. Clases e interfaces	166
5.4.3. Relaciones	167
5.5. OQL	168
5.6. Consulta y manipulación de datos con el Java binding	170
5.7. La base de datos de objetos Matisse	170
5.7.1. Creación de una base de datos mediante ODL con Matisse	171
5.7.2. Utilización de la base de datos mediante el Java binding de Matisse	174
5.7.3. Consultas mediante el SQL de Matisse	181
5.8. SQL:99 y bases de datos objeto-relacionales	183
5.9. Características objeto-relacionales de Oracle	183
5.9.1. Tipos de objetos	184
5.9.2. Herencia	184
5.9.3. Objetos de fila y objetos de columna	185
5.9.4. Tipos de objetos con referencias a otros tipos de objetos	186
5.9.5. Tipos de datos de colección: VARRAY y tablas anidadas	186
Resumen	188
Ejercicios propuestos	188
Actividades de autoevaluación	189
 6. XML	 193
Objetivos	193
Mapa conceptual	194
Glosario	194
6.1. El lenguaje XML	195
6.2. Estructura de un documento de XML	195
6.3. DOM	197
6.4. <i>Parsers</i> o analizadores sintácticos, serialización y deserialización	197
6.5. DOM con Java	198
6.5.1. Parsing DOM	200
6.5.2. Creación de documentos con DOM	203
6.5.3. Serialización de documentos DOM	204
6.6. SAX	206
6.7. Validación de documentos de XML	209
6.7.1. Validación con DTD	209
6.7.2. Validación con esquemas de XML	210
6.8. <i>Parsing</i> con validaciones con Java	211
6.9. <i>Binding</i> con JAXB	217
6.9.1. Esquemas de XML para <i>binding</i>	218
6.9.2. Compilador de <i>binding</i>	219

6.9.3. Clases generadas por el compilador de <i>binding</i>	220
6.9.4. <i>Unmarshalling</i> (deserialización) con JAXB	221
6.9.5. <i>Marshalling</i> (serialización) con JAXB	223
6.10. El lenguaje de consulta XPath	225
6.11. El lenguaje XSL	228
Resumen	231
Ejercicios propuestos	232
Actividades de autoevaluación	235
7. BASES DE DATOS DE XML	237
Objetivos	237
Mapa conceptual	238
Glosario	238
7.1. XML como soporte para almacenamiento e intercambio de datos	239
7.2. Alternativas para el almacenamiento de documentos de XML	239
7.3. Almacenamiento de XML en SGBD relacionales	240
7.4. Características de las bases de datos de XML nativas	241
7.5. Gestores comerciales y libres	242
7.6. Instalación y configuración del SGBD de XML nativo eXist	243
7.7. API para gestión de bases de datos nativas de XML	245
7.8. La API XML:DB	245
7.8.1. Establecimiento de conexiones y acceso a servicios con XML:DB	246
7.8.2. Creación y borrado de colecciones con XML:DB	248
7.8.3. Creación y borrado de documentos con XML:DB	249
7.9. El lenguaje XQuery	251
7.9.1. Consultas con XQuery	252
7.9.2. Sentencias de modificación de datos con XQuery	255
7.10. La API XQJ	256
7.10.1. Establecimiento de conexiones con XQJ	257
7.10.2. Consultas con XQJ	260
7.10.3. Modificaciones de documentos con XQJ	261
7.10.4. Transacciones con XQJ	262
Resumen	264
Ejercicios propuestos	265
Actividades de autoevaluación	267
8. COMPONENTES PARA EL ACCESO A DATOS	271
Objetivos	271
Mapa conceptual	272
Glosario	272
8.1. Componentes de software	273
8.2. Modelos de componentes	273
8.3. La plataforma Java: Java SE y Java EE	274
8.4. JavaBeans	275
8.5. El modelo MVC para desarrollo de aplicaciones web con Java	276
8.6. JSP (JavaServer Pages)	277
8.6.1. Directivas de JSP	278
8.6.2. Scriptlets	278

8.6.3.	Variables implícitas	278
8.6.4.	Referencias a variables de Java	279
8.7.	Servlets	279
8.8.	Desarrollo de una aplicación web MVC basada en JavaBeans	280
8.8.1.	Creación de la aplicación web	280
8.8.2.	Persistencia de objetos con Hibernate	281
8.8.3.	Creación del servlet controlador	281
8.8.4.	Uso de JavaBeans asociado a formularios HTML con JSP	284
8.8.5.	Creación de los JSP	284
8.9.	Enterprise JavaBeans	289
8.10.	Desarrollo de una aplicación web MVC para Java EE con EJB	289
8.10.1.	Creación de la aplicación web	290
8.10.2.	Creación de la unidad de persistencia	290
8.10.3.	Creación de las clases de entidad	291
8.10.4.	Creación de los EJB de sesión para las clases de entidad	292
8.10.5.	Creación del servlet controlador	295
8.10.6.	Configuración básica inicial de la aplicación web	297
8.10.7.	Creación de los JSP	297
8.10.8.	Incluir los ficheros jar de una versión reciente de Hibernate	301
8.10.9.	Despliegue de la aplicación	301
8.10.10.	Solución de errores adicionales en tiempo de ejecución	303
	Resumen	304
	Ejercicios propuestos	304
	Actividades de autoevaluación	306

Presentación

Este libro se plantea como una completa introducción a la persistencia de datos en diversos soportes: ficheros, bases de datos relacionales, bases de datos de objetos y XML, tanto documentos individuales como bases de datos de XML. Va dirigido en especial al alumnado del ciclo superior de Desarrollo de Aplicaciones Multiplataforma que cursan el módulo Acceso a Datos. Pero es de interés también para estudiantes universitarios o para profesionales interesados en estas tecnologías.

Por encima de las particularidades de cada uno de estos tipos de sistemas diversos para persistencia de datos, existen aspectos comunes que sirven de hilo conductor y permiten un tratamiento unificado y coherente de todos ellos, de manera que este libro sea algo más que una exposición secuencial de tecnologías, técnicas y herramientas diversas, y haga hincapié en los aspectos fundamentales y, en gran medida, comunes. Los lenguajes de definición de datos para definir un esquema de almacenamiento apropiado y efectivo, la introducción de restricciones de integridad para evitar redundancias, inconsistencias e incorrecciones en los datos, los lenguajes de consulta para obtener la información que en cada momento o para cada aplicación se necesite, los mecanismos de iteración para obtener los resultados de las consultas realizadas, y las transacciones para garantizar la consistencia e integridad de los datos cuando se realizan modificaciones en ellos. Todos estos aspectos se cubren para cada uno de los tipos de sistemas para persistencia de datos.

Aparte de eso, se dedica un primer capítulo de introducción a proporcionar una perspectiva general y una breve introducción a estos aspectos fundamentales y recurrentes. La persistencia de datos es un campo muy amplio y diverso, pero en resumen consiste en utilizar programas que pueden almacenar y recuperar datos en sistemas de almacenamiento de datos. Sugerido así, y teniendo en cuenta el planteamiento eminentemente práctico y pragmático de este libro, tienen una importancia capital los lenguajes orientados a objetos y las bases de datos relacionales. Estas últimas tienen, desde hace décadas (desde los años ochenta del siglo *xx*), un predominio incuestionable como sistemas de almacenamiento de datos. Se asume un conocimiento básico

por parte del lector de ambos, y en particular de Java para programación de aplicaciones y del lenguaje SQL para manejo de bases de datos relacionales. En consonancia con esto, se dedica mucha atención a ORM, la correspondencia objeto-relacional, y a otros planteamientos existentes para el almacenamiento en bases de datos relacionales de objetos y de XML, facilitados además por la inclusión en sucesivas revisiones del estándar SQL de soporte para objetos y para documentos de XML. El planteamiento es eminentemente práctico, pero sin renunciar a una consistente y sólida base teórica.

Cada capítulo comienza con una breve introducción histórica y una explicación de la importancia actual del planteamiento en cuestión para persistencia de datos, lo que permite hacerse una idea propia de la importancia que es de esperar que tengan en el futuro. Después, se explican brevemente los estándares disponibles y los lenguajes para definición, consulta y modificación de datos, y las API que permiten utilizarlos desde programas de aplicación. Se presentan programas de ejemplo que ilustran todos los aspectos teóricos planteados. Los programas de ejemplo suelen desarrollarse siempre sobre varios escenarios comunes que se plantean en todos los temas. Se plantea, pues, la persistencia del mismo modelo conceptual con los distintos sistemas y con los esquemas de almacenamiento que cada uno proporciona (fundamentalmente el relacional, de objetos y de XML). Con ello se persigue fomentar el espíritu crítico y la propia capacidad para evaluar la aplicabilidad de distintos planteamientos en persistencia de datos con problemas concretos, tras analizar objetivamente las ventajas e inconvenientes de cada uno.

También se intercalan actividades, que en general consisten en la realización de programas similares a —o basados en, o ampliaciones de— los programas de ejemplo. El objetivo es fomentar la capacidad de comprender programas ya existentes, como paso previo para modificarlos según las propias necesidades. Para ello es necesario consultar y buscar información, formular hipótesis y experimentar diversos planteamientos para la resolución de problemas concretos. Todas estas son habilidades fundamentales en el desempeño profesional en el sector del desarrollo de aplicaciones. La capacidad de investigar de forma autónoma y de evaluar críticamente diversas soluciones alternativas, una vez asimilados los conceptos teóricos sobre los que se fundamentan, hace posible la necesaria actualización constante de conocimientos y habilidades en un campo como el desarrollo de aplicaciones, en el que es patente que todo cambia continuamente, pero en el que, a la vez, y aunque muchas veces no parezca tan evidente, los principios fundamentales son, en gran medida, y en lo primordial, siempre los mismos, y en el que es precisamente un entendimiento de estos conceptos fundamentales lo que permite comprender los continuos cambios que se suceden y formarse una idea propia de su importancia e implicaciones reales.

En cada capítulo se añade un resumen y unas actividades de autoevaluación, que permiten hacerse una idea del grado de asimilación de los conceptos teóricos fundamentales, así como una lista de ejercicios que requieren un mayor grado de reflexión y elaboración previa, y en ocasiones de investigación, presentados en orden creciente de dificultad.

Además, en la página web de la editorial (www.sintesis.com) se incluye un archivo con los programas de ejemplo y varios recursos adicionales para el libro. Estos contenidos están especificados en recuadros llamados *Recurso digital* dentro del libro.

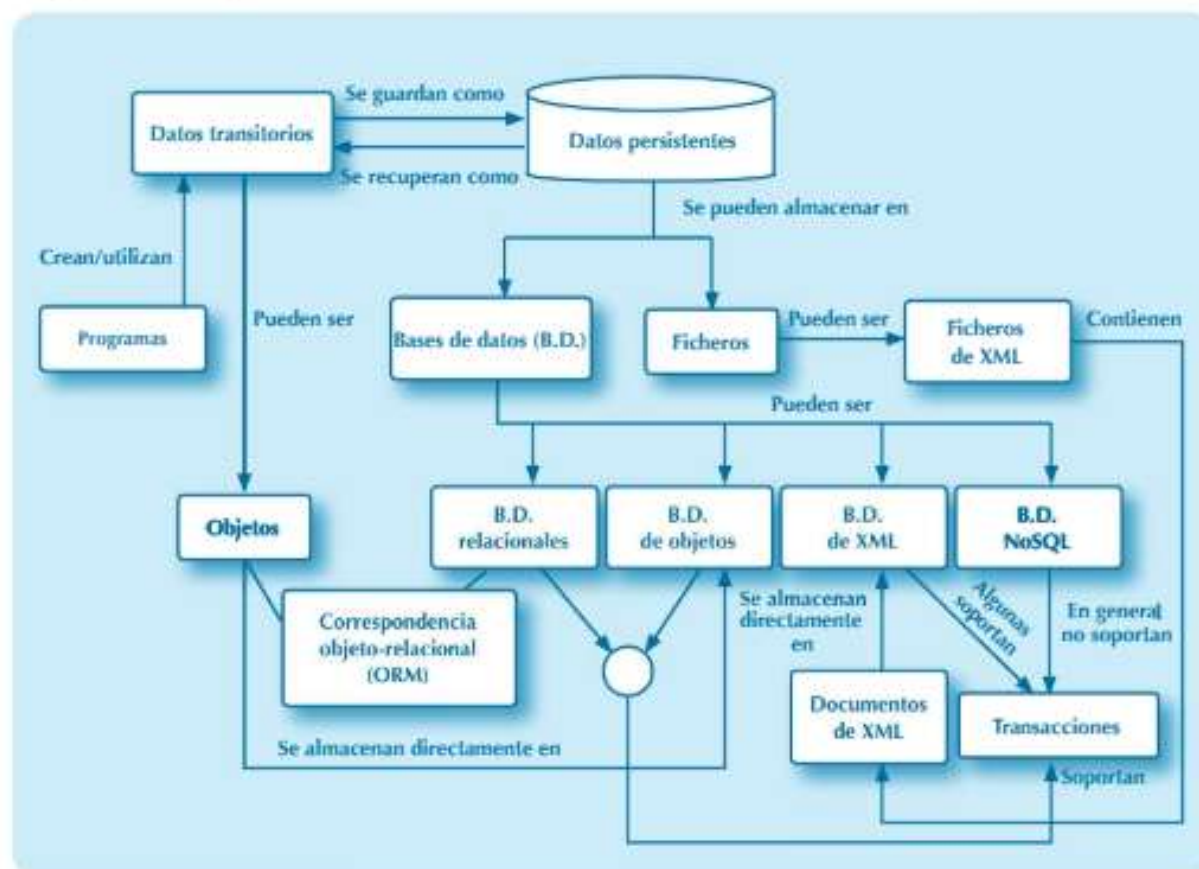
Dedico este libro a mis padres, por todas las oportunidades que me han dado y por hacer posible este libro con su esfuerzo y cariño. A Ana, que siempre me ayuda a encontrar el camino. Y a Celia, cuya sonrisa ilumina el mundo.

Introducción y conceptos básicos

Objetivos

- ✓ Proporcionar una breve introducción a los contenidos del libro.
- ✓ Realizar un breve repaso a conceptos preliminares, algunos probablemente ya conocidos.
- ✓ Introducir el concepto de *persistencia de datos*, los medios de almacenamiento que sirven de soporte para datos persistentes, y los mecanismos que pueden utilizar los programas de aplicación para trabajar de manera efectiva con datos persistentes.

Mapa conceptual



Glosario

API (Application Programming Interface). Interfaz de programación de aplicaciones. Conjunto de funciones disponibles para su uso por parte de programas de aplicaciones, y que estos pueden utilizar para acceder a determinados servicios.

Base de datos de objetos o base de datos orientada a objetos. Base de datos que almacena directamente objetos, en el sentido que tiene el término *objeto* en los lenguajes de programación orientados a objetos.

Base de datos de XML. Base de datos que almacena documentos en formato XML.

Base de datos NoSQL. Base de datos que no es de ninguno de los tipos anteriores, que normalmente tiene almacenamiento basado en estructuras básicas muy sencillas y flexibles, y que prima la disponibilidad y la eficiencia sobre el soporte de transacciones. Con frecuencia en este grupo se incluyen las bases de datos de XML.

Base de datos relacional. Base de datos que representa los datos y las relaciones entre ellos de acuerdo al modelo relacional, y que utiliza tablas como estructuras básicas para el almacenamiento de los datos.

Desfase objeto-relacional. Conjunto de dificultades que plantea la persistencia de objetos en las bases de datos relacionales.

Fichero. Secuencia de *bytes* almacenada en un medio de almacenamiento secundario, a la que se puede acceder indicando su nombre y su ubicación dentro de una jerarquía de directorios existente en dicho medio.

Persistencia de datos. Paso de datos de memoria principal a un medio de almacenamiento secundario, de manera que más adelante se puedan recuperar en memoria principal.

Transacción. Secuencia de operaciones de consulta y modificación de datos persistentes realizadas por un programa, que se realizan como un todo, y de manera aislada con cualquier otra operación sobre los mismos datos que pudiera realizar cualquier otro programa.

XML (*eXtensible Markup Language*). Lenguaje que tiene una sintaxis muy sencilla y que permite la representación de cualquier tipo de información.

1.1. Programas y datos

Se puede decir, a grandes rasgos, que los ordenadores ejecutan programas que gestionan información. La información se representa en forma de datos que pueden estar estructurados de distintas formas.

El concepto de *ordenador* es muy amplio. Hoy en día incluye no solo los servidores, ordenadores de sobremesa y portátiles tradicionales, sino también los teléfonos móviles, tabletas, televisores inteligentes, etc., sumándose cada día más dispositivos.

Los tipos de datos que manejan y los tipos de operaciones que realizan con ellos son muy diversos. Pero siempre cuentan con dos tipos de medios de almacenamiento de datos:

1. *Almacenamiento primario o memoria principal:* donde se almacenan los datos con los que en cada momento está trabajando el programa. Sus contenidos se borran cuando se apaga el ordenador. Tiene una capacidad relativamente baja y el tiempo de acceso a los datos es muy corto.
2. *Almacenamiento secundario:* donde se almacenan datos de manera permanente. Medios de almacenamiento secundario son discos duros, memorias *flash* de distintos tipos (*pen drives* o tarjetas de memoria), etc. Los datos que se almacenan en estos medios son *datos persistentes*, no se borran cuando se apaga el ordenador. Tienen una capacidad de almacenamiento relativamente alta y el tiempo de acceso a los datos es relativamente largo.

1.2. Persistencia de datos

Los programas solo pueden consultar directamente datos almacenados en memoria principal. Un programa puede crear datos en memoria principal y guardarlos en almacenamiento secundario. De esa manera, estos datos se convierten en datos persistentes. Y a la inversa, un programa puede recuperar datos persistentes desde un medio de almacenamiento secundario para pasarlos

a memoria principal. Una vez ahí se convierten en datos transitorios que el programa puede consultar, modificar y, llegado el caso, volver a almacenar en almacenamiento secundario, es decir, volver a hacer persistentes.

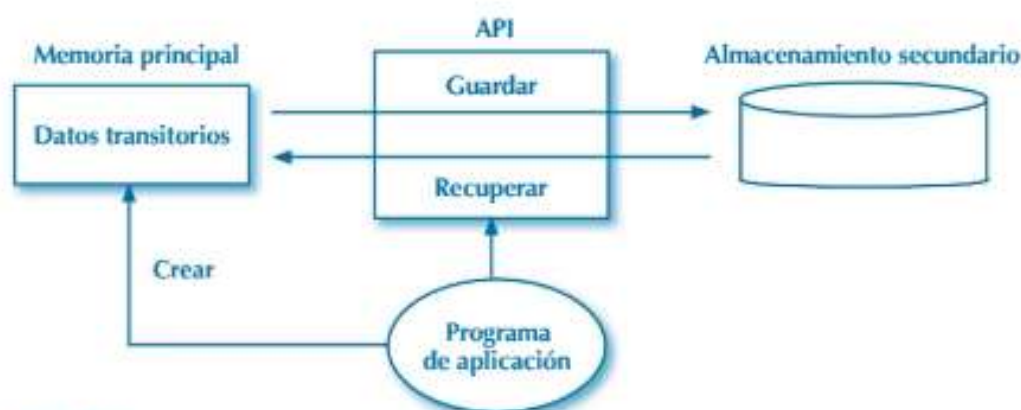


Figura 1.1
Persistencia de datos

Es muy habitual que el medio de almacenamiento secundario esté en un ordenador distinto a aquel en que se ejecuta el programa que los utiliza. Puede ser un servidor que proporciona servicios de persistencia de datos a múltiples aplicaciones que se ejecutan en distintos ordenadores y que se comunican con el servidor mediante protocolos estándares de red. Los programas de aplicación suelen acceder a los servicios para almacenamiento y consulta de datos persistentes mediante funciones de alto nivel proporcionadas por diversas API (interfaces de programación de aplicaciones).

1.3. Sistemas de persistencia de datos

En última instancia, los datos persistentes se guardan siempre en ficheros dentro de un sistema de ficheros en un medio de almacenamiento secundario. Un sistema de ficheros consiste en una jerarquía de directorios o carpetas, y en cada carpeta de esta jerarquía puede haber ficheros que consisten básicamente en una secuencia de *bytes*.

Sobre este sistema básico de almacenamiento se pueden organizar sistemas de almacenamiento de datos más o menos sofisticados.

Dirección: **X** /var/lib/mysql

Directorios	Nombre	Tamaño
mysql	..	
mysql	cb_synt_corpus	
mysql	mysql	
mysql	performance_schema	
mysql	phpmyadmin	
mysql	sys	
mysql-files	auto.cnf	56 bytes
mysql-keyring	debian-5.5.flag	0 bytes
mysql-upgrade	debian-5.7.flag	0 bytes
NetworkManager	ib_buffer_pool	893 bytes
nssdb	ib_logfile0	50.3 MB
ntpdate	ib_logfile1	50.3 MB
ofono	ibdata1	15.6 GB
openvpn	ibtmp1	12.6 MB
os-prober	mysql_upgrade_info	6 bytes
pam		
php		

Figura 1.2
Contenidos de un sistema de ficheros

dos. De hecho, en la figura 1.2 se pueden ver los ficheros en los que se almacenan los datos de una base de datos de MySQL.

En este capítulo se hace una revisión general de los principales sistemas disponibles en la actualidad para persistencia de datos. En los siguientes capítulos se explicarán con más detalle. Cada sistema puede ser más o menos apropiado y tendrá sus ventajas e inconvenientes para cada aplicación. Hay que tenerlas en cuenta y evaluarlas críticamente para decidir la mejor solución.

En los siguientes apartados se describirán varios aspectos fundamentales de los sistemas de persistencia de datos. Por último, se describirán brevemente los tipos más importantes.

TOMA NOTA



No solo hay que tener en cuenta las necesidades actuales, sino también las futuras. Un aspecto muy importante es la escalabilidad, es decir, la posibilidad de hacer frente a mayores volúmenes de datos y cargas de trabajo. También son importantes aspectos como la estandarización, el grado de madurez o consolidación, y si se puede esperar soporte y esfuerzo de desarrollo en el futuro para las tecnologías y productos utilizados.

1.4. Almacenamiento de datos

Cada sistema proporciona una estructura básica de almacenamiento para los datos.

1.4.1. Ficheros

Los ficheros proporcionan por sí mismos una organización secuencial de los datos. Un fichero no es otra cosa que una secuencia de *bytes*.

Sobre este soporte elemental se puede representar cualquier tipo de información. De hecho, como ya se ha comentado, todos los sistemas de almacenamiento de datos almacenan los datos en ficheros. Pero cuando se

habla de sistemas basados en ficheros, se suele hacer referencia a un tipo particular ampliamente utilizado hasta la irrupción de las bases de datos relacionales en los años ochenta. En este tipo, la

información dentro de cada fichero se almacena en una secuencia de registros de longitud fija, y cada registro está compuesto por varios campos de longitud fija. Por ejemplo, un registro podría tener los datos de un cliente, almacenados en campos tales como DNI, nombre, dirección, etc. A un fichero así se le llama *fichero secuencial*, porque está formado por una secuencia de registros.

1100	0001	0101	0001		0001	0001
1011	0110	0110	0000		1110	0110

Figura 1.3
Fichero

89012345E ROJAS	23456789D DORCE	12345678Z ARCOS	56789012B SAMPER	78901234X NADALES	45678901G JUÁREZ	67890123C GAMBOA	34567890V NADAL
--------------------	--------------------	--------------------	---------------------	----------------------	---------------------	---------------------	--------------------

Figura 1.4
Fichero secuencial

Para acelerar las consultas sobre un fichero secuencial se pueden crear ficheros de índice, que permiten acceder a sus contenidos en un orden determinado. Para el fichero secuencial anterior se podría crear un fichero de índice para el campo *nombre*. Al existir un índice para él, pasa a ser un fichero secuencial indexado. Nada impide crear tantos índices como se quiera sobre un fichero secuencial. Nótese que un fichero de índice no deja de ser un tipo particular de fichero secuencial que incluye un solo campo del fichero, con sus registros ordenados por ese campo, y un campo que indica la posición en el fichero secuencial.

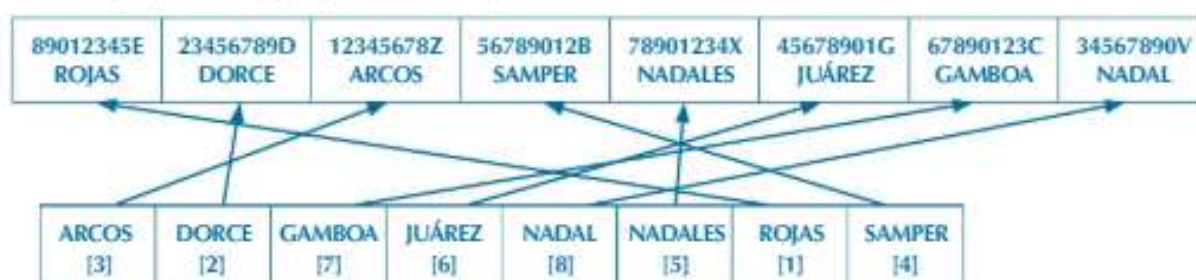


Figura 1.5
Fichero secuencial indexado

1.4.2. Bases de datos relacionales

Las bases de datos relacionales organizan los datos en tablas, y permiten especificar las relaciones entre dichas tablas.

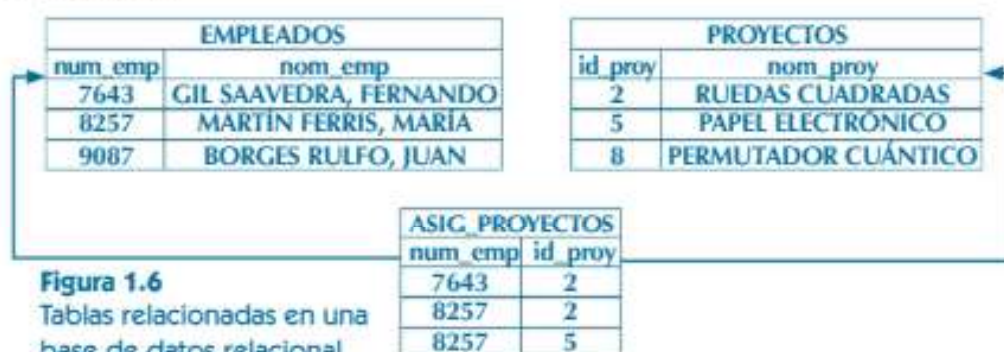


Figura 1.6
Tablas relacionadas en una
base de datos relacional

Hoy en día es abrumador el predominio de las bases de datos relacionales, que en los años ochenta desplazaron a los antiguos sistemas basados en ficheros. SQL es el lenguaje estándar para bases de datos relacionales. Por ello, frecuentemente, se denomina *base de datos NoSQL* a cualquier base de datos no relacional. Dentro de esta categoría, de todas formas, cabe diferenciar algunos tipos particulares, como las bases de datos de XML y las bases de datos de objetos. Excluyendo estos tipos quedan diversas bases de datos, muy diferentes entre sí, pero que comparten algunas características generales y que, además, desde hace un tiempo, se vienen utilizando cada vez más. También se usa con frecuencia el término NoSQL, con un sentido más restrictivo, para referirse a ellas. Y es este último el sentido con el que se utilizará, en general, en este libro.

1.4.3. Documentos de XML

En los documentos de XML la información se organiza de manera jerárquica, es decir, en forma de árbol. El modelo DOM es un modelo estándar para representar los contenidos de un documento XML como una jerarquía de nodos.



Figura 1.7

Fichero XML y organización jerárquica de los datos que contiene (modelo DOM)

Un documento de XML se puede almacenar en un fichero de texto. Una colección de ficheros de XML se puede organizar en una jerarquía de directorios dentro de un sistema de ficheros, junto con ficheros de otros tipos. Pero también se puede almacenar en una base de datos de XML. Las bases de datos de XML organizan los datos en una jerarquía de colecciones, y cada colección puede contener múltiples documentos de tipo XML, e incluso a veces de otros tipos.

1.4.4. Bases de datos de objetos

Las bases de datos de objetos almacenan objetos. Un objeto complejo puede incluir referencias a otros objetos y a colecciones de objetos relacionados con él. Una colección de objetos complejos tiene estructura de grafo.

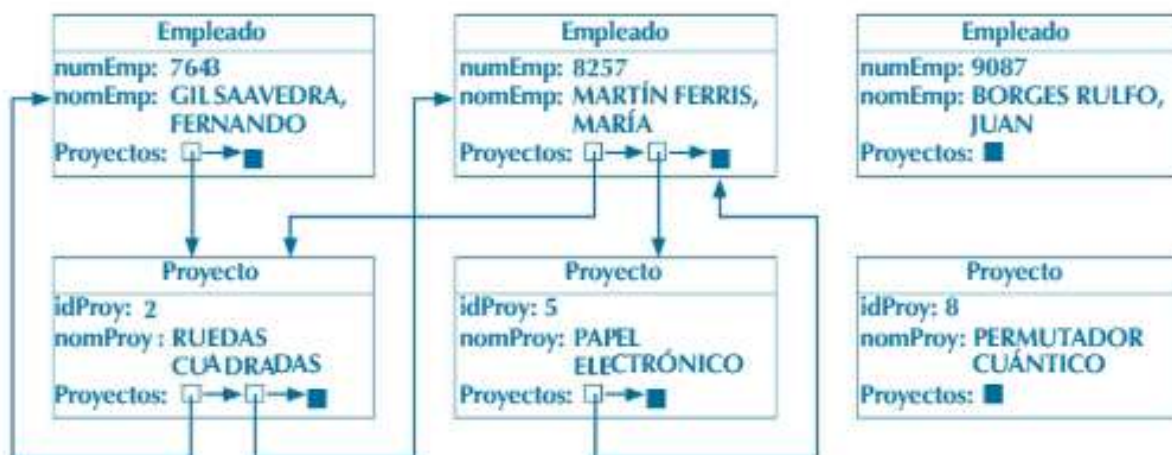


Figura 1.8. Representación de una colección de objetos complejos en forma de grafo

1.4.5. Bases de datos NoSQL

Las diversas bases de datos NoSQL representan la información de manera muy diversa y, en general, diferente de todos los modelos vistos hasta ahora. Pero en general utilizan estructuras muy sencillas y a la vez muy flexibles.

1.5. Restricciones de integridad

Las restricciones de integridad son condiciones que siempre deben cumplir los datos almacenados. Se pueden definir para datos particulares (por ejemplo, no se permite que el nombre de un cliente no esté indicado), y para datos relacionados (por ejemplo, no se permite que el cliente para una factura no exista, y eso significa que esté indicado en la factura, y además que exista como entidad independiente, por supuesto cumpliendo las restricciones de integridad que se hayan especificado para los clientes, como que el nombre esté indicado). Si el resultado de una modificación sobre los datos viola alguna restricción de integridad, no se permitirá. Por ejemplo, no se permitirá dar de alta un cliente si no se indica su nombre, y no se permitirá borrar un cliente para el que exista alguna factura.

1.6. Acceso a los datos con iteradores

Se utilizan iteradores (a veces también se les llama *cursores*) para obtener los resultados de consultas a diversos sistemas de persistencia de datos. En algunos casos es posible utilizar los iteradores no solo para consultar los datos, sino también para modificarlos.

Pasar a memoria principal todos los resultados de una consulta de una vez no es conveniente en general, porque podrían ser muchos. Un iterador se crea para una consulta y permite obtener uno a uno sus resultados. En el caso de bases de datos relacionales, por ejemplo, un iterador permite obtener los resultados fila a fila; en el caso de bases de datos de XML, nodo a nodo; y en el caso de bases de datos de objetos, objeto a objeto. Como las operaciones de recuperación de datos desde almacenamiento secundario son relativamente lentas, se suelen recuperar bloques de datos en cada una, aunque después el iterador los proporcione uno a uno.

1.7. Control de accesos concurrentes y transacciones

Los datos almacenados pueden compartirse entre múltiples aplicaciones, y estas pueden realizar lecturas y modificaciones concurrentes (simultáneas) sobre ellos. Por ello, debe arbitrarse el acceso a los datos para evitar problemas que puedan surgir cuando distintos programas realizan consultas y modificaciones simultáneas sobre los mismos datos. Es además muy habitual que un grupo de operaciones sobre datos relacionados formen un todo que debe llevarse a cabo conjuntamente y de manera aislada con respecto a otras operaciones simultáneas sobre esos datos. En ese caso se consideran una *transacción*.

Ejemplo

Si se hace una transferencia de dinero de una cuenta a otra, el importe se debe restar del saldo de la cuenta de origen y sumar al de la cuenta de destino. Si, por la razón que fuera, no se pudiera sumar el saldo a la cuenta de destino, no debe hacerse efectiva la resta del saldo a la cuenta de origen, sino que debe deshacerse para que todo quede como al principio. Las dos operaciones de las que consta una transferencia constituyen una transacción.

Además, no se pueden realizar simultáneamente dos transferencias que involucren a la misma cuenta. En todo caso, una de ellas debería esperar a que termine la otra.

Si un programa consulta el saldo de cualquiera de las dos cuentas involucradas en una transferencia que se está llevando a cabo, no obtendrá este saldo hasta que la transferencia haya concluido. Si se completa con éxito, obtendrá el nuevo importe, y si no, el importe antiguo, como si la transferencia nunca se hubiera intentado. Si solicita modificar el importe de cualquiera de las dos cuentas, esta modificación no se hará antes de que la transferencia haya concluido.

La sincronización de operaciones de lectura y escritura y las transacciones aseguran que los programas de aplicación siempre tengan una vista consistente, actualizada y correcta de los datos almacenados.

Una transacción es una secuencia de operaciones de lectura y actualización de datos que forman una unidad lógica y que tienen que ejecutarse como un todo. Las bases de datos relacionales proporcionan, en general, un completo y excelente soporte para transacciones, mientras que las bases de datos NoSQL no proporcionan, en general, soporte para transacciones.

Las características que debe satisfacer una transacción se suelen resumir con el acrónimo ACID (*atomic, consistent, isolated, durable*):

1. *Atomic* (atómica). Una transacción debe ejecutarse completamente y sin errores. Si ocurre algún error, los cambios que haya hecho deben deshacerse, de manera que todo quede como si nunca se hubiera iniciado la transacción.
2. *Consistent* (consistente). Las restricciones de integridad definidas para los datos deben cumplirse tras cada operación incluida en la transacción y también al finalizar esta.
3. *Isolated* (aislada). Una transacción está aislada de otras transacciones que se ejecutan simultáneamente. Las distintas transacciones tienen una visión consistente del conjunto de los datos, y no ven las modificaciones hechas por otras transacciones en curso pero no concluidas. Otra manera de expresar esto mismo es que las transacciones son serializables. En un momento dado puede haber varias transacciones ejecutándose simultáneamente. Pero el resultado final de la ejecución de todas ellas debe ser equivalente a su ejecución secuencial, una detrás de otra, en algún orden determinado, uno cualquiera de entre todos los posibles. Se entiende que alguna o varias de ellas podrían no completarse, lo que, al ser atómicas, equivale a que no se ejecuten en absoluto. Por lo demás, todas las transacciones podrán iniciarse inmediatamente, y no se pondrá ninguna restricción a la ejecución en paralelo de múltiples transacciones, siempre que no entren en conflicto. Por ejemplo, si una transacción en curso consulta un dato que ha sido modificado por otra en curso, quedará en suspenso hasta que esta última haya finalizado. Es posible que se produzca lo que se llama *interbloqueo*, es decir, que dos o más transacciones queden bloqueadas esperando cada una de ellas a que termine alguna de las otras. En este caso, el sistema devolvería un código de error a alguna o a varias de ellas y desharía sus cambios, y las demás podrían así continuar.
4. *Durable* (duradera). Una vez completadas todas las operaciones que forman la transacción (y no antes), se confirman los cambios, con lo que quedan grabados de forma permanente.

Todas las ventajas que proporcionan las transacciones tienen como contrapartida un menor rendimiento, aunque los sistemas gestores de bases de datos relacionales de hoy en día están muy

optimizados y son muy escalables, lo que significa que el rendimiento se puede mejorar añadiendo los recursos de *hardware* y *software* necesarios y realizando cambios en la configuración del sistema.

En cualquier caso, las ventajas que ofrecen las transacciones pueden no ser estrictamente necesarias y no compensar la merma en el rendimiento para algunas aplicaciones. Las bases de datos NoSQL, por ejemplo, renuncian a un soporte completo de transacciones a cambio de una mayor disponibilidad y rendimiento.

1.8. Persistencia de datos en ficheros

Los ficheros son el medio de almacenamiento más elemental. Un fichero es, en esencia, una secuencia de *bytes*, con lo que en principio puede almacenar cualquier tipo de información. Un fichero tiene un nombre y está situado en un directorio determinado dentro de una jerarquía de directorios. En última instancia, todos los sistemas de almacenamiento de datos, por sofisticados que sean, utilizan como medio de almacenamiento los ficheros.

El uso de ficheros sencillos de texto con una organización sencilla puede ser suficiente para algunas aplicaciones. Si las consultas que se vayan a realizar son complejas o requieren relacionar mucha información diversa, será difícil escribir un programa para realizarlas. Si el volumen de datos para manejar es muy grande, o si es necesario realizar con mucha frecuencia operaciones de borrado o modificación de datos, el rendimiento será muy pobre. Permitir que varias aplicaciones realicen a la vez operaciones de consulta y actualización requiere elaborados mecanismos de control de acceso que añaden complejidad al sistema, y mucho más permitir transacciones.

En sistemas de este tipo se evidencian enseguida sus limitaciones intrínsecas para evitar la redundancia e inconsistencia de los datos, así como para definir y preservar restricciones de integridad. Además, no son apropiados para dar servicio a muchas aplicaciones concurrentes, como no sea que las operaciones que realicen sean, en general, solo de lectura y muy ocasionalmente de modificación de datos.

Estos son los motivos por los que, desde hace décadas, han sido relegados para la mayor parte de aplicaciones en favor de las bases de datos relacionales. Antes de ellas, la manera más habitual de almacenar la información era en ficheros indexados, para los que el lenguaje COBOL, aún hoy ampliamente utilizado en determinados ámbitos (por ejemplo, el sector bancario), ha proporcionado desde su creación, en 1959, un excelente soporte.

De todas formas, los ficheros siguen teniendo sus aplicaciones importantes. Los documentos de XML se suelen almacenar en ficheros, muchas veces agrupados en colecciones organizadas sobre una sencilla jerarquía de directorios. Los sistemas de correo electrónico suelen mantener sus datos en ficheros con un formato relativamente sencillo. Y para algunos tipos de procesos es suficiente un programa que lea secuencialmente la información almacenada en un fichero. Los ficheros secuenciales se suelen utilizar para procesos masivos que se ejecutan periódica o puntualmente, y para copias de seguridad.

1.9. Persistencia de datos en bases de datos relacionales

Los sistemas de bases de datos relacionales son, con muchísima diferencia, los más utilizados en la actualidad para el almacenamiento de datos. Su gran fortaleza viene de sus sólidos fundamentos matemáticos, al estar basados en un elegante modelo formal: el modelo relacional. Pero a la vez son sencillos e intuitivos. La estructura básica de almacenamiento que proporcionan es la tabla. Todo se almacena en tablas. Para su uso existe un lenguaje estándar y soportado universalmente: SQL.

Mediante SQL se pueden crear, de manera sencilla, esquemas relacionales, que consisten en una colección de tablas. Se pueden definir restricciones de integridad para tablas individuales y para pares de tablas relacionadas (para evitar, por ejemplo, que se borre un cliente si existen facturas para ese cliente). Los esquemas relacionales normalizados son simples y concisos, y permiten evitar redundancias e inconsistencias de datos, a la vez que se optimiza el espacio de almacenamiento utilizado.

SQL es un lenguaje declarativo, de muy alto nivel, que permite especificar la consulta que se quiere realizar para que sea el propio sistema el que decida la manera más eficiente de realizarla, y saque el máximo partido de los índices disponibles. El propio sistema crea automáticamente algunos índices de acuerdo a las restricciones de integridad definidas. Pero se pueden crear manualmente índices adicionales para optimizar determinados tipos de consultas.

Las actuales bases de datos relacionales son muy escalables, lo que significa que son apropiadas para gestionar bases de datos pequeñas con un pequeño volumen de operaciones, pero también para bases de datos enormes que soportan un continuo y elevado volumen de operaciones simultáneas de consulta y modificación de datos. Tienen un magnífico soporte para transacciones. Tienen mecanismos sofisticados de copia de seguridad y de recuperación que permiten recuperar los datos ante fallos de cualquier tipo.

Existen API para bases de datos relacionales para todos los lenguajes de programación ampliamente utilizados. Por ejemplo, JDBC para el lenguaje Java.



PARA SABER MÁS

Actualmente, las bases de datos relacionales son imbatibles como medio de almacenamiento para la mayoría de las aplicaciones. Relegaron a los sistemas basados en ficheros, y alternativas como bases de datos de objetos o de XML nunca han pasado de tener un uso muy restringido. Solo recientemente ha surgido con frecuencia la necesidad de buscar alternativas para algunas aplicaciones que requieren trabajar con gigantescos volúmenes de datos heterogéneos, lo que se conoce como *big data*. Para ello han surgido multitud de bases de datos alternativas, conocidas en general como *bases de datos NoSQL*.

1.9.1. Persistencia de datos de XML en bases de datos relacionales

XML es un formato muy utilizado actualmente para el intercambio de datos, y las principales bases de datos relacionales se han dotado de diversas funcionalidades y mecanismos para facilitar la persistencia de datos en formato XML. El estándar SQL incluye, desde SQL:2003, SQL/XML. SQL/XML incluye un tipo de datos XML. Permite, además, realizar consultas en SQL tanto sobre tablas como sobre documentos de XML, y a la vez sobre ambos, y puede obtener el resultado en forma de tabla o como XML. Las bases de datos relacionales que implementan SQL/XML se califican como *XML-enabled* o con capacidades para XML.

1.9.2. Persistencia de objetos en bases de datos relacionales

El inconveniente de utilizar el modelo relacional con lenguajes orientados a objetos es que almacenar objetos en un esquema relacional no es en principio sencillo. No desde luego cuando

se trata de objetos complejos. Un objeto complejo puede contener referencias a otros objetos y a colecciones de objetos relacionados con él. Al representar gráficamente una colección de objetos complejos, resulta evidente que tiene estructura de grafo, mientras que el modelo relacional ofrece almacenamiento basado en tablas. El término *desfase objeto-relacional* (en inglés, *object-relational impedance mismatch*), o *desajuste de impedancia objeto-relacional*, se utiliza para hacer referencia al conjunto de problemas que plantea la persistencia de objetos utilizando bases de datos relacionales.



Figura 1.9
Desfase objeto-relacional

Ante esta problemática, han surgido varios planteamientos para una solución, cada uno con sus ventajas e inconvenientes:

1. *Bases de datos objeto-relacionales.* Son bases de datos relacionales con capacidades para gestionar objetos. En SQL:99 se introdujeron tipos estructurados definidos por el usuario, entre ellos los tipos objetos (clases). Las bases de datos relacionales que permiten almacenar objetos se califican como bases de datos objeto-relacionales. Entre ellas cabe destacar Oracle y PostgreSQL.
2. *Correspondencia objeto-relacional.* O, en inglés, ORM (*object-relational mapping*), frecuentemente traducido como *mapeo objeto-relacional*. Es una solución más flexible, con la ventaja, además, de proporcionar soporte para múltiples bases de datos. Existen múltiples herramientas, bibliotecas o *frameworks* para ORM, entre las que cabe destacar Hibernate.

1.10. Persistencia de datos en bases de datos de objetos

Las bases de datos de objetos (o bases de datos orientadas a objetos) permiten almacenar directamente objetos. La persistencia de objetos en bases de datos de objetos es en principio la solución natural frente a soluciones de compromiso, tales como bases de datos objeto-relacionales o correspondencia objeto-relacional (ORM). Existen unas cuantas bases de datos de objetos, entre las que cabe citar Matisse y db4o. Pero hoy por hoy su uso es muy limitado.

En contraposición a la ventaja obvia de permitir almacenar directamente objetos, las bases de datos de objetos tienen como principales inconvenientes:

- La falta de un modelo formal y ampliamente aceptado en el que basarse, a diferencia de las bases de datos relacionales, basadas sobre el modelo relacional.
- La falta de estándares ampliamente adoptados, como en el caso de las bases de datos relacionales con el lenguaje SQL. El grupo ODMG (Object Database Management

Group) se disolvió en 2001 tras publicar la última versión de su estándar, ODMG 3.0. Las distintas bases de datos suelen proporcionar implementaciones limitadas del estándar ODMG, y ofrecer como alternativa práctica lenguajes y mecanismos propios.

Las bases de datos de objetos soportan, en general, transacciones.

1.11. Persistencia de datos en bases de datos de XML nativas

La creciente importancia que ha adquirido XML como medio de representación de datos en múltiples aplicaciones ha hecho que, aparte de soluciones de compromiso como las bases de datos de XML con capacidades para XML o *XML-enabled*, se desarrollen bases de datos de XML nativas. Estas tienen estructuras y mecanismos de almacenamiento de datos diseñados y optimizados específicamente para XML, además de soporte para lenguajes estándares que permiten realizar diversos tipos de operaciones sobre documentos de XML: consulta (XPath y XQuery), actualización (diversas extensiones de XQuery) y transformación (XSL), así como validación (XML Schema y DTD). Estas bases de datos suelen proporcionar implementaciones de API estándares para bases de datos de XML, tales como XML:DB y XQJ (XQuery for Java), que son a estas bases de datos lo que JDBC es a las bases de datos relacionales.

Estas bases de datos cuentan con un amplio cuerpo de estándares sobre el que basarse, pero hay algunos aspectos para los que no existen estándares, y en los que hay importantes diferencias entre unas y otras, en particular la organización de documentos de XML en colecciones y la indexación.

Las bases de datos de XML nativas suelen organizar los documentos en colecciones. Algunas permiten una jerarquía de colecciones. Otras permiten almacenar no solo documentos de XML, sino de otros tipos.

Existen diversas extensiones de XQuery para operaciones de modificación de datos. Una de ellas, XQUF o XQuery Update Facility, es un estándar de W3C, pero surgió cuando ya existían otras propuestas anteriores, que se siguen utilizando ampliamente.

No todas las bases de datos nativas de XML soportan transacciones, ni mucho menos. El soporte para transacciones en este tipo de bases de datos plantea distintos tipos de problemas que en el caso de las bases de datos relacionales, objeto-relacionales o de objetos, porque las bases de datos de XML almacenan documentos enteros, no registros u objetos individuales.



Actividades propuestas

- 1.1.** Haz un esquema con las distintas posibilidades disponibles para almacenar tanto objetos como documentos de XML en diversos tipos de bases de datos, al menos: relacionales, de objetos, objeto-relacionales y de XML nativas. Si existe un nombre específico para la tecnología que hace posible el almacenamiento de un tipo particular de datos en un tipo particular de base de datos, indícalo.
- 1.2.** Para cada tipo de base de datos vista hasta ahora, haz una lista, por una parte, de estándares y, por otra parte, de API, e indica en pocas palabras su propósito o utilidad. Con frecuencia, una API proporciona la implementación de uno o varios estándares. Indica, siempre que se pueda y sea relevante, los estándares con los que se relaciona, se implementa o donde se basa una API. Por ejemplo: la API XQJ (XQuery for Java) es una API que permite ejecutar sentencias del lenguaje estándar XQuery.

1.12. Persistencia de datos en bases de datos NoSQL

Con frecuencia se incluyen, dentro de las bases de datos NoSQL, las bases de datos de objetos y de XML. En este libro, en general, se excluirán de esta categoría.

El auge de las bases de datos NoSQL, en la actualidad, se debe principalmente a la necesidad de recopilar, gestionar y analizar gigantescos conjuntos de datos heterogéneos, que crecen continuamente a una velocidad cada vez mayor. Esto es lo que se conoce como *big data*. A este enorme aumento de la cantidad de información contribuye el *internet de las cosas* o, por sus siglas en inglés, IoT (*internet of things*), es decir, la conexión a internet de un creciente número de dispositivos. Este tipo de bases de datos da también respuesta a las necesidades de aplicaciones que ofrecen servicio, a través de la web, a un enorme y creciente número de usuarios que no solo consultan información, sino que la añaden y la modifican de manera continua.

Las tradicionales bases de datos relacionales no pueden dar respuesta a estas necesidades, ni tampoco las de objetos o las de XML. Por ello han surgido soluciones alternativas muy diversas, conocidas en conjunto como bases de datos NoSQL.

NoSQL se interpreta a veces como *lo que no es SQL*, y a veces como *not only SQL*, es decir, *no solo SQL*, lo que no se limita a la funcionalidad que proporciona SQL. Más bien suele significar lo primero, y las bases de datos NoSQL no dan soporte para SQL ni para nada que se le parezca. En un sentido amplio, según esta interpretación, NoSQL es todo aquello que no se adhiere 100% al modelo relacional y a las características que se dan por sentadas en las bases de datos relacionales, a saber:

1. Almacenamiento basado en tablas.
2. SQL como lenguaje para interactuar con la base de datos.
3. Soporte para restricciones de integridad y transacciones.

Si NoSQL se entiende como todo lo que no es relacional, entonces incluye todo lo que no satisface la primera condición anterior, y dentro de esta categoría podrían incluirse, y a menudo se hace, las bases de objetos y de XML. Pero para estos tipos de bases de datos sí existen lenguajes estándares inspirados en SQL, y muchas veces muy similares. A saber: OQL para bases de datos de objetos y XQuery para bases de datos de XML. Y si no en tablas, si almacenan los datos en estructuras relativamente regulares. Además, las bases de datos de objetos suelen proporcionar buen soporte para transacciones, y también algunas bases de datos de XML. Aquí se utiliza el concepto NoSQL con un sentido más restrictivo, para referirse a bases de datos que no cumplen ninguna de las tres condiciones anteriores. Cada una utiliza un tipo particular de estructuras para el almacenamiento de datos. Cada una tiene su propio lenguaje particular para consulta y modificación de datos. Y no ponen tanto énfasis en estrictas restricciones de integridad ni en todas las características de las transacciones que impone el modelo relacional, con el objetivo fundamental de primar la disponibilidad, es decir, que la base de datos pueda dar respuesta lo antes posible a peticiones de consulta y actualización de datos.

Algunas características comunes a estas bases de datos son:

- El almacenamiento no está basado en tablas. En general, utilizan estructuras de almacenamiento muy sencillas, pero, sobre todo, muy flexibles. Pueden ser *arrays* asociativos o de tipo clave-valor (como Redis), o documentos (como MongoDB), o basadas en columnas (como Cassandra), o basadas en grafos, como otras bases de datos, por poner solo algunos ejemplos.

- No se manejan con SQL ni nada que se le parezca. Cada una tiene su propio lenguaje, en general no declarativo. SQL es un lenguaje declarativo, es decir, que especifica el resultado que se quiere obtener, pero no los pasos a seguir para obtenerlo. Estos lenguajes están pensados para el tipo particular de estructuras utilizadas para el almacenamiento de los datos. Para consultas o modificaciones no triviales, suele ser necesario escribir programas con instrucciones muy detalladas.
- En cuanto a restricciones de integridad y transacciones, se presta a confusión razonar sobre estas bases de datos utilizando términos y conceptos que habitualmente se aplican al modelo relacional. En resumen, se puede decir que, en lugar de primar las características de las transacciones ACID (atomicidad, consistencia, aislamiento y durabilidad), prima la disponibilidad, lo que a veces se resume con el acrónimo BASE (*basic availability, soft state, eventual consistency*), es decir: disponibilidad básica, estado flexible y consistencia con el tiempo. *Soft state* significa que, aun sin recibir nuevas peticiones de actualización, el valor de algunos datos puede cambiar, debido a peticiones de cambios recibidos pero no llevadas a cabo. Consistencia con el tiempo significa que, pasado un periodo de tiempo, en general corto, sin que se reciban nuevas peticiones de actualización, el valor de un dato tomará un valor definitivo. Por cierto, ya que se ha contrapuesto el tradicional planteamiento ACID de los sistemas relacionales al planteamiento BASE de los nuevos sistemas NoSQL, hay que recalcar que el sentido de *consistency* en ACID y en BASE (*eventual consistency*) es muy diferente.



Actividad propuesta 1.3

Haz un cuadro resumen en forma de tabla incluyendo filas para los distintos tipos de sistemas de persistencia de datos, y columnas para diversos aspectos relevantes tales como: estructura básica para almacenamiento de datos, soporte para transacciones, grado de estandarización (es decir, existencia de estándares establecidos y ampliamente adoptados), madurez (es decir, si las tecnologías se han venido utilizando durante un periodo largo de tiempo, aunque no sea de manera muy extendida, con lo que ha habido tiempo para corregir errores, perfilar características, hacer ajustes, optimizar, adaptar a nuevas situaciones), grado de implantación (uso extendido en la actualidad), y cualquier otro que consideres relevante. Se trata de dar una idea aproximada y muy sintética, con lo que para algunos de estos criterios basta indicar posibles valores como "alta", "media", "baja", o bien "siempre", "a veces".

Resumen

- Los programas de aplicación trabajan con datos transitorios almacenados en memoria principal, y con datos persistentes almacenados en medios de almacenamiento secundario.

- Existen diversos sistemas de persistencia de datos que permiten a los programas de aplicación grabar datos transitorios como datos persistentes, y recuperar datos persistentes en memoria principal como datos transitorios.
- Los sistemas de persistencia de datos suelen funcionar en servidores que proporcionan servicios a las aplicaciones a través de protocolos estándares de red. Las aplicaciones suelen utilizar estos servicios a través de API (interfaces de programación de aplicaciones).
- Hay distintos tipos de sistemas de persistencia de datos. Entre ellos los basados en ficheros, las bases de datos relacionales y objeto-relacionales, las bases de datos de objetos, las bases de datos de XML y de otros tipos, denominadas en conjunto bases de datos NoSQL. Las bases de datos de XML se suelen considerar con frecuencia como NoSQL.
- Cada tipo de sistema proporciona una estructura básica para el almacenamiento de datos: las bases de datos relacionales, tablas; las bases de datos de XML, documentos XML con estructura jerárquica (en forma de árbol); las bases de datos de objetos, objetos complejos, con referencias a otros objetos y a colecciones de objetos; las diversas bases de datos NoSQL, diversas estructuras; y los ficheros son simples secuencias de bytes.
- Las API que permiten a los programas de aplicación acceder a datos persistentes suelen proporcionar iteradores para obtener uno a uno los resultados de las consultas.
- Los sistemas de persistencia de datos suelen proporcionar control y sincronización de accesos concurrentes y transacciones. Las bases de datos relacionales suelen tener un excelente soporte para transacciones. Las bases de datos NoSQL –sin incluir entre ellas las bases de datos de objetos ni las de XML– no suelen ofrecer soporte para transacciones porque prima la disponibilidad sobre la consistencia e integridad de los datos.
- Las bases de datos relacionales son, con diferencia, las más utilizadas. Están basadas en un sólido modelo formal: el modelo relacional. Se manejan mediante SQL, un lenguaje estándar declarativo y de muy alto nivel. Suelen utilizarse incluso para almacenar objetos y documentos de XML, introducidos en sucesivas revisiones del estándar SQL.
- El almacenamiento de objetos en bases de datos relacionales plantea una serie de problemas, conocidos en conjunto como *desfase objeto-relacional*. Pero existen técnicas y herramientas de ORM o correspondencia objeto-relacional que lo hacen posible.
- Las bases de datos de objetos permiten almacenar directamente objetos complejos que contienen referencias a otros objetos y a colecciones de objetos. Muchas se basan en o dan soporte al estándar ODMG 3.0, que incluye los lenguajes ODL para definición de datos y OQL para consulta de datos.
- Las bases de datos de XML nativas suelen organizar los documentos en una jerarquía de colecciones, en la que cada una puede contener documentos XML y de otros tipos. Estas bases de datos proporcionan, además, soporte para lenguajes estándares del W3C, como XQuery, XML Schema y XSL, y para API estándares tales como XML:DB y XQJ.

ACTIVIDADES DE AUTOEVALUACIÓN

1. La persistencia de datos:
 - ☐ a) Se utiliza para datos cuyo valor muy rara vez cambia. El resto se mantiene siempre en memoria principal.
 - ☐ b) Comprende un conjunto de técnicas que permiten transferir datos de memoria principal a almacenamiento secundario y viceversa.
 - ☐ c) Solo es posible para algunos tipos de datos.
 - ☐ d) Permite utilizar siempre las mismas operaciones para manejar los datos, tanto si están en memoria principal como si están en almacenamiento secundario.
2. Los programas de aplicación acceden a datos persistentes:
 - ☐ a) Bien directamente si están almacenados en el mismo ordenador, bien mediante API si están almacenados en otro.
 - ☐ b) Mediante API, que permiten utilizar los servicios de persistencia.
 - ☐ c) Mediante protocolos de red, que dan acceso a los servicios de persistencia.
 - ☐ d) Previo volcado binario en memoria principal desde almacenamiento secundario.
3. Un documento de XML:
 - ☐ a) Se almacena normalmente en varios directorios organizados jerárquicamente.
 - ☐ b) Tiene estructura jerárquica.
 - ☐ c) Al tener estructura jerárquica, no se puede almacenar en una estructura lineal como un fichero.
 - ☐ d) Se puede almacenar en un fichero, pero debe ser necesariamente binario.
4. Las restricciones de integridad:
 - ☐ a) Solo se pueden definir para bases de datos relacionales.
 - ☐ b) Solo se pueden preservar en una base de datos utilizando transacciones.
 - ☐ c) Se pueden definir para bases de datos de cualquier tipo.
 - ☐ d) Se definen para garantizar la consistencia de datos relacionados entre sí.
5. Los iteradores:
 - ☐ a) Permiten recuperar datos persistentes, pero no modificarlos.
 - ☐ b) Solo permiten recorrer estructuras lineales o tabulares. No documentos de XML, por ejemplo.
 - ☐ c) Obtienen los datos siempre uno a uno del almacenamiento secundario.
 - ☐ d) Permiten obtener uno a uno los resultados de una consulta.
6. Una transacción:
 - ☐ a) Podría fallar en caso de interbloqueo, es decir, cuando esa transacción y otras estén paradas esperando a que alguna de ellas termine.
 - ☐ b) Solo terminará con éxito si lo hacen todas las operaciones que incluye.
 - ☐ c) Puede ejecutarse en paralelo con otras transacciones.
 - ☐ d) Todas las respuestas anteriores son correctas.

7. Las bases de datos relacionales:

- ☐ a) Almacenan los datos en tablas.
- ☐ b) Tienen, en general, un excelente soporte para transacciones.
- ☐ c) Se manejan con el lenguaje SQL.
- ☐ d) Todas las respuestas anteriores son correctas.

8. Las bases de datos de objetos:

- ☐ a) Permiten almacenar directamente objetos, pero no objetos complejos.
- ☐ b) No suelen tener soporte para transacciones.
- ☐ c) Son las únicas que permiten almacenar directamente objetos.
- ☐ d) Ninguna de las opciones anteriores es correcta.

9. Las bases de datos de XML nativas:

- ☐ a) Incluyen soporte para colecciones de acuerdo a estándares del W3C.
- ☐ b) Son las únicas que permiten almacenar directamente documentos de XML, a la vez que proporcionan soporte para el estándar XQuery de W3C.
- ☐ c) Suelen tener soporte para transacciones.
- ☐ d) No tienen un lenguaje estándar universalmente adoptado para operaciones de modificación de datos.

10. Las bases de datos NoSQL:

- ☐ a) Priman la disponibilidad sobre la consistencia e integridad de los datos.
- ☐ b) Son importantes para las aplicaciones de *big data*.
- ☐ c) Suelen utilizar estructuras de almacenamiento muy sencillas y flexibles.
- ☐ d) Todas las opciones anteriores son correctas.

SOLUCIONES:

1. ☐ a ☒ b ☐ c ☐ d
 2. ☐ a ☒ b ☐ c ☐ d
 3. ☐ a ☒ b ☐ c ☐ d
 4. ☐ a ☐ b ☒ c ☐ d

5. ☐ a ☐ b ☐ c ☒ d
 6. ☐ a ☐ b ☐ c ☒ d
 7. ☐ a ☐ b ☐ c ☒ d
 8. ☐ a ☐ b ☒ c ☐ d

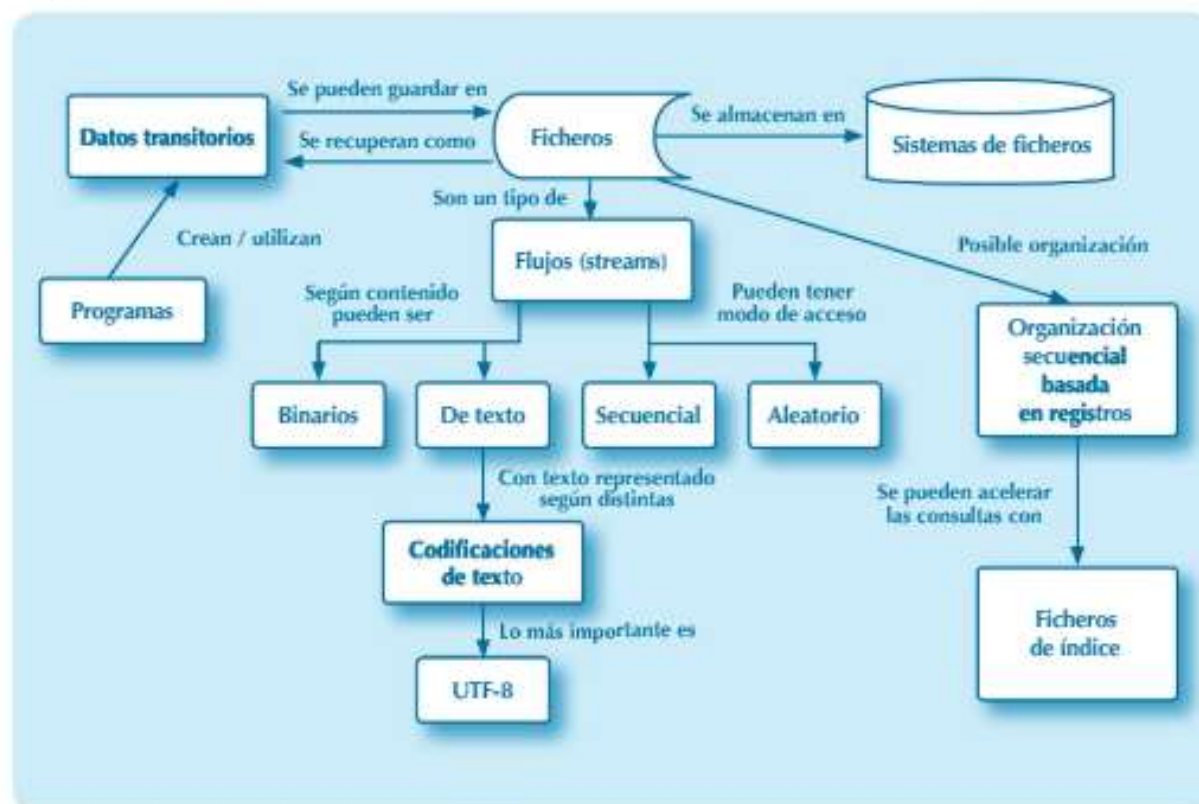
9. ☐ a ☐ b ☐ c ☒ d
 10. ☐ a ☐ b ☐ c ☒ d

Ficheros

Objetivos

- ✓ Conocer las diferencias en la gestión de ficheros de texto y ficheros binarios.
- ✓ Diferenciar entre acceso secuencial y aleatorio a ficheros.
- ✓ Aprender las principales clases de Java para manejo de ficheros (y flujos en general) y su uso.
- ✓ Comprender el mecanismo de *buffering*, cómo permite acelerar las operaciones de lectura y escritura en ficheros tanto binarios como de texto, y cómo permite la lectura y escritura por líneas en ficheros de texto.
- ✓ Acceder correctamente a los contenidos de ficheros de texto con distintas codificaciones.
- ✓ Analizar algunas organizaciones sencillas de ficheros y la manera en que se pueden utilizar para la persistencia de datos.

Mapa conceptual



Glosario

Acceso aleatorio. Tipo de acceso a un fichero que permite acceder directamente a los datos situados en cualquier posición del fichero.

Acceso secuencial. Tipo de acceso a un fichero con el que la única manera de acceder a los datos situados en una posición determinada es leer todos los contenidos desde el principio hasta dicha posición.

Codificación de texto. Una manera particular de representar una secuencia de caracteres de texto mediante una secuencia de bytes.

Fichero de texto. Fichero que contiene texto.

Fichero. Unidad fundamental de almacenamiento. Consiste en una secuencia de bytes. Con una adecuada organización, se puede utilizar para almacenar cualquier tipo de información.

Índice. Fichero que permite recorrer los contenidos de otro fichero en un orden determinado.

Registro. Estructura para representar información. Consta de una serie de campos, en cada uno de los cuales se puede almacenar un dato particular.

2.1. Persistencia de datos en ficheros

Los ficheros son el método de almacenamiento de información más elemental. De hecho, en última instancia, todos los métodos de almacenamiento, por sofisticados que sean, almacenan los datos en ficheros.

Hasta que fueron relegados en los años ochenta por las bases de datos relacionales, los ficheros fueron el principal medio de almacenamiento de datos. El nombre *fichero* se utilizó por analogía con los antiguos ficheros que contenían fichas de papel, todas con la misma estructura, consistente en un conjunto fijo de campos. En el caso de los libros de una biblioteca, por ejemplo, los campos podían ser el título, nombre del autor, tema, etc. Los ficheros que contenían fichas de papel fueron reemplazados por ficheros de ordenador que contenían registros, equivalentes a las antiguas fichas de papel. Un registro contiene un conjunto de campos de longitud fija. Para acelerar las búsquedas se empezaron a utilizar ficheros auxiliares de índice que permitían acceder a los registros según un orden determinado. IBM desarrolló un avanzado sistema de gestión de ficheros llamado ISAM (*indexed sequential access method*). El lenguaje COBOL, creado en 1959, pero aún hoy ampliamente utilizado en determinados ámbitos (por ejemplo, el sector bancario), proporciona un excelente soporte para ficheros indexados.

Los ficheros como medio de almacenamiento masivo de datos fueron relegados progresivamente en favor de las bases de datos relacionales. En realidad, las bases de datos relacionales utilizan internamente sofisticados sistemas de gestión de ficheros para el almacenamiento de los datos.

En este capítulo se presentarán algunas organizaciones sencillas de ficheros, y se explicará todo lo necesario para escribir sencillos programas en Java para consultar, añadir, borrar y modificar la información contenida en ellos.

TOMA NOTA



El almacenamiento de datos en ficheros de texto con organizaciones sencillas puede ser una solución perfectamente válida para algunas aplicaciones, pero nunca hay que perder de vista sus limitaciones intrínsecas y su limitada escalabilidad. Si hay que realizar consultas complejas o que requiere relacionar mucha información diversa, será difícil escribir un programa para realizarlas. Si el volumen de datos para manejar es muy grande, o si es necesario realizar con mucha frecuencia operaciones de borrado o modificación de datos, el rendimiento será muy pobre. Permitir que varios programas realicen a la vez operaciones de consulta y actualización puede introducir inconsistencias en los datos e incluso dañar los ficheros. Para evitar estos problemas son necesarios elaborados mecanismos de control de acceso que añaden mucha complejidad al sistema, y mucho más complicado es añadir soporte para transacciones. Es difícil establecer y hacer que se cumplan restricciones de integridad sobre los datos. Y también es difícil evitar redundancias en los datos que, además de desperdiciar espacio de almacenamiento, puede hacer que surjan inconsistencias cuando se añaden o modifican datos. Porque si la misma información está en más de un lugar, puede acabar teniendo un valor distinto en cada uno.

Se siguen utilizando ficheros para la persistencia de datos en muchas aplicaciones, y muy importantes. Las bases de datos relacionales han reemplazado los sistemas basados en ficheros para grandes colecciones de datos con una estructura muy regular, lo que es muy importante, pero no lo es todo. Hoy en día se utilizan mucho los ficheros en formato XML (al que se dedicará un capítulo posterior) para el almacenamiento de todo tipo de datos. Los sistemas de

correo electrónico suelen mantener sus datos en ficheros con un formato relativamente sencillo. Muchos procesos masivos que se ejecutan periódica o puntualmente se realizan basándose en datos proporcionados en ficheros de texto con una estructura muy sencilla. También se usan ficheros de texto sencillos para exportación e importación de datos entre sistemas, y también para copias de seguridad. Aparte de eso, está la infinidad de aplicaciones que almacenan los datos con los que trabajan en ficheros con infinidad de formatos diferentes.

2.2. Tipos de ficheros según su contenido

Un fichero es simplemente una secuencia de *bytes*, con lo que en principio puede almacenar cualquier tipo de información (véase figura 1.3).

Un fichero se identifica por su nombre y su ubicación dentro de una jerarquía de directorios.

Los ficheros pueden contener información de cualquier tipo, pero a grandes rasgos cabe distinguir entre dos tipos: los ficheros de texto y los ficheros binarios:

1. *Ficheros de texto*: contienen única y exclusivamente una secuencia de caracteres. Estos pueden ser caracteres visibles tales como letras, números, signos de puntuación, etc., y también espacios y separadores tales como tabuladores y retornos de carro. Su contenido se puede visualizar y modificar con cualquier editor de texto, como por ejemplo el bloc de notas en Windows o gedit en Linux.
2. *Ficheros binarios*: son el resto de los ficheros. Pueden contener cualquier tipo de información. En general, hacen falta programas especiales para mostrar la información que contienen. Los programas también se almacenan en ficheros binarios.



Figura 2.1
Ficheros
en una jerarquía
de directorios

2.3. Codificaciones para texto

Aunque la cuestión de las codificaciones para texto se ha incluido en este capítulo dedicado a los ficheros, es relevante para cualquier medio de almacenamiento de datos porque, allá donde se almacene un texto, debe hacerse con una codificación determinada.

Un texto es una secuencia de caracteres. Un texto, como cualquier tipo de información, se almacena en memoria o en cualquier dispositivo de almacenamiento como una secuencia de *bytes*. Una codificación es un método para representar cualquier texto como una secuencia de *bytes*. El mismo texto, según la codificación empleada, se puede representar como una secuencia de *bytes* distinta.

La variedad de codificaciones es un problema cada vez menos importante en la práctica, debido a la implantación general de Unicode y su codificación UTF-8, pero todavía es relativamente frecuente encontrar textos con otras codificaciones, sobre todo en entornos Windows.

Las más frecuentes son ISO 8859-1 y Windows-1252, que se puede considerar una variante no estándar de Microsoft del estándar ISO 8859-1. UTF-8 es compatible con el código ASCII,

lo que significa que cualquier texto codificado en ASCII se representa exactamente igual en UTF-8. Este ha sido un motivo fundamental para la adopción generalizada de UTF-8.

Para las nuevas aplicaciones, siempre hay que utilizar UTF-8 para almacenar texto, a no ser que haya alguna razón de peso para utilizar otra, que normalmente no la hay. A veces hay que importar u obtener textos de otras fuentes, desde donde podrían venir con otras codificaciones. Hay que identificar estas situaciones y hacer la conversión necesaria para recodificar los textos.

Recurso digital



En el anexo web 2.1, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás información sobre la codificación del contenido de ficheros.

RECUERDA

Cualquier editor de texto debería permitir visualizar correctamente un fichero de texto independientemente de su codificación. Por ejemplo, Notepad en Windows y gedit en Linux. Normalmente, con la opción "Guardar como..." se puede seleccionar la codificación que se va a utilizar, y UTF-8 suele aparecer como una de las opciones.

La manera más fiable y segura de confirmar el tipo de un fichero en Linux es con el comando `file`. Este analiza los contenidos del fichero e indica su tipo. Si es de texto, indica la codificación utilizada para el texto, típicamente: ASCII, UTF-8 o iso-8859-1.

El comando `iconv` de Linux permite recodificar ficheros de texto. El siguiente comando, por ejemplo, se podría utilizar para recodificar a UTF-8 un fichero codificado en ISO 8859-1.

```
iconv -f iso8859-1 -t utf-8 fichero_8859-1.txt > fichero_utf8.txt
```

2.4. La clase File de Java

La versión de Java utilizada para este libro es Java SE 8, una versión LTS (*long term support*, es decir, con soporte a largo plazo). En los servidores web de Oracle se puede consultar la documentación de la biblioteca estándar de clases de Java SE 8.

Las clases que permiten trabajar con ficheros están en el paquete `java.io`.

Recurso web

www

Se recomienda consultar en <http://docs.oracle.com/javase/8/docs/api> la documentación de Java SE8 para más información acerca de cualquier clase utilizada en este capítulo. Arriba, a la izquierda, aparece la lista de paquetes por orden alfabético. En ella se puede localizar el paquete `java.io`. Si se pulsa sobre él, aparece debajo la lista de interfaces y clases que contiene el paquete.

La clase `File` permite obtener información relativa a directorios y ficheros dentro de un sistema de ficheros y realizar diversas operaciones con ellos tales como borrar, renombrar, etc. En el siguiente cuadro se proporciona un resumen de la funcionalidad de esta clase, mostrando solo los principales métodos agrupados por categorías.

CUADRO 2.1

Métodos de la clase `File`

Categoría	Modificador/tipo	Método(s)	Funcionalidad
Constructor		<code>File(String ruta)</code>	Crea objeto <code>File</code> para la ruta indicada, que puede corresponder a un directorio o a un fichero.
Consulta de propiedades	<code>boolean</code>	<code>canRead()</code> <code>canWrite()</code> <code>canExecute()</code>	Comprueban si el programa tiene diversos tipos de permisos sobre el fichero o directorio, tales como de lectura, escritura y ejecución (si se trata de un fichero). Para un directorio, <code>canExecute()</code> significa que se puede establecer como directorio actual.
	<code>boolean</code>	<code>exists()</code>	Comprueba si el fichero o directorio existe.
	<code>boolean</code>	<code>isDirectory()</code> <code>isFile()</code>	Comprueban si se trata de un directorio o de un fichero.
	<code>long</code>	<code>length()</code>	Devuelve longitud del fichero.
	<code>File</code>	<code>getParent()</code> <code>getParentFile()</code>	Devuelven el directorio padre.
	<code>String</code>	<code>getName()</code>	Devuelve nombre del fichero.
Enumeración	<code>String[]</code>	<code>list()</code>	Devuelve un array con los nombres de los directorios y ficheros dentro del directorio.
	<code>File[]</code>	<code>listFiles()</code>	Devuelve un array con los directorios y ficheros dentro del directorio.
Creación, borrado y renombrado	<code>boolean</code>	<code>createNewFile()</code>	Crea nuevo fichero.
	<code>static File</code>	<code>createTempFile()</code>	Crea nuevo fichero temporal y devuelve objeto de tipo <code>File</code> asociado, para poder trabajar con él.
	<code>boolean</code>	<code>delete()</code>	Borra fichero o directorio.
	<code>boolean</code>	<code>renameTo()</code>	Renombra fichero o directorio.
	<code>boolean</code>	<code>mkdir()</code>	Crea un directorio.
Otras	<code>java.nio.file.Path</code>	<code>toPath()</code>	Devuelve un objeto que permite acceder a información y funcionalidad adicional proporcionada por el paquete <code>java.nio</code> .

El siguiente programa de ejemplo muestra por defecto un listado de los ficheros y directorios que contiene el directorio desde el que se ejecuta el programa. Pero si se le pasa la ruta de un directorio o fichero, muestra información acerca de él y, si se trata de un directorio, muestra los ficheros y directorios que contiene.

```
// Uso de la clase File para mostrar información de ficheros y directorios
package listadodirectorio;
import java.io.File;
public class ListadoDirectorio {
    public static void main(String[] args) {
        String ruta=".";
        if(args.length>=1) ruta=args[0];
        File fich=new File(ruta);
        if(!fich.exists()) {
            System.out.println("No existe el fichero o directorio (" + ruta + ").");
        }
        else {
            if(fich.isFile()) {
                System.out.println(ruta + " es un fichero.");
            }
            else {
                System.out.println(ruta + " es un directorio. Contenidos: ");
                File[] ficheros=fich.listFiles(); // Ojo, ficheros o directorios
                for(File f: ficheros) {
                    String textoDescr=f.isDirectory() ? "/" :
                    f.isFile() ? "_" : "?";
                    System.out.println("(" + textoDescr + ") " + f.getName());
                }
            }
        }
    }
}
```



PARA SABER MÁS

Se pueden pasar argumentos desde la línea de comandos a cualquier programa. Cualquier programa Java debe tener un método `main(String args [])` que se invoque en el momento de ejecutar el programa. El parámetro `String args[]` proporciona los argumentos de línea de comandos. Para pasar argumentos de línea de comando a un programa que se está desarrollando utilizando un IDE (entorno integrado de desarrollo), lo más cómodo suele ser utilizar las opciones que este IDE proporcione para ello dentro de su interfaz de usuario.



Actividad propuesta 2.1

Modifica el programa anterior para que muestre más información acerca de cada fichero y directorio, al menos el tamaño (si es un fichero), los permisos de los que se dispone sobre el fichero o

directorio, y la fecha de última modificación (en cualquier formato). Los permisos hay que mostrarlos en el formato habitual de Linux, a saber, con tres caracteres seguidos: una *r* si hay permiso para lectura o un guion si no lo hay; una *w* si hay permiso para escritura o un guion si no lo hay; y una *x* si hay permiso para ejecución, en el caso de que se trate de un fichero, o para entrar en el directorio, en el caso de que se trate de un directorio, o un guion si no lo hay.

2.5. Gestión de excepciones en Java

Antes de seguir avanzando con el contenido del capítulo, se incluye un breve repaso a la gestión de excepciones en el lenguaje Java. Cualquier programa escrito en Java debe realizar una adecuada gestión de excepciones. En este apartado se explicará lo más importante de la gestión de excepciones en Java, o al menos todo lo que se vaya a necesitar para este libro.

Una excepción es un evento que ocurre durante la ejecución de un programa y que interrumpe su curso normal de ejecución. Una excepción podría producirse, por ejemplo, al dividir por cero, como se muestra en el siguiente ejemplo.

```
// Excepción no gestionada durante la ejecución de un programa.
package excepcionesdivporcero;
public class ExcepcionesDivPorCero {
    public int divide(int a, int b) {
        return a/b;
    }
    public static void main(String[] args) {
        int a,b;
        a=5; b=2; System.out.println(a+"/"+b+"="+a/b);
        b=0; System.out.println(a+"/"+b+"="+a/b);
        b=3; System.out.println(a+"/"+b+"="+a/b);
    }
}
```

Al ejecutar este programa se obtendrá algo similar a lo siguiente:

```
5/2=2
Exception in thread "main" java.lang.ArithmeticException: / by zero at excepcionesdivporcero.
ExcepcionesDivPorCero.main(ExcepcionesDivPorCero.java:25)
```

2.5.1. Captura y gestión de excepciones

Cuando tiene lugar una excepción no gestionada, como con el programa anterior, aparte de mostrarse un mensaje de error, se aborta la ejecución del programa. Por ese motivo, el programa anterior no muestra el resultado de la última división entera. Cualquier fragmento de programa que pueda generar una excepción debería capturarlas y gestionarlas.

La salida del programa anterior indica el tipo de excepción que se ha producido, a saber, `ArithmeticException`. Se puede capturar esta excepción con un sencillo bloque `try {} catch {}` y gestionarla, con lo que el programa anterior podría quedar así:

```
// Excepción gestionada durante la ejecución de un programa.
package excepcionesdivporcerogest;
public class ExcepcionesDivPorCeroGest {
    public int divide(int a, int b) {
        return a / b;
    }

    public static void main(String[] args) {
        int a, b;
        a = 5; b = 2;
        try {
            System.out.println(a + "/" + b + "=" + a / b);
        } catch (ArithmeticException e) {
            System.err.println("Error al dividir: " + a + "/" + b);
        }
        try {
            b = 0; System.out.println(a + "/" + b + "=" + a / b);
        } catch (ArithmeticException e) {
            System.err.println("Error al dividir: " + a + "/" + b);
        }
        try {
            b = 3; System.out.println(a + "/" + b + "=" + a / b);
        } catch (ArithmeticException e) {
            System.err.println("Error al dividir: " + a + "/" + b);
        }
    }
}
```

Al ejecutar este programa, se captura y gestiona cualquier excepción que se pueda producir, de manera que se muestra un mensaje apropiado y se continúa con la ejecución. La salida del anterior programa sería la siguiente:

```
5/2=2
Error al dividir: 5/0
5/3=1
```

RECUERDA

- ✓ Es recomendable escribir los mensajes de error en la salida de error `System.err` en lugar de en la salida estándar `System.out`.



Actividad propuesta 2.2

¿Cómo cambiaría la funcionalidad del programa anterior si, en lugar de haber un bloque `try ... catch ...` para cada división, hubiera uno solo para las tres divisiones?

Las excepciones son objetos de Java, pertenecientes a la clase `Exception` o a una subclase de ella. Esta clase tiene varios métodos interesantes para obtener información acerca de la excepción que se ha producido.

CUADRO 2.2

Métodos de la clase `Exception`

Modificador/tipo	Método(s)	Funcionalidad
<code>void</code>	<code>printStackTrace()</code>	Muestra información técnica muy detallada acerca de la excepción y el contexto en que se produjo. Lo hace en la salida de error, <code>System.err</code> . Al principio del desarrollo de un programa, y para programas de prueba, puede ser una buena opción utilizar esta función para mostrar información de todas las excepciones, y perfilar más adelante cómo se gestionan excepciones de tipos particulares.
<code>String</code>	<code>getMessage()</code>	Proporciona un mensaje detallado acerca de la excepción.
<code>String</code>	<code>getLocalizedMessage()</code>	Proporciona una descripción localizada (es decir, traducida a la lengua local) de la excepción.

2.5.2. Gestión diferenciada de distintos tipos de excepciones

En un bloque `try {} catch {}` se pueden gestionar por separado distintos tipos de excepciones. Es conveniente incluir un manejador para `Exception` al final para que ninguna excepción se quede sin gestionar.



PARA SABER MÁS

En aplicaciones profesionales, la práctica habitual es utilizar herramientas de *logging* (de registro), y no solo para mensajes de error. Los mensajes se suelen registrar en ficheros. Estas herramientas permiten generar de forma diferenciada distintos tipos de mensaje, típicamente, y por orden de importancia, de mayor a menor: *error*, *warning* (de aviso), *info* (informativo) y *debug* (para depuración). Permiten configurar el nivel de *logging*, de manera que solo se registren los mensajes a partir del nivel de importancia establecido. Si este se establece como *warning*, se mostrarían los de tipo *warning* y *error*. Entre las herramientas más ampliamente utilizadas para Java están:

- Java Logging API (<http://docs.oracle.com/javase/8/docs/technotes/guides/logging>)
- Log4j (<http://logging.apache.org/log4j>)

El siguiente programa rellena un *array* con números y después realiza un cálculo aritmético para cada uno con ellos y muestra la segunda cifra del resultado. Esto no es realmente muy útil, como no sea para mostrar cómo se puede hacer saltar excepciones de diversos tipos, capturarlas y gestionarlas por separado. Solo para un elemento del *array* se realiza el cálculo sin que salte ninguna excepción.

```
// Gestión diferenciada de distintos tipos de excepciones
package excepcionesdiversas;
public class ExcepcionesDiversas {
    public static void main(String[] args) {
        // Rellenar array con números variados
        int nums[][] = new int[2][3];
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                nums[i][j] = i + j;
            }
        }
        // Realizar cálculo para cada posición del array.
        // Se producen excepciones de diversos tipos.
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                try {
                    System.out.print("Segunda cifra de 5*nums["+i+"["
                        +j+"]/"+j+": ");
                    System.out.println(String.valueOf(5 * nums[i][j] /
                        j).charAt(1));
                } catch (ArithmeticException e) {
                    System.out.println("ERROR: aritmético 5*"+nums[i][j]+"/"+j);
                } catch (ArrayIndexOutOfBoundsException e) {
                    System.out.println("ERROR: No existe nums["+i+"["+j+"]");
                } catch (Exception e) {
                    System.out.println("ERROR: de otro tipo al calcular segunda
                        cifra de: 5*"+nums[i][j]+"/"+j);
                    System.out.println();
                    e.printStackTrace();
                }
            }
        }
    }
}
```

Para poder interpretar más fácilmente la salida de este programa, todos los mensajes de error se han dirigido hacia `System.out` y no `System.err`. Su salida sería:

```
Segunda cifra de 5*nums[0][0]/0: ERROR: aritmético 5*0/0
Segunda cifra de 5*nums[0][1]/1: ERROR: de otro tipo al calcular segunda cifra de: 5*1/1
java.lang.StringIndexOutOfBoundsException: String index out of range: 1
at java.lang.String.charAt(String.java:658)
at excepcionesdiversas.ExcepcionesDiversas.main(ExcepcionesDiversas.java:31)
Segunda cifra de 5*nums[0][2]/2: ERROR: de otro tipo al calcular segunda cifra de: 5*2/2
java.lang.StringIndexOutOfBoundsException: String index out of range: 1
at java.lang.String.charAt(String.java:658)
at excepcionesdiversas.ExcepcionesDiversas.main(ExcepcionesDiversas.java:31)
Segunda cifra de 5*nums[1][0]/0: ERROR: aritmético 5*1/0
Segunda cifra de 5*nums[1][1]/1: 0
Segunda cifra de 5*nums[1][2]/2: ERROR: de otro tipo al calcular segunda cifra de: 5*3/2
```

```

java.lang.StringIndexOutOfBoundsException: String index out of range: 1
at java.lang.String.charAt(String.java:658)
at excepcionesdiversas.ExcepcionesDiversas.main(ExcepcionesDiversas.java:31)
Segunda cifra de 5*nums[2][0]/0: ERROR: No existe nums[2][0]
Segunda cifra de 5*nums[2][1]/1: ERROR: No existe nums[2][1]
Segunda cifra de 5*nums[2][2]/2: ERROR: No existe nums[2][2]

```

2.5.3. Declaración de excepciones lanzadas por un método de clase

Si el compilador es capaz de determinar que un método de una clase puede originar un tipo de excepción, pero no lo gestiona mediante un bloque `catch() {}`, la compilación terminará con un error. Una posibilidad entonces es gestionar la excepción en el método mediante un bloque `catch() {}`. La otra es añadir el modificador `throws` seguido de la clase de excepción.

La siguiente clase tiene un método que crea un fichero temporal con un nombre que empieza por un prefijo dado, y escribe en él un carácter dado, un número dado de veces. Durante este proceso puede saltar la excepción `IOException` en varias ocasiones, pero no se quiere gestionarla en el método. Por ello se incluye `throws IOException` en su declaración. Por lo demás, el método es sencillo y, en cualquier caso, lo importante es comprender la gestión de excepciones. Las clases y procedimientos utilizados se verán más adelante en este capítulo.

```

// Declaración de excepciones lanzadas por método de clase con throws
package excepcionesconthrows;

import java.io.File;
import java.io.IOException;
import java.io.FileWriter;

public class ExcepcionesConThrows {

    public File creaFicheroTempConCar(String prefNomFich, char car, int
        numRep) throws IOException {
        File f = File.createTempFile(prefNomFich, "");
        FileWriter fw = new FileWriter(f);
        for (int i = 0; i < numRep; i++) fw.write(car);
        fw.close();
        return f;
    }

    public static void main(String[] args) {
        try {
            File ft = new ExcepcionesConThrows().
                creaFicheroTempConCar("AAAA_", 'A', 20);
            System.out.println("Creado fichero: " + ft.getAbsolutePath());
            ft.delete();
        } catch (IOException e) {
            System.err.println(e.getMessage());
        }
    }
}

```

2.5.4. Excepciones, inicialización y liberación de recursos: bloques `finally` y `try` con recursos

Es muy frecuente que un bloque de programa de Java esté estructurado de la siguiente forma:

```
Inicialización y asignación de recursos
Cuerpo
Finalización y liberación de recursos
```

Este bloque puede estar dentro de un método de clase, como por ejemplo, el método `main()`, o de un bucle.

La primera y última parte deben ejecutarse siempre, independientemente de los errores que puedan suceder durante la ejecución del cuerpo. El bloque anterior, con gestión de excepciones, podría quedar de la siguiente manera:

```
Inicialización y asignación de recursos
try {
    Cuerpo
} catch (Excepcion_Tipo_1 e1) {
    Gestión de excepción de tipo 1
} catch (Excepcion_Tipo_2 e2) {
    Gestión de excepción de tipo 2
} catch (Exception e) {
    Gestión del resto de tipos de excepciones
}
Finalización y liberación de recursos
```

Pero es muy frecuente que, cuando sucede algún error en mitad del cuerpo, se quiera terminar inmediatamente la ejecución, bien sea con una sentencia `return` (en un método de clase), o con `break` o `continue` (dentro de un bucle, para salir de él o para saltar a la siguiente iteración). De esta manera no se ejecuta la parte final para finalización y liberación de recursos. Pero si se pone esta parte dentro de un bloque `finally {}`, se ejecutará justo antes de abandonar.

```
Inicialización y asignación de recursos
try {
    Cuerpo
} catch (Excepcion_Tipo_1 e1) {
    Gestión de excepción de tipo 1
} catch (Excepcion_Tipo_2 e2) {
    Gestión de excepción de tipo 2
} catch (Exception e) {
    Gestión del resto de tipos de excepciones
}
finally {
    Finalización y liberación de recursos
}
```

Los bloques `try` con recursos son de utilidad para simplificar la gestión de recursos de clases que implementan una de las interfaces `Closeable` o `AutoCloseable`. Para estos se invocará automáticamente el método `close()` en el bloque `finally`. Si no hay un bloque `finally`, se puede entender que existe uno vacío.

```

Inicialización y asignación de recursos
try (T1 r1=new T1(); T2 r2=new T2()) {
    // T1, T2 implementan Closeable o AutoCloseable

    Cuerpo

} catch (Excepcion_Tipo_1 e1) {
    Gestión de excepción de tipo 1
} catch (Excepcion_Tipo_2 e2) {
    Gestión de excepción de tipo 2
} catch (Exception e) {
    Gestión del resto de tipos de excepciones
}
finally {
Finalización y liberación de recursos
    // No es necesario r1.close()
    // No es necesario r2.close()
}

```

Recurso digital



En el anexo web 2.2 encontrarás un ejemplo que simula un programa que abre para lectura dos ficheros: f1.dat y f2.dat, y que crea dos ficheros temporales: f1.info.tmp y f2.info.tmp, que borra al final.

El anexo está disponible en www.sintesis.com y es accesible con el código indicado en la primera página del libro.

2.6. Formas de acceso a los ficheros

Existen dos formas de acceder a los contenidos de los ficheros. A saber:

1. *Acceso secuencial.* Se accede comenzando desde el principio del fichero. Para llegar a cualquier parte del fichero, hay que pasar antes por todos los contenidos anteriores, empezando desde el principio del fichero.
2. *Acceso aleatorio.* Se accede directamente a los datos situados en cualquier posición del fichero.

Ambas formas de acceso se pueden utilizar para operaciones tanto de lectura como de escritura. La mayoría de los medios de almacenamiento permiten el acceso secuencial y el acceso aleatorio a los ficheros que almacenan. Pero en algunos casos podría ser posible solo el acceso secuencial. Este sería el caso si el fichero está almacenado en una cinta magnética, si bien este medio de almacenamiento está hoy en desuso. Si se considera el concepto más general de flujo de datos o *stream*, del que un fichero sería un caso particular, existen más casos en los que solo es posible el acceso secuencial. Cuando dos aplicaciones se comunican mediante una conexión de red, por ejemplo, la aplicación de origen envía los datos secuencialmente, y la de destino los recibe secuencialmente. En muchos lenguajes de programación, entre ellos Java, se pueden utilizar las mismas funciones para enviar y recibir datos por una conexión de red que para escribir y leer datos en ficheros.

2.7. Operaciones sobre ficheros con Java

Independientemente del tipo de fichero (binario o de texto) y del tipo de acceso (secuencial o aleatorio), las operaciones básicas sobre ficheros son en lo esencial iguales, y se explicarán en este apartado. En los apartados siguientes se introducirán las particularidades de las clases que proporciona Java para diversos tipos de operaciones con diversos tipos de ficheros, y las operaciones adicionales que cada una permite realizar.

El mecanismo de acceso a un fichero está basado en un puntero y en una zona de memoria que se suele llamar *buffer*. El puntero siempre apunta a un lugar del fichero, o bien a una posición especial de fin de fichero, que a veces se denomina EOF (del inglés *end of file*). Esta se puede entender que está situada inmediatamente a continuación del último *byte* del fichero. Todas las clases para operar con ficheros disponen de las siguientes operaciones básicas:

1. *Apertura*. Antes de hacer nada con un fichero, hay que abrirlo. Esto se hace al crear una instancia de una clase que se utilizará para operar con él.
2. *Lectura*. Mediante el método `read()`. Consiste en leer contenidos del fichero para volcarlos a memoria y poder trabajar con ellos. El puntero se sitúa justo después del último carácter leído.
3. *Salto*. Mediante el método `skip()`. Consiste en hacer avanzar el puntero un número determinado de *bytes* o caracteres hacia delante.
4. *Escritura*. Mediante el método `write()`. Consiste en escribir contenidos de memoria en un lugar determinado del fichero. El puntero se sitúa justo después del último carácter escrito.
5. *Cierre*. Mediante el método `close()`. Para terminar, hay que cerrar el fichero.

La diferencia entre el acceso secuencial y el acceso aleatorio es que, con el último, se puede, en cualquier momento, situar el puntero en cualquier lugar del fichero. En cambio, con el acceso secuencial solo se mueve el cursor tras realizar operaciones de lectura, escritura o salto.

2.7.1. Operaciones de lectura

Cuando se lee desde el fichero, hay que indicar el *buffer*, que recibirá los datos que se lean desde él. Si no se indica el número de *bytes* o de caracteres para leer, se leerá hasta llenar el *buffer*. Lógicamente, no se leerá nada si el puntero apunta a la posición EOF (final del fichero).

Los ejemplos que siguen, por claridad, son para un fichero de texto en el que cada carácter ocupa un *byte*. Pero la operación de lectura puede realizarse para un fichero tanto binario como de texto. En el primer caso, se leerán *bytes*, y en el segundo, caracteres, cada uno de los cuales puede estar formado por un número variable de *bytes*, dependiendo del carácter y de la codificación empleada para el texto.

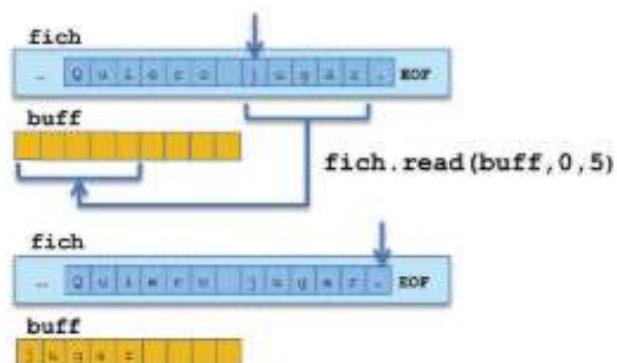


Figura 2.2
Lectura desde fichero

2.7.2. Operaciones de escritura

Cuando se escribe en el fichero, hay que indicar el *buffer* y el número de *bytes* para leer. Desde el *buffer* se transfieren los datos a la posición a la que apunta el puntero.

Si el puntero apunta al final del fichero, se añaden los datos al final de este.

Para concluir con las operaciones básicas, poco más se puede hacer que lo visto hasta ahora. No existe, por ejemplo, ninguna forma directa de insertar datos en medio de un fichero, de eliminar un fragmento de un fichero, ni de sustituir los contenidos de un fragmento de un fichero por otros, como no sea que tengan la misma longitud. Esto es así en Java y en prácticamente todos los lenguajes de programación, y se debe a que las implementaciones de los sistemas de ficheros más habituales no proporcionan ninguna manera directa para hacer operaciones de este tipo. Pero se puede hacer utilizando ficheros auxiliares, como se verá más adelante.

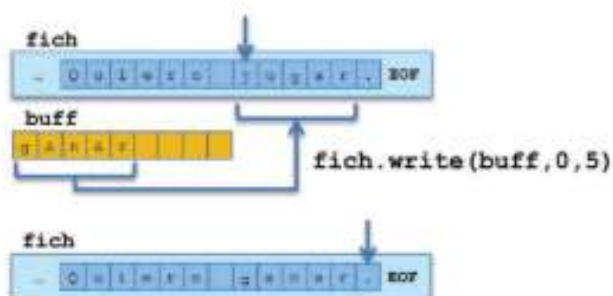


Figura 2.3

Escritura a un fichero cuando el puntero apunta en medio del fichero

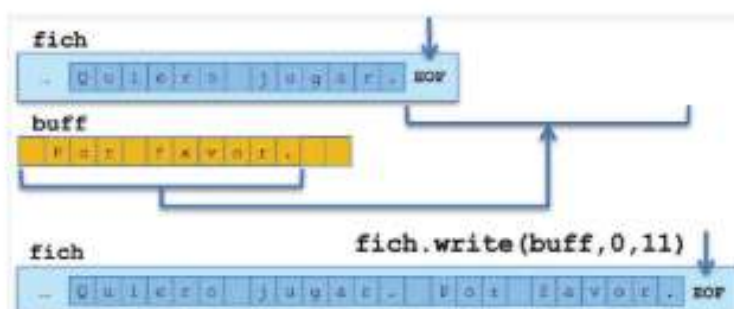


Figura 2.4

Escritura a un fichero cuando el puntero apunta al final del fichero

2.8. Acceso secuencial a ficheros en Java

Las operaciones con ficheros secuenciales en Java se realizan utilizando flujos (*streams*, en inglés) mediante clases del paquete `java.io`. Un flujo es una abstracción de alto nivel para cualquier secuencia de datos. Los ficheros secuenciales son flujos, lo mismo que los datos enviados a través de una conexión de red, o la información contenida en una secuencia de posiciones consecutivas de memoria, o la entrada y la salida estándar y de error de un programa en ejecución. Este paquete saca partido de la herencia, y proporciona clases con funcionalidad genérica de entrada y salida, común para cualquier tipo de flujo, y clases con funcionalidad específica para los distintos tipos de flujos.

Existe una jerarquía de clases derivada de cuatro clases, correspondientes a las cuatro combinaciones posibles, considerando, por una parte, flujos binarios y de texto y, por otra, flujos de entrada y de salida.

2.8.1. Clases relacionadas con flujos de datos

Para leer de un fichero o escribir en un fichero hay que crear un flujo (*stream*) asociado al mismo. El tipo de flujo para crear será distinto según se trate de un fichero binario o de texto.

Los flujos binarios no plantean mayor problema: se leen *bytes* de un flujo binario hacia una variable de tipo `byte[]`, o se escriben *bytes* de una variable de tipo `byte[]` a un flujo binario. Cuando se lee o escribe en flujos de texto, en cambio, todo es más complicado, porque entran en juego las codificaciones de texto. Todo funcionará bien mientras se trabaje con ficheros de texto con la codificación que Java asume por defecto. Normalmente será así, pero puede no serlo.

La codificación que utiliza Java por defecto para flujos de texto la toma de la configuración de la máquina virtual de Java, o en su defecto de la configuración del sistema operativo, o asume UTF-8. En la práctica, normalmente será UTF-8, que es la más habitual hoy en día. Esta codificación utiliza de uno a cuatro *bytes* para representar un carácter. Uno para todo lo que esté en el antiguo código ASCII. Dos para todo lo que no existe en inglés, como vocales acentuadas, la letra ñ, etc. Y hasta cuatro con algunos sistemas de escritura no basados en el alfabeto latino.

En cambio, para almacenar texto en memoria, Java utiliza UTF-16. Por tanto, las lecturas y escrituras con ficheros de texto suelen llevar implícito un proceso de recodificación. UTF-16 utiliza dos o cuatro *bytes* para representar un carácter o, lo que es lo mismo, uno o dos `char` (que en Java son dos *bytes*). Una cadena de texto en Java se almacena, pues, en un `array char[]` o en un `String` codificado en UTF-16 como una secuencia de caracteres, cada uno representado por dos *bytes* (normalmente) o por cuatro (con algunos sistemas de escritura no basados en el alfabeto latino).

RECUERDA

- ✓ Java codifica el texto en memoria en UTF-16, en variables de tipo `String` o `char[]`. La mayoría de los ficheros de texto están codificados en UTF-8, y normalmente Java utiliza esta codificación por defecto para leer y escribir en ellos. Por lo tanto, las operaciones de lectura o escritura de texto con ficheros de texto llevan implícito, generalmente, un proceso de recodificación.

Para lectura y escritura en flujos, Java proporciona dos jerarquías de clases: una para flujos binarios y otra para flujos de texto.

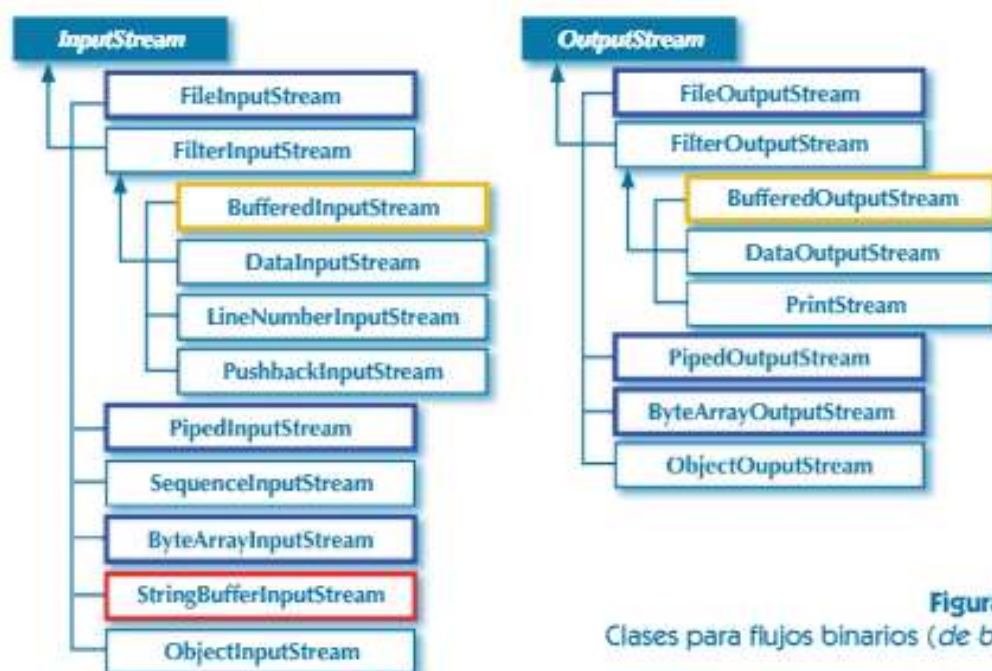


Figura 2.5
Clases para flujos binarios (de bytes)



TOMA NOTA

La clase `StringBufferInputStream` está obsoleta (*deprecated* según la terminología de Java). En su lugar habría que usar `StringReader`.

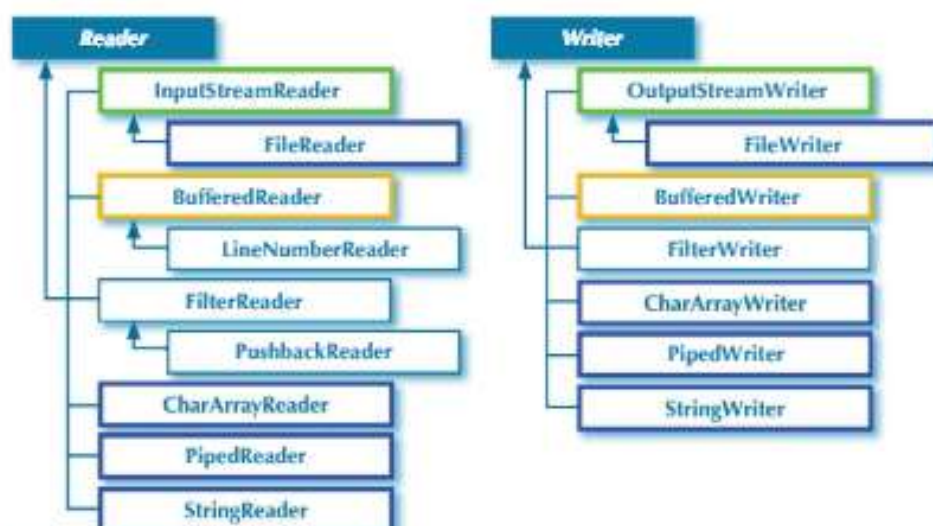


Figura 2.6
Clases para flujos de texto

En estas dos jerarquías, algunas clases ofrecen la funcionalidad básica de lectura y escritura en flujos asociados a distintos tipos de fuentes de datos. Otras ofrecen recodificación de texto. Otras ofrecen *buffering*, que permite aumentar la velocidad de las operaciones de lectura y escritura, y hace posible leer y escribir líneas de texto para los ficheros de texto. Todas estas clases se pueden componer unas sobre otras, de manera que cada una añada funcionalidad nueva sobre la proporcionada por las anteriores.

- Las clases que ofrecen la funcionalidad básica de lectura y escritura son las representadas con borde azul marino. Java usa una nomenclatura sistemática para ellas. El nombre indica primero el tipo de fuente (`File` para fichero, `ByteArray` para memoria, `Piped` para tubería). Después, si es de entrada (`Input` o `Reader`) o de salida (`Output` o `Writer`). Si acaba con `Stream` es para flujos de *bytes*, y si no, para flujos de texto.
- Las clases que ofrecen recodificación de texto son las representadas con borde verde claro.
- Las clases que ofrecen *buffering* son las representadas con borde verde oscuro.

CUADRO 2.3
Clases básicas para entrada y salida a flujos

	Fuente de datos	Lectura	Escritura
Flujo binario	Ficheros	<code>FileInputStream</code>	<code>FileOutputStream</code>
	Memoria (<code>byte[]</code>)	<code>ByteArrayInputStream</code>	<code>ByteArrayOutputStream</code>
	Tuberías	<code>PipedInputStream</code>	<code>PipedOutputStream</code>
Flujo de texto	Ficheros	<code>FileReader</code>	<code>FileWriter</code>
	Memoria (<code>char[]</code>)	<code>CharArrayReader</code>	<code>CharArrayWriter</code>
	Memoria (<code>String</code>)	<code>StringReader</code>	<code>StringWriter</code>
	Tuberías	<code>PipedReader</code>	<code>PipedWriter</code>

A continuación, se dan algunos ejemplos de cómo se crean objetos de estas clases. Para más información acerca de ellas, se puede consultar la documentación de Java.

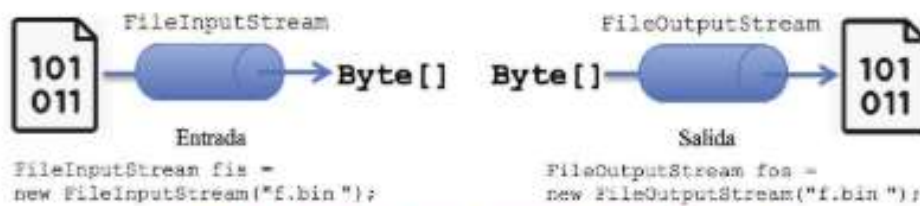


Figura 2.7. Lectura y escritura en archivos binarios



Figura 2.8. Lectura y escritura en archivos de texto con la codificación por defecto

2.8.2. Clases para recodificación

Las clases `InputStreamReader` y `OutputStreamWriter` sirven de enlace entre ambas jerarquías: la de flujos binarios y la de flujos de texto. Dado que convierten flujos binarios en flujos de texto y viceversa, es necesario especificar una codificación a su constructor. Se pueden entender como clases que permiten recodificar texto, es decir, leer o escribir texto en archivos con una codificación diferente a la codificación por defecto (que, como ya se ha comentado, suele ser UTF-8).

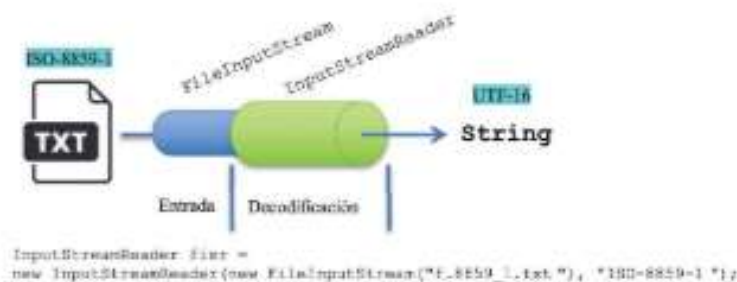


Figura 2.9. Lectura desde archivos de texto con codificación distinta a la codificación por defecto

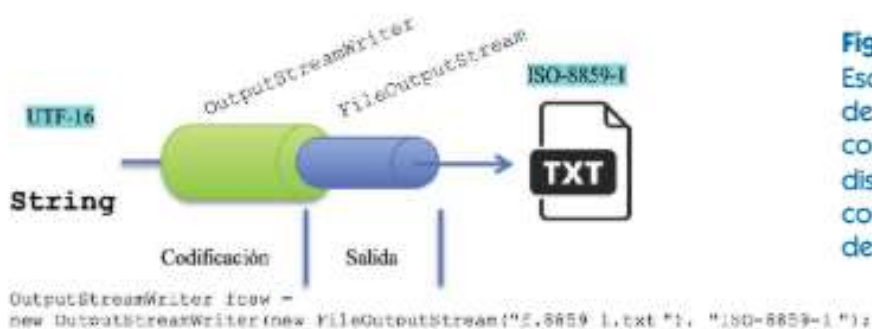


Figura 2.10. Escritura a archivos de texto con codificación distinta a la codificación por defecto

2.8.3. Clases para *buffering*

Existen algunas clases que proporcionan *buffering* sobre el servicio proporcionado por las clases anteriores. El *buffering* es una técnica que permite acelerar las operaciones de lectura o escritura utilizando una zona de intercambio en memoria llamada *buffer*. Estas clases permiten además leer líneas de texto y escribir líneas de texto en ficheros de texto.



TOMA NOTA

En general conviene utilizar las clases que proporcionan *buffering*. El rendimiento aumenta para ficheros grandes y cuando se realizan lecturas o escrituras consecutivas en posiciones contiguas, que es lo habitual. Cuando no es así, la merma en el rendimiento no es significativa.

El *buffer* siempre representa el contenido actual de una sección del fichero. Cuando se lee de un fichero, la lectura no se limita a los datos solicitados, sino que se traen datos suficientes para llenar el *buffer*. Si la siguiente lectura del fichero es en posiciones consecutivas, lo que es muy habitual, los datos ya estarán en el *buffer*, y así se ahorra un nuevo acceso al fichero. Se puede utilizar un *buffer* de manera análoga para las operaciones de escritura. En lugar de escribir directamente en el fichero, se escribe en el *buffer*. Solo se vuelcan los contenidos del *buffer* en el fichero cuando una operación de escritura afecta a una parte del fichero fuera de la sección representada en el *buffer*, o cuando otro programa lee información de esa sección. A la actualización del fichero con los contenidos del *buffer* se le llama en inglés *flushing*.

CUADRO 2.4

Clases que proporcionan *buffering* para entrada y salida a flujos

	Lectura	Escritura
Flujo binario	BufferedInputStream	BufferedOutputStream
Flujo de texto	BufferedReader	BufferedWriter

Cada una de las clases anteriores proporciona *buffering* para objetos de una clase cuyo nombre viene después de **Buffered**, y además es subclase de ella, por lo que se puede usar en su lugar. Se podrían cambiar algunos de los ejemplos anteriores para utilizar *buffering*:

CUADRO 2.5

Flujo sin *buffering* y con *buffering*

Flujo sin <i>buffering</i>	Flujo con <i>buffering</i>
<code>new FileInputStream("f.bin")</code>	<code>new BufferedInputStream(new FileInputStream("f.bin"))</code>
<code>new FileOutputStream("f.bin")</code>	<code>new BufferedOutputStream(new FileOutputStream("f.bin"))</code>

[.../...]

CUADRO 2.5 (CONT.)

<code>new FileReader("f.txt")</code>	<code>new BufferedReader(new FileReader("f.txt"))</code>
<code>new FileWriter("f.txt")</code>	<code>new BufferedWriter(new FileWriter("f.txt"))</code>

Como ya se ha dicho, las clases que proporcionan *buffering* para ficheros de texto permiten además leer y escribir líneas de texto.

CUADRO 2.6

Métodos para lectura y escritura de líneas en clases para *buffering* con ficheros de texto

Clase	Método	Funcionalidad
<code>BufferedReader</code>	<code>String readLine()</code>	Lee hasta el final de la línea actual.
<code>BufferedWriter</code>	<code>void newLine()</code>	Escribe un separador de líneas. El separador de líneas puede depender del sistema operativo, y suele ser distinto en Linux y en Windows. El método <code>readLine()</code> de <code>BufferedReader</code> tiene en cuenta estas particularidades.

2.8.4. Operaciones de lectura para flujos de entrada

Son similares para todas las clases que implementan flujos (*streams*) de entrada en Java. Las funciones `read` de clases que heredan de `InputStream` leen *bytes*, mientras que las que heredan de `Reader` leen caracteres. Según la variante, leen un *byte* o un carácter hasta llenar el *buffer* o el número de *bytes* o caracteres indicados, que se copiarán en la posición indicada (*offset*) dentro del *buffer*. Devuelven el número de *bytes* o caracteres leídos, o -1 si no se pudo leer nada porque el puntero estaba al final del fichero. Las funciones `skip` saltan el número indicado de *bytes* o caracteres, aunque podrían ser menos si se alcanza el final del fichero. En cualquier caso, devuelven el número de *bytes* o de caracteres que se han saltado.

Con ficheros de texto, para leer líneas se puede utilizar el método `readLine()` de un `BufferedReader` construido sobre un `FileReader`.

CUADRO 2.7

Métodos para lectura de las clases para gestión de flujos de entrada

<code>InputStream</code>	<code>Reader</code>
<code>int read()</code>	<code>int read()</code>
<code>int read(byte[] buffer)</code>	<code>int read(char[] buffer)</code>
<code>int read(byte[] buffer, int offset, int longitud)</code>	<code>int read(char[] buffer, int offset, int longitud)</code>
	<code>int read(CharBuffer buffer)</code>
<code>long skip(long n)</code>	<code>long skip(long n)</code>
	<code>BufferedReader</code>
	<code>String readLine()</code>

Como ejemplo, el siguiente programa muestra los contenidos de un fichero de texto línea a línea, numerando las líneas. Para leer líneas de texto se usa el método `readLine()` de la clase `BufferedReader`. En este programa, y en todos a partir de ahora, se utilizarán bloques `try` con recursos para crear distintos tipos de flujos (*stream*), con lo que `close()` se ejecutará automáticamente al final.

```
// Uso de readLine() de BufferedReader

import java.io.FileReader;
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.IOException;

public class EscribeConNumeroDeLineas {

    public static void main(String[] args) {
        if (args.length < 1) {
            System.out.println("Indicar por favor nombre de fichero.");
            return;
        }
        String nomFich = args[0];

        try(BufferedReader fbr = new BufferedReader(new FileReader(nomFich))) {
            int i = 0;
            String linea = fbr.readLine();
            while (linea != null) {
                System.out.format("[%5d] %s", i++, linea);
                System.out.println();
                linea = fbr.readLine();
            }
        } catch (FileNotFoundException e) {
            System.out.println("No existe fichero " + nomFich);
        } catch (IOException e) {
            System.out.println("Error de E/S: " + e.getMessage());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Actividades propuestas



- 2.3.** Crea un programa que busque un texto dado en un fichero de texto, y que muestre para cada aparición la línea y la columna. Se recomienda leer el fichero línea a línea y, dentro de cada línea, buscar las apariciones del texto utilizando un método apropiado de la clase `String`. Se puede consultar la documentación de dicha clase en la API de Java (<http://docs.oracle.com/javase/8/docs/api>).
- 2.4.** Crea un programa que, a partir de un fichero de texto codificado en UTF-8, genere un fichero de texto codificado en ISO-8859-1 y otro en UTF-16. El fichero codificado en UTF-8 debe crearse con un editor de texto, y debe incluir al menos vocales acentuadas. Puedes leer el fichero línea a línea con `readLine()`. Para generar el fichero de salida, puedes utilizar un `BufferedWriter` (para escribir línea a línea) construido sobre un `OutputStreamWriter` (para recodificar el texto) construido sobre un `FileOutputStream` (para escribir a un fichero). Busca una manera de verificar la codificación de los

ficheros de texto en el sistema operativo que estés utilizando mediante algún comando del sistema operativo o algún programa de utilidad. Puedes utilizar los programas de ejemplo para volcado binario para verificar qué caracteres se codifican de manera distinta. Puedes crear otro programa que haga la conversión inversa, para comprobar que se vuelve a obtener un fichero igual al inicial, utilizando las clases `InputStreamReader` y `FileInputStream`.

Ahora un ejemplo con flujos binarios. El siguiente programa hace un volcado binario de un fichero indicado desde línea de comandos. Los contenidos del fichero se leen en bloques de 32 bytes, y el contenido de cada bloque se escribe en una línea de texto. Los bytes se escriben en hexadecimal (base 16) y, por tanto, cada byte se escribe utilizando dos caracteres. El programa muestra como máximo los primeros 2 kilobytes (`MAX_BYTES=2048`). Por supuesto, este programa se puede utilizar tanto con ficheros binarios como con ficheros de texto. Hacer notar que esta clase permite hacer el volcado binario de un `InputStream`, y un `FileInputStream` es un caso particular. Siempre que sea posible, debemos hacer que las clases que desarrollemos funcionen con *streams* en general, y no solo con ficheros en particular.

```
// Volcado hexadecimal de un fichero con FileInputStream
package volcadobinario;

import java.io.InputStream;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;

public class VolcadoBinario {
    static int TAM_FILA=32;
    static int MAX_BYTES=2048;
    InputStream is=null;

    public VolcadoBinario(InputStream is) {
        this.is=is;
    }

    public void volcar() throws IOException {
        byte buffer[]=new byte[TAM_FILA];
        int bytesLeidos;
        int offset=0;
        do { bytesLeidos=is.read(buffer);
            System.out.format("%5d", offset);
            for(int i=0; i<bytesLeidos; i++) {
                System.out.format(" %2x", buffer[i]);
            }
            offset+=bytesLeidos;
            System.out.println();
        } while (bytesLeidos==TAM_FILA && offset<MAX_BYTES);
    }

    public static void main(String[] args) {
        if(args.length<1) {
            System.out.println("No se ha indicado ningún fichero");
            return;
        }
    }
}
```

```

String nomFich=args[0];
try (FileInputStream fis = new FileInputStream(nomFich)) {
    VolcadoBinario vb = new VolcadoBinario(fis);
    vb.volcar();
}
catch(FileNotFoundException e) {
    System.err.println("ERROR: no existe fichero "+nomFich);
}
catch(IOException e) {
    System.err.println("ERROR de E/S: "+e.getMessage());
}
catch(Exception e) {
    e.printStackTrace();
}
}
}

```

Actividad propuesta 2.5



Modifica la clase `VolcadoBinario` para que pueda hacer el volcado a cualquier `PrintStream`, en lugar de siempre a `System.out`. Modifica el método `main()` para que realice el volcado hacia un fichero.

2.8.5. Operaciones de escritura para flujos de salida

Son similares para todas las clases que implementan flujos (*streams*) de salida en Java. Las funciones `write` de clases que heredan de `OutputStream` escriben *bytes*, mientras que las que heredan de `Writer` escriben caracteres. Según la variante, escriben un *byte* o un carácter, o todos los contenidos del *buffer*, o el número de *bytes* o caracteres indicados a partir de la posición indicada. Si se alcanza el fin del fichero, siguen escribiendo. Las que heredan de `Writer` tienen métodos para escribir los contenidos de un `String`, y métodos `append` para añadir al final del fichero.

Una característica muy útil de las clases `FileOutputStream` y `Writer` es que disponen de constructores con un parámetro que permite abrir ficheros para añadir contenidos al final (*append* en inglés). También disponen de constructores sin ese parámetro.

```

FileOutputStream(File file, boolean append)
FileOutputStream(String nombreFichero, boolean append)
Writer (File file, boolean append)
Writer(String nombreFichero, boolean append)

```

CUADRO 2.8

Métodos para escritura de clases para gestión de flujos de salida

OutputStream	Writer
<code>void write(int b)</code>	<code>void write(int c)</code>
<code>void write(byte[] buffer)</code>	<code>void write(char[] buffer)</code>
<code>void write(byte[] buffer, int offset, int longitud)</code>	<code>void write(char[] buffer, int offset, int longitud)</code>
	<code>void write(String str)</code>
	<code>void write(String str, int offset, int longitud)</code>

[.../...]

CUADRO 2.8 (CONT.)

	Writer append(char c) Writer append(CharSequence csq) Writer append(CharSequence csq, int offset, int longitud)
	BufferedWriter
	void newLine()

El siguiente programa escribe un texto en un fichero. Después lo cierra y lo vuelve a abrir en modo *append* para añadir nuevos contenidos al final. A menos que el fichero ya exista, en cuyo caso no hace nada. Si se hicieran estas mismas operaciones sobre un fichero existente, se perderían los contenidos del fichero. Se añaden saltos de línea con `newLine()`.

```
// Añadir contenidos al final de un fichero de texto
package escribeenflujosalida;
import java.io.File;
import java.io.FileWriter;
import java.io.BufferedWriter;
import java.io.IOException;
public class EscribeEnFlujoSalida {
    public static void main(String[] args) {
        String nomFichero="f_texto.txt";
        File f=new File(nomFichero);
        if(f.exists()) {
            System.out.println("Fichero "+nomFichero+" ya existe. No se hace nada");
            return;
        }
        try {
            BufferedWriter bfw=new BufferedWriter(new FileWriter(f));
            bfw.write(" Este es un fichero de texto. ");
            bfw.newLine();
            bfw.write(" quizá no está del todo bien.");
            bfw.newLine();
            bfw.close();
            bfw=new BufferedWriter(new FileWriter(f, true));
            bfw.write(" Pero se puede arreglar.");
            bfw.newLine();
            bfw.close();
        }
        catch(IOException e) {
            System.out.println(e.getMessage());
        }
        catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

No es posible eliminar o reemplazar contenidos de un fichero directamente utilizando solo flujos, porque para ello es necesario leer y escribir en el mismo fichero. Se puede hacer utilizando ficheros auxiliares, que se pueden crear con `createTempFile()` de la clase `File`.

El siguiente programa realiza diversos cambios en los contenidos de un fichero de texto tales como eliminar secuencias de espacios al principio de línea, sustituir secuencias de espacios en otros lugares por un solo espacio, y hacer que todas las líneas empiecen por mayúsculas. Se puede probar con el fichero generado por el programa anterior.

Para las transformaciones en el texto se utiliza funcionalidad de la clase `Character`. Lo más importante no es entender en detalle lo que hace dentro del bucle `while` para cada línea, sino la técnica que emplea para modificar los contenidos de un fichero, leyendo de un `BufferedReader`, escribiendo en un `BufferedWriter`, utilizando ficheros temporales, y renombrando ficheros. Antes que nada, y por si acaso, se hace una copia del fichero en uno nuevo en cuyo nombre aparece una marca de tiempo incluyendo la fecha y hora exactas. Los nuevos contenidos del fichero se escriben en un fichero temporal. Al terminar, se borra el fichero original y se renombra el fichero temporal con el nombre del fichero original. El renombrado de ficheros consiste no solo en un cambio de nombre, sino también de ubicación, si se especifica un directorio distinto a aquel en que está ubicado el fichero. Es recomendable, y se puede ver que es muy sencillo, hacer que los programas realicen copias de seguridad de todos aquellos ficheros que vayan a modificar, al menos hasta que se hayan probado lo suficiente, después de lo cual se puede eliminar esta parte del programa.

```
// Cambio de contenidos de ficheros utilizando flujos de lectura y escritura y
// ficheros temporales

package arreglaficheroTexto;

import java.io.File;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Date;
import java.text.SimpleDateFormat;

public class ArreglaFicheroTexto {

    public static void main(String[] args) {

        String nomFichero = "f_texto.txt";
        File f = new File(nomFichero);
        if (!f.exists()) {
            System.out.println("Fichero " + nomFichero + " no existe.");
            return;
        }
        try (BufferedReader bfr = new BufferedReader(new FileReader(f))) {
            File fTemp = File.createTempFile(nomFichero, "");
            System.out.println("Creado fich. temporal " + fTemp.
                getAbsolutePath());
            BufferedWriter bfw = new BufferedWriter(new FileWriter(fTemp));
            String linea = bfr.readLine();
            while (linea != null) { // En resumen, lee de bfr y escribe en bfw
                boolean principioLinea = true, espacios = false,
                    primerAlfab=false;

```

```

    for (int i = 0; i < linea.length(); i++) {
        char c = linea.charAt(i);
        if (Character.isWhitespace(c)) {
            if (!espacios && !principioLinea) {
                bfw.write(c);
            }
            espacios = true;
        } else if (Character.isAlphabetic(c)) {
            if (!primerAlfab) {
                bfw.write(Character.toUpperCase(c));
                primerAlfab=true;
            }
            else bfw.write(c);
            espacios = false;
            principioLinea = false;
        }
    }
    bfw.newLine();
    linea = bfr.readLine();
}
bfw.close();
f.renameTo( // Copia de seguridad
    new File(nomFichero+
        ". "+new SimpleDateFormat("yyyyMMddHHmmss").format(new
            Date())+".bak"
    )
);
fTemp.renameTo(new File(nomFichero));
} catch (IOException e) {
    System.out.println(e.getMessage());
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Los ficheros temporales creados con `createTempFile()` normalmente se crean en un directorio especial para ficheros temporales del sistema operativo (`/tmp` en Linux). Es conveniente borrarlos al final si no se necesitan más (no es el caso en el ejemplo anterior, porque se renombra para pasar a ser un fichero definitivo). Si no se hace, debería ser el propio sistema operativo el que los elimine más adelante, por ejemplo, la próxima vez que se reinicie el sistema o pasado un tiempo. Pero eso depende del sistema operativo y de cómo esté configurado. Algunos ordenadores, como por ejemplo servidores, no se reinician durante periodos muy largos de tiempo.

2.9. Operaciones con ficheros de acceso aleatorio en Java

Para el acceso aleatorio a ficheros se utiliza la clase `RandomAccessFile`.

Las novedades fundamentales que aporta respecto a lo ya visto hasta ahora son:

1. En todo momento el cursor se puede situar en cualquier posición mediante la función `seek()`. En eso consiste el acceso aleatorio.
2. Sobre el fichero se pueden realizar operaciones tanto de lectura como de escritura.

A pesar de estas nuevas posibilidades, las operaciones con ficheros siguen teniendo muchas limitaciones que no vienen del lenguaje Java ni del paquete `java.io`, sino de los sistemas de ficheros habituales. A saber, no es posible eliminar o insertar bloques de *bytes* o de caracteres en mitad de un fichero, ni reemplazar un bloque de un fichero por otro, a no ser que tengan exactamente el mismo tamaño en *bytes*. Para ello habrá que utilizar ficheros temporales auxiliares, de manera similar a como se ha visto en el ejemplo anterior.

En la siguiente tabla se proporciona un resumen de los principales métodos de esta clase. El resto de los métodos se puede consultar en la documentación del paquete `java.io`.

CUADRO 2.9

Principales métodos de la clase `RandomAccessFile`

Método	Funcionalidad
<code>RandomAccessFile(File file, String mode)</code> <code>RandomAccessFile(String name, String mode)</code>	Constructor. Abre el fichero en el modo indicado, si se dispone de permisos suficientes. <i>r</i> : modo de solo lectura. <i>rw</i> : modo de lectura y escritura. <i>rwd</i> , <i>rws</i> : Como <i>rw</i> pero con escritura síncrona. Esto significa que todas las operaciones de escritura (de datos con <i>rwd</i> y de datos y metadatos con <i>rws</i>) deben haberse completado cuando termina la función. Esto puede hacer que la llamada a la función tarde más, pero asegura que no se pierde información crítica ante una caída del sistema.
<code>void close()</code>	Cierra el fichero. Es conveniente hacerlo siempre al final.
<code>void seek(long pos)</code>	Posiciona el puntero en la posición indicada.
<code>int skipBytes(int n)</code>	Intenta avanzar el puntero el número de <i>bytes</i> indicado. Se devuelve el número de <i>bytes</i> que se ha avanzado. Podría ser menor que el solicitado, si se alcanza el fin del fichero.
<code>int read()</code> <code>int read(byte[] buffer)</code> <code>int read(byte[] buffer, int offset, int longitud)</code>	Lee del fichero. Según la variante, un <i>byte</i> o hasta llenar el <i>buffer</i> , o el número de <i>bytes</i> indicados, que se copiarán en la posición indicada (<i>offset</i>) del <i>buffer</i> . Devuelve el número de <i>bytes</i> leídos, o -1 si no se pudo leer nada porque el puntero estaba al final del fichero.
<code>void readFully(byte[] buffer)</code> <code>readFully(byte[] buffer, int offset, int longitud)</code>	Como <code>int read(byte[] b)</code> , pero si no se puede leer hasta llenar el <i>buffer</i> , o el número de <i>bytes</i> indicado, porque se llega al final del fichero, se lanza la excepción <code>IOException</code> . Útil cuando se sabe que se podrá leer hasta llenar el <i>buffer</i> o el número de <i>bytes</i> indicados. En cualquier caso, la eventualidad de que no se pueda completar la lectura se puede gestionar capturando la excepción.
<code>String readLine()</code>	Lee hasta el final de la línea de texto actual.
<code>void write(int b)</code> <code>void write(byte[] buffer)</code> <code>void write(byte[] buffer, int offset, int longitud)</code>	Escribe en el fichero. Según la variante, un <i>byte</i> , o todos los contenidos del <i>buffer</i> , o el número de <i>bytes</i> indicados a partir de la posición indicada (<i>offset</i>). Si alcanza el fin del fichero, siguen escribiendo.

La clase desarrollada en el siguiente ejemplo permite almacenar registros con datos de clientes en un fichero de acceso aleatorio. Los datos de cada cliente se almacenan en un registro, que es una estructura de longitud fija dividida en campos de longitud fija. En este caso, los campos son DNI, nombre y código postal. El constructor de la clase toma una lista con la definición del registro. Cada elemento de la lista contiene la definición de un campo en un par <nombre, longitud>. Los valores de los campos para un registro se almacenan en un `HashMap`, que contiene pares <nombre, valor>, cada uno de los cuales contiene el valor para un campo.

Al constructor se le proporciona un nombre de fichero. Si el fichero no existe, se crea. Si el fichero existe, se calcula el número de registros que contiene, dividiendo la longitud del fichero en *bytes* por la longitud de cada registro.

El método más interesante es `insertar()`. Tiene dos variantes. Si no se le indica la posición, añade el registro al final del fichero. Si no, en la posición que se le indique. La posición del primer registro es 0, no 1. Los textos se almacenan siempre codificados en UTF-8. Como es relativamente habitual en los métodos, no gestionan las excepciones que puede generar (`throws IOException`), y dejan esto para el programa principal.

```
// Almacenamiento de registros de longitud fija en fichero acceso aleatorio
package ficheroaccesoaleatorio;

import java.io.File;
import java.io.RandomAccessFile;
import java.io.IOException;
import java.util.List;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;
import javafx.util.Pair;

public class FicheroAccesoAleatorio {

    private File f;
    private List<Pair<String, Integer> campos;
    private long longReg;
    private long numReg = 0;

    FicheroAccesoAleatorio(String nomFich, List<Pair<String, Integer> campos)
        throws IOException {
        this.campos = campos;
        this.f = new File(nomFich);
        longReg = 0;
        for (Pair<String, Integer> campo: campos) {
            this.longReg += campo.getValue();
        }
        if(f.exists()) {
            this.numReg=f.length()/this.longReg;
        }
    }

    public long getNumReg() {
        return numReg;
    }

    public void insertar(Map<String, String> reg) throws IOException {
        insertar(reg, this.numReg++);
    }
}
```

```

public void insertar(Map<String, String> reg, long pos) throws IOException
{
    try(RandomAccessFile faa = new RandomAccessFile(f, "rwa")) {
        faa.seek(pos * this.longReg);
        for (Pair<String, Integer> campo: this.campos) {
            String nomCampo=campo.getKey();
            Integer longCampo = campo.getValue();
            String valorCampo = reg.get(nomCampo);
            if (valorCampo == null) {
                valorCampo = "";
            }
            String valorCampoForm = String.format("%15s" + longCampo + "s",
                valorCampo);
            faa.write(valorCampoForm.getBytes("UTF-8"), 0, longCampo);
        }
    }
}

public static void main(String[] args) {
    List campos = new ArrayList();
    campos.add(new Pair("DNI", 9));
    campos.add(new Pair("NOMBRE", 32));
    campos.add(new Pair("CP", 5));

    try {
        FicheroAccesoAleatorio faa = new FicheroAccesoAleatorio("fic_acceso_
            aleat.dat", campos);
        Map reg = new HashMap();
        reg.put("DNI", "56789012B");
        reg.put("NOMBRE", "SAMPER");
        reg.put("CP", "29730");
        faa.insertar(reg);
        reg.clear();
        reg.put("DNI", "89012345E");
        reg.put("NOMBRE", "ROJAS");
        faa.insertar(reg);
        reg.clear();
        reg.put("DNI", "23456789D");
        reg.put("NOMBRE", "DORCE");
        reg.put("CP", "13700");
        faa.insertar(reg);
        reg.clear();
        reg.put("DNI", "78901234X");
        reg.put("NOMBRE", "NADALES");
        reg.put("CP", "44126");
        faa.insertar(reg,1);
    } catch (IOException e) {
        System.err.println("Error de E/S: " + e.getMessage());
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

**Actividades propuestas**

- 2.6.** ¿Qué crees que pasaría si se intenta usar la clase `FicheroAccesoAleatorio` para almacenar un registro en una posición mayor que el número de registros que contiene el fichero? Compruébalo modificando el método `main()` para hacer las pruebas oportunas.
- 2.7.** Completa la clase `FicheroAccesoAleatorio` con un método que permita obtener el valor de un campo de un registro, dada la posición del registro y el nombre del campo. Por ejemplo: se podría acceder al valor del campo "Nombre" del registro situado en la posición 5 con `obtenValorCampo(4, "Nombre")`. Por coherencia con la manera en que se ha implementado el método `insertar()`, la posición del primer registro debe entenderse que es 0, no 1. `String` debe tener el mismo formato que en la línea siguiente. Se recomienda utilizar el constructor `String(byte[] bytes, Charset charset)` de `String`.

2.10. Organizaciones de ficheros

Una organización de ficheros es una manera de organizar y estructurar los datos dentro de los ficheros, de manera que se puedan interpretar correctamente sus contenidos. El último programa de ejemplo para acceso aleatorio a ficheros almacena los datos de clientes en registros de longitud fija, compuestos por campos de longitud fija. Esta es una organización muy habitual. También es muy habitual que exista un campo, o un conjunto de campos, que identifique cada registro, de manera que no pueda existir más de un registro con los mismos valores para la clave. Se denomina *clave* a este campo o conjunto de campos. En el ejemplo anterior, en el que cada registro almacena los datos de un cliente, la clave sería el DNI del cliente.

Algunas organizaciones de ficheros pueden incluir estructuras complementarias en el propio fichero o en ficheros auxiliares. Por ejemplo, se podría incluir en el propio fichero un bloque de cabecera antes de la secuencia de campos o un bloque de cierre después. En dichos bloques se podría incluir el número de registros que contiene, la longitud de cada registro, la definición de los campos, etc. Algunas organizaciones de ficheros pueden incluir estructuras complementarias en ficheros auxiliares, como, por ejemplo, índices para acelerar las operaciones de consulta.

En definitiva, hay muchísimas posibles organizaciones de los contenidos de un fichero. Pero este apartado se centrará en dos muy sencillas, útiles y representativas: la organización secuencial y la organización secuencial indexada, basadas ambas en registros de longitud fija.

Con todo lo que se ha aprendido en este capítulo, con los ejemplos proporcionados y con la documentación de Java SE 8, se está en perfectas condiciones para desarrollar clases que implementen estas organizaciones de ficheros.

2.10.1. Organización secuencial

Los registros que forman el fichero se almacenan uno tras otro y no están ordenados de ninguna manera. No hay ningún mecanismo para localizar directamente un registro dado o para agilizar las búsquedas. La única manera es leer los registros uno a uno hasta en-

contrar el que se está buscando o hasta llegar al final del fichero. La figura 1.4 muestra los contenidos de un fichero con datos de clientes y con registros de longitud fija, siendo la clave el DNI.

La principal ventaja que tiene esta organización es su sencillez. Según el uso que se le vaya a dar al fichero, esto podría compensar sus inconvenientes, a saber:

- Las búsquedas son muy ineficientes. Para buscar cualquier registro, es necesario recorrer secuencialmente el fichero desde el principio hasta que se encuentre el registro que se busca o hasta llegar al final del fichero sin encontrarlo. Ordenar los registros evitaría tener que recorrer todos los ficheros cuando no está el registro que se busca. Pero ordenar por DNI no servirá de nada si se busca por nombre, por ejemplo. Además, mantener el fichero siempre ordenado complica las operaciones de inserción, borrado o modificación de registros (esto último en el caso en que se modifique el campo por el que está ordenado el fichero).
- El borrado de un registro es muy ineficiente porque obliga a correr una posición hacia atrás todos los registros siguientes. Una posible mejora podría ser marcar el registro como borrado mediante alguna marca especial. Por ejemplo, un carácter determinado en la primera posición, o dejar en blanco los campos que forman la clave, en este caso el DNI.
- La inserción de registros sería muy eficiente. Bastaría añadir el nuevo registro al final. A no ser, como ya se ha visto, que el fichero estuviera ordenado.

2.10.2. Organización secuencial indexada

Esta organización permite búsquedas muy eficientes por cualquier campo que se quiera, a cambio de crear y mantener un fichero de índice para ese campo.

Un fichero de índice no es más que un fichero secuencial ordenado cuyos registros contienen dos campos: uno para un valor del campo para el que se crea el índice, y otro que indica la posición en el fichero secuencial (véase figura 1.5).

Se pueden crear todos los índices que se quiera. Podría crearse uno por DNI, otro por nombre, o por el campo que se quiera. Podrían crearse también índices compuestos por más de un campo.

Utilizando esta organización, no es necesario reorganizar el fichero principal cuando se añaden nuevos registros o se modifican los que ya hay, solo es necesario reorganizar los índices, que son ficheros mucho más pequeños que el fichero principal. El marcar los registros como borrados en lugar de eliminarlos puede hacer innecesario reorganizar los índices cuando se elimina un registro.

El beneficio para las consultas que suponen los índices se consigue a cambio de espacio de almacenamiento y de tener que reorganizar todos los índices cuando se realizan operaciones de inserción, borrado o modificación. Hay que tenerlo en cuenta para evitar crear índices si los beneficios no compensan los inconvenientes.

Resumen

- Un fichero consiste en una secuencia de *bytes*. Un fichero se identifica por su nombre y el directorio donde está situado dentro de la jerarquía de directorios de un sistema de ficheros.
- Todos los sistemas para persistencia de datos almacenan los datos, en última instancia, en ficheros.
- Los sistemas para persistencia de datos basados en ficheros fueron relegados desde la década de los ochenta por los sistemas de bases de datos relacionales.
- Los ficheros pueden ser de texto o binarios. Los ficheros de texto contienen solo texto representado mediante una secuencia de *bytes*. Una codificación de texto es un método que permite representar un texto como una secuencia de *bytes*. La codificación de texto más ampliamente utilizada, y cada vez más, es UTF-8.
- El lenguaje de programación Java proporciona un completo soporte para operaciones con ficheros y directorios en el paquete `java.io`. Cuando se utiliza Java para operaciones con ficheros, y en general para cualquier cosa, hay que gestionar apropiadamente las excepciones que puedan producirse.
- La clase `File` permite acceder a propiedades de directorios y ficheros, así como crearlos, borrarlos, copiarlos y cambiar su ubicación dentro de la jerarquía de directorios de un sistema de ficheros.
- Hay dos formas fundamentales de acceso a ficheros: acceso secuencial y acceso aleatorio. Con acceso secuencial, para leer información en cualquier posición dentro del fichero, es necesario leer antes la información en todas las posiciones anteriores. Con acceso aleatorio es posible leer directamente la información presente en cualquier posición del fichero.
- Para el acceso secuencial en Java se utilizan *streams*. Un fichero es un tipo particular de *stream*. Java proporciona cuatro jerarquías de clases para *streams*. Corresponden a las cuatro combinaciones de valores posibles entre *streams*, por una parte, de entrada y de salida y, por otra, binarios y de texto. Para *streams* binarios las clases de origen de las jerarquías son `InputStream` y `OutputStream`, y para *streams* de texto, `Reader` y `Writer`. Las clases `BufferedInputStream` y `BufferedOutputStream` proporcionan *buffering* para *streams* binarios, lo que permite acelerar las operaciones de entrada y salida, respectivamente. Las clases `BufferedReader` y `BufferedWriter` hacen lo propio para *streams* de texto, y permiten, además, leer y escribir, respectivamente, líneas de texto. Las clases `InputStreamReader` y `OutputStreamWriter` proporcionan recodificación de texto y sirven de enlace entre las jerarquías para *streams* binarios y de texto.
- Para el acceso aleatorio a ficheros, Java proporciona la clase `RandomAccessFile`. Un `RandomAccessFile` solo proporciona operaciones para lectura y escritura de *bytes*, lo que no significa que no se puedan utilizar para el acceso aleatorio a ficheros de texto.
- Las organizaciones más sencillas para almacenar datos en ficheros están basadas en registros de longitud fija, en los que cada registro está a su vez formado por varios campos de longitud fija. En cada registro puede existir un campo clave, de manera que no pueden existir dos registros en el fichero con el mismo valor para el campo clave. La organización más sencilla es la organización secuencial. Para acelerar las búsquedas se pueden utilizar ficheros de índice externos, que permiten acceder a los registros contenidos en el fichero en un orden determinado.



Ejercicios propuestos

Los ejercicios propuestos a continuación se centran en las organizaciones de ficheros y las operaciones para cada una de ellas: inserción, borrado, modificación de ficheros, indexación y ordenación. Para la realización de estas actividades podría ser necesario utilizar clases, métodos u opciones no vistos en el capítulo, por lo que se recomienda consultar la documentación de Java SE 8 (<http://docs.oracle.com/javase/8/docs/api/>).

Nota: cuando se desarrolle una clase, debe crearse un método `main()` que pruebe suficientemente la funcionalidad de la clase. Si se piden modificaciones de una clase existente, las pruebas deberán incidir sobre la funcionalidad añadida o modificada y sobre cualquier otra funcionalidad que haya podido verse afectada por el cambio.

1. Crea una clase que implemente las operaciones para añadir, recuperar y modificar registros de un fichero con organización secuencial. Los nuevos registros se añadirán siempre al final. El fichero debe tener un campo clave, de manera que no se permita que existan dos registros con el mismo valor para el campo clave. Para recuperar un registro se proporcionará el valor de su campo clave. Para modificarlo se proporcionará el valor del campo clave para buscar el registro que se va a modificar, el nombre de un campo y el nuevo valor de ese campo. Ten en cuenta que el hecho de que el registro tenga organización secuencial no significa que no se pueda utilizar acceso aleatorio para él, si esto supone un beneficio para realizar algunas operaciones puedes implementar la clase con una estructura fija de fichero para que sea más sencillo. Por ejemplo: campos DNI y nombre de cliente, campo clave DNI.
2. Basándote en el ejercicio anterior, crea una nueva clase, más genérica, que permita definir la estructura del registro al crear un nuevo fichero, como se hace en un programa de ejemplo para ficheros de acceso aleatorio. Todo funcionará igual, solo que la clase valdrá para cualquier estructura de registro, en lugar de para una estructura fija.
3. Añade un método para borrar registros a la clase anterior. Pero los registros realmente no se borrarán, sino que se marcarán como borrados. Se puede, sencillamente, poner un carácter especial al principio del registro. Cuando se inserte un registro, se insertará, como antes, al final del fichero. Se pueden explorar otras posibilidades para marcar los registros como borrados. Por ejemplo, asignar todo espacios o asignar un valor cero a todos los bytes.
4. Añade a la clase anterior un método para compactar el fichero, es decir, para eliminar los registros marcados como borrados. Habrá que utilizar un fichero temporal para construir el nuevo fichero y, finalmente, sustituir el antiguo por el nuevo.
5. Crea una clase que implemente un fichero secuencial que contenga registros de longitud fija, compuestos por campos de longitud fija, y que siempre esté ordenado por el valor del campo clave. Cuando se inserta un nuevo registro hay que hacerlo en la posición que deba ocupar en el fichero para preservar su ordenación. Se sugiere generar el nuevo fichero en un fichero temporal y sustituir el antiguo por este al final. En el fichero temporal se irán copiando los registros del fichero hasta llegar a uno con un valor mayor para el campo clave. Entonces, se insertará el nuevo registro en el fichero temporal, y después se copiará el resto de los registros en el fichero temporal.

Por último, el fichero actual se sustituirá por el fichero temporal. En este capítulo se han visto programas de ejemplo que utilizan ficheros temporales como ayuda para modificar los contenidos de un fichero.

6. Crea una clase que implemente operaciones para añadir, buscar, borrar y modificar registros de un fichero con organización secuencial indexada. Existirá un único índice para el campo clave del fichero. El índice será un fichero adicional cuyo nombre indique el fichero principal (de datos) y el campo de indexación (que, en este caso, y según se ha dicho, es el campo clave). El índice lo creará el constructor de la clase, junto con el fichero principal. Los registros borrados no se deben borrar, sino marcarse como borrados tanto en el fichero principal como en el fichero de índice. En este último se podría poner un número negativo como posición para indicar que el registro está borrado. Los nuevos registros se añadirán siempre al final del fichero principal. La operación de inserción debe recomponer el índice, para ello hay que insertar una nueva entrada en el lugar apropiado. El índice no es más que un tipo especial de fichero secuencial ordenado, y el fichero principal un fichero secuencial no ordenado, por lo que se pueden utilizar las clases desarrolladas en ejercicios anteriores.

ACTIVIDADES DE AUTOEVALUACIÓN

1. Un fichero:
 - ☐ a) Es una secuencia de bytes.
 - ☐ b) Puede almacenar cualquier tipo de información
 - ☐ c) Está situado en un lugar dentro de una jerarquía de directorios en un sistema de ficheros.
 - ☐ d) Todas las respuestas anteriores son correctas.
2. La codificación de texto UTF-8:
 - ☐ a) Utiliza el mismo número de bytes para representar cualquier carácter.
 - ☐ b) Es compatible con ASCII.
 - ☐ c) No es una codificación de Unicode
 - ☐ d) Es compatible con ISO 8859-1.
3. Un método de clase:
 - ☐ a) No tiene permitido generar excepciones, por lo que cualquiera que el compilador detecte que podría generar debe capturarla y gestionarla.

- ☐ b) Puede generar excepciones, pero no con la sentencia `throw()`, y debe declararlo en su definición con `throws`.
 - ☐ c) Debe declarar con `throws` cualquier tipo de excepción que el compilador detecte que puede generar, y por supuesto cualquiera que lance con `throw()`.
 - ☐ d) Ninguna de las respuestas anteriores es correcta.
4. Se puede inicializar un recurso en la parte de inicialización de recursos de un bloque `try`:
- ☐ a) Siempre, pero si la clase del recurso no tiene definido el método `close()`, dará un error en tiempo de ejecución al cerrar el recurso.
 - ☐ b) Siempre que en el bloque `finally` se le asigne valor `null`.
 - ☐ c) Solo si el recurso es algún tipo de `stream`.
 - ☐ d) Siempre que la clase del recurso implemente `Closeable` o `AutoCloseable`.
5. La clase `File`:
- ☐ a) Como su nombre indica, permite obtener información acerca de ficheros, pero no de directorios.
 - ☐ b) Tiene métodos estáticos que permiten crear ficheros y directorios temporales.
 - ☐ c) Permite consultar información acerca de ficheros y directorios, pero no hacer ningún cambio en ellos.
 - ☐ d) Tiene un método `createNewFile` para crear ficheros y otro `createNewDir` para crear directorios.
6. Con acceso secuencial a ficheros:
- ☐ a) Solo se puede avanzar hacia delante, empezando desde el principio del fichero.
 - ☐ b) Se puede avanzar y retroceder por el fichero, pero siempre *byte a byte*.
 - ☐ c) No se puede acceder a cualquier posición del fichero, pero sí se puede acceder directamente al final.
 - ☐ d) Solo se puede acceder a medios de almacenamiento secuenciales, para el resto hay que acceder obligatoriamente con acceso aleatorio.
7. Con acceso aleatorio a ficheros:
- ☐ a) Es posible insertar un bloque de *bytes* en cualquier posición del fichero, de manera que los contenidos que vienen después se desplacen hacia delante.
 - ☐ b) Es posible escribir un bloque de *bytes* en cualquier posición del fichero, pero, a menos que se haga al final, se sobrescribirá la información presente en el fichero.
 - ☐ c) Es posible insertar un bloque de *bytes* al final o al principio del fichero, pero no en medio del fichero.
 - ☐ d) No se puede utilizar *buffering*.
8. Se puede usar `readLine()` para leer hasta el final de la línea de texto actual:
- ☐ a) Con ficheros de acceso aleatorio (clase `RandomAccessFile`).
 - ☐ b) Con la clase `BufferedReader`.

- ☐ c) Con ficheros de texto con cualquier codificación.
 - ☐ d) Todas las respuestas anteriores son correctas.
9. El puntero utilizado para acceso a un fichero:
- ☐ a) Puede apuntar a un *byte* del fichero, a una posición especial antes del primer *byte* o a una posición especial después del último *byte*.
 - ☐ b) Siempre apunta a un *byte* del fichero.
 - ☐ c) Puede apuntar a un *byte* de fichero o a una posición especial después del último *byte*.
 - ☐ d) Ninguna de las anteriores respuestas es correcta.
10. La recodificación de texto al leer de o escribir a un stream:
- ☐ a) Se puede hacer con `InputStreamReader` y `OutputStreamWriter`.
 - ☐ b) Solo es necesario cuando se lee de o se escribe a un fichero con una codificación que no es una codificación de Unicode.
 - ☐ c) Solo es posible en Java para cuando se lee un texto con una codificación distinta de UTF-8 o se escribe en una codificación distinta de UTF-8.
 - ☐ d) Solo se hace cuando el programa indica explícitamente que se haga.

SOLUCIONES:

- 1. ☐ a ☐ b ☐ c ☐ d
- 2. ☐ a ☒ b ☐ c ☐ d
- 3. ☐ a ☐ b ☒ c ☐ d
- 4. ☐ a ☐ b ☐ c ☒ d

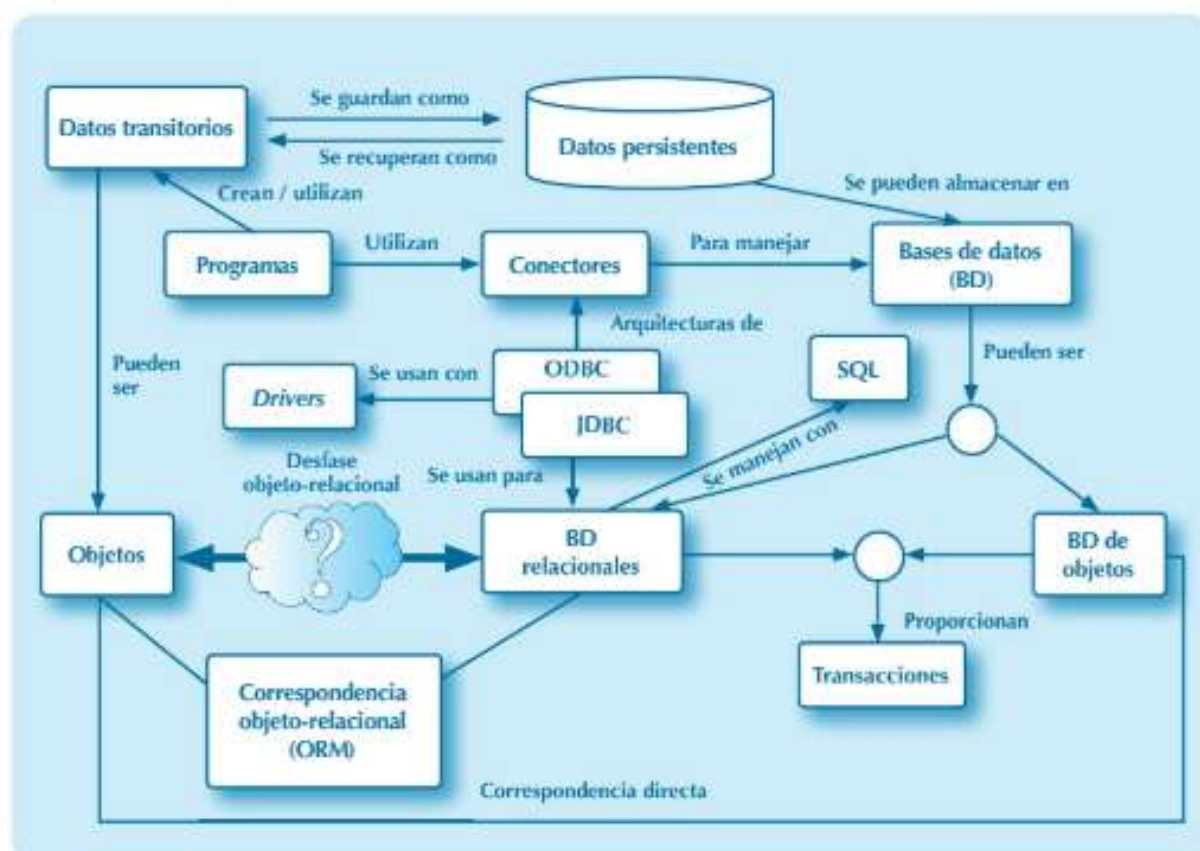
- 5. ☐ a ☒ b ☐ c ☐ d
- 6. ☒ a ☐ b ☐ c ☐ d
- 7. ☐ a ☒ b ☐ c ☐ d
- 8. ☐ a ☐ b ☐ c ☒ d

- 9. ☐ a ☐ b ☒ c ☐ d
- 10. ☒ a ☐ b ☐ c ☐ d

Bases de datos relacionales

Objetivos

- ✓ Conocer las características fundamentales de las API (*application programming interface*) que proporcionan conectores a bases de datos y, en particular, de JDBC (Java Database Connectivity) para bases de datos relacionales.
- ✓ Comprender el desfase objeto-relacional y, en particular, las dificultades para el almacenamiento de objetos complejos en estructuras de almacenamiento basadas en tablas.
- ✓ Abrir una conexión a una base de datos con JDBC utilizando un *driver* de JDBC para ella.
- ✓ Escribir programas en Java utilizando JDBC para ejecutar todo tipo de sentencias de SQL: de definición de datos (DDL) y de modificación y consulta de datos (DML).
- ✓ Utilizar sentencias preparadas para ejecutar sentencias de SQL de manera segura y eficiente.
- ✓ Trabajar con transacciones para ejecutar atómicamente un grupo de sentencias de SQL.



Pool de conexiones. Conjunto de conexiones a una base de datos que se mantienen abiertas y a disposición de los procesos que puedan necesitarlas, lo que evita el retraso y la sobrecarga que supone la apertura de una nueva conexión cada vez que un proceso necesita una.

Procedimientos y funciones almacenados. Procedimientos y funciones escritos en un lenguaje procedural que es una extensión de SQL, y que se ejecutan en el propio SGBD.

Sentencia preparada. Sentencia de SQL parametrizada que, una vez precompilada en el SGBD, permite su ejecución de manera segura y eficiente.

Transacción. Conjunto de sentencias de SQL que forma una unidad lógica y que se ejecuta de manera atómica y aislada de otras transacciones u operaciones con la base de datos.

3.1. Conectores

Los sistemas gestores de bases de datos (SGBD) de distintos tipos (relacionales, de XML, de objetos o de otros tipos) tienen sus propios lenguajes especializados para operar con los datos que almacenan. En cambio, los programas de aplicación se escriben con lenguajes de programación de propósito general, como por ejemplo Java. Para que los programas de aplicación puedan interactuar con los SGBD, se necesitan mecanismos que permitan a los programas de aplicación comunicarse con las bases de datos en estos lenguajes. Estos se implementan en API y se denominan *conectores*.



Figura 3.1
Interacción con bases de datos utilizando conectores

3.2. Conectores para bases de datos relacionales

Los sistemas de bases de datos más utilizados hoy en día, con mucha diferencia, son los relacionales. Para trabajar con ellos se utiliza SQL. SQL es un lenguaje estándar. Pero existen multitud de bases de datos relacionales distintas, y cada una tiene su propia versión de SQL con sus propias particularidades. Aparte de eso, cada base de datos tiene sus propias interfaces de bajo nivel.

El uso de *drivers* permite desarrollar una arquitectura genérica en la que el conector tiene una interfaz común tanto para las aplicaciones como para las distintas bases de datos, y los *drivers* se ocupan de las particularidades de las distintas bases de datos.

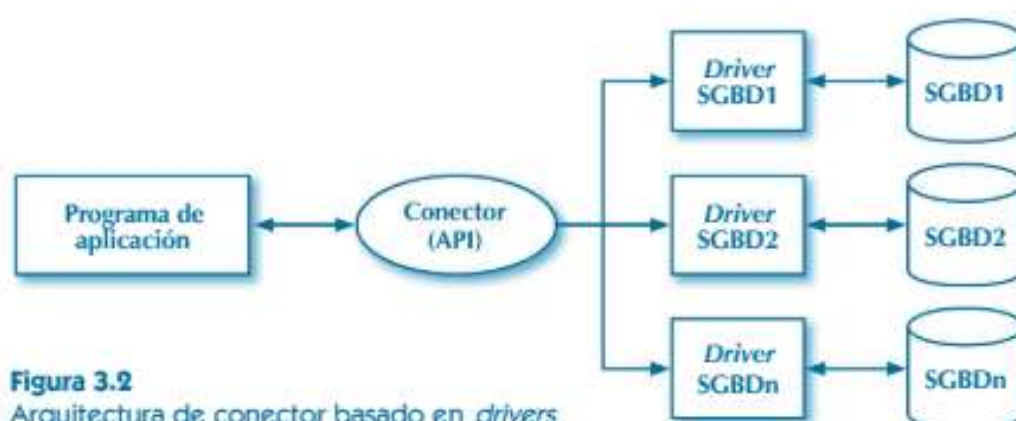


Figura 3.2
Arquitectura de conector basado en *drivers*

De esta manera, el conector no es solo una simple API, sino una arquitectura, porque especifica unas interfaces que los distintos *drivers* tienen que implementar para acceder a las bases de datos particulares. La primera arquitectura de conectores que surgió fue ODBC (Open DataBase Connectivity), desarrollada por Microsoft para Windows a principios de los noventa. ODBC es una API para el lenguaje C, y se usa ampliamente hoy en día tanto en entornos Windows como en Linux y Unix. Con el tiempo surgieron otras arquitecturas como sucesoras de ODBC, tales como OLE-DB y ADO (ActiveX Data Objects). Existen *drivers* para ellas, no solo para bases de datos relacionales, sino también para formatos de ficheros tabulares como hojas de cálculo y ficheros CSV, e incluso para fuentes de datos no relacionales como documentos XML. Estos sucesores de ODBC son API nativas para Windows, y en entornos Windows se utilizan hoy como alternativa o evolución de ODBC. Pero en entornos Linux o Unix, ODBC sigue siendo la principal solución para acceso a bases de datos relacionales desde C.



PARA SABER MÁS

A finales de los años noventa surgió JDBC como el equivalente a ODBC para Java, y es similar en muchos aspectos. De hecho, según se indica en la propia especificación de JDBC, ambos están basados en el estándar X/Open SQL CLI que especifica la manera en que un programa debe enviar sentencias de SQL a un SGBD y operar con los *recordsets* (conjuntos de registros o filas) obtenidos. Este estándar se definió a principios de los noventa y solo para los lenguajes C y COBOL.

Todas las bases de datos importantes proporcionan hoy en día *drivers* de ODBC y de JDBC. Existe un *driver* de JDBC para ODBC que se puede utilizar cuando no se dispone de un *driver* de JDBC específico para una base de datos, pero sí de uno de ODBC.

Existen *drivers* de JDBC no solo para bases de datos relacionales, sino para sistemas de almacenamiento basados en ficheros que almacenan los datos en forma más o menos tabular, como por ejemplo ficheros CSV (ya vistos en capítulos anteriores) y hojas de cálculo, e incluso para XML, que almacena los datos no de forma tabular, sino jerárquica.

Los beneficios que proporcionan los conectores basados en *drivers* —principalmente, independencia de la base de datos— se consiguen a cambio de una mayor complejidad y, en algunos casos, de un menor rendimiento. Existen también API que proporcionan acceso directo a determinadas bases de datos, y están ganando importancia con las aplicaciones web, cada vez más frecuentes, que se ejecutan en un entorno de servidor, que tienen una interfaz de usuario basada en HTML, y a las que se accede a través de un navegador web. Un ejemplo es el lenguaje PHP, que se ejecuta en un módulo de un navegador web Apache, y que tiene API para acceder directamente a bases de datos MySQL, todo funcionando en Linux. Este entorno se denomina con las siglas LAMP (Linux, Apache, MySQL, PHP). Para el acceso a bases de datos MySQL desde PHP se dispone de la extensión MySQLi, que proporciona una API con dos interfaces distintas: una procedural y otra orientada a objetos. También existe para PHP una API de conectores con una interfaz orientada a objetos y basada en *drivers*, llamada PDO (Portable Data Objects), con *drivers* para MySQL, Oracle y PostgreSQL, entre otras.

Recursos web

www

Más información acerca de las API MySQLi y PDO para PHP en la web PHP.net:

<http://php.net/manual/es/book.mysqli.php>

<http://php.net/manual/es/book.pdo.php>

En cualquier caso, el uso de un conector para una base de datos relacional es siempre igual en lo esencial, y conforme al estándar X/Open SQL CLI.

3.3. Acceso a resultados de consultas sobre bases de datos relacionales mediante conectores

Los conectores permiten realizar todo tipo de operaciones sobre una base de datos relacional. Pero este apartado se centrará en las operaciones de consulta.

La cuestión fundamental que se plantea con los conectores es la correspondencia entre las estructuras de datos utilizadas para el almacenamiento en la base de datos y las estructuras de datos de las que dispone el lenguaje de programación. En el caso de las bases de datos relacionales, la estructura de datos fundamental para el almacenamiento de la información es la tabla. Cada tabla tiene un conjunto fijo de columnas, cada una con un tipo de datos determinado. Una consulta de SQL devuelve un conjunto de filas o *recordset*. Los conectores permiten recuperar estos resultados fila a fila, mediante un objeto que actúa como *iterador* o *cursor*. A continuación, se muestra la manera de realizar una consulta y obtener sus resultados utilizando conectores. Se ha utilizado la sintaxis del lenguaje Java y las clases de JDBC, pero en esencia es igual para cualquier base de datos relacional y para cualquier lenguaje de programación.

```

Connection c = getConnection(datos de conexión)
Statement s = c.createStatement();
ResultSet rs = s.executeQuery("SELECT... ");
while(rs.next()) { //En rs están disponibles más resultados de la consulta
    String dato1 = rs.getString(1); // obtener String de primera columna
    int dato2 = rs.getInt(2); // obtener int de segunda columna
}
s.close();
c.close();

```

Figura 3.3

Obtención de resultados de una consulta mediante iteradores

El objeto de tipo `ResultSet` actúa como iterador sobre los resultados de la consulta, que son un conjunto de filas. Una vez recuperada una fila, se puede acceder a cada dato indicándole su posición (como en el ejemplo anterior) o su nombre (como se verá más adelante).

Este modelo de acceso es válido, con algunas diferencias, para otros tipos de bases de datos, tales como bases de datos de objetos y de XML. Se trata siempre de abrir una conexión, realizar una consulta y utilizar un iterador o cursor para obtener uno a uno los resultados. Según el tipo de base de datos, habrá diferentes operaciones para hacer avanzar el cursor, y se utilizarán diferentes estructuras de datos para recuperar los resultados individuales.

3.4. Desfase objeto-relacional

Hacer a la inversa que en el apartado anterior, es decir, almacenar los resultados de variables de memoria en una base de datos relacional, puede ser más complicado. Especialmente cuando se trabaja con un lenguaje orientado a objetos y con objetos complejos, es decir, con objetos que contienen referencias a otros objetos, y referencias a colecciones de objetos relacionados. Una colección de objetos complejos tiene estructura de grafo, y no es sencillo almacenar esta información en tablas con filas y columnas. Al conjunto de dificultades que eso plantea se le conoce como *desfase objeto-relacional*, del inglés *object-relational impedance mismatch* o *desajuste de impedancia objeto-relacional* (véase figura 1.9).

Para la persistencia de objetos complejos hay dos posibilidades. Una es utilizar bases de datos de objetos, que permiten almacenar directamente objetos. La otra, utilizar técnicas o herramientas de correspondencia objeto-relacional (ORM). Ambas se verán en temas posteriores.

3.5. Java Database Connectivity

La arquitectura de conectores JDBC para Java está basada en *drivers*, como ya se ha explicado, y su API está disponible en el paquete `java.sql`. Los *drivers* de JDBC proporcionan clases que implementan las interfaces de la API JDBC para una base de datos particular. Como ya se ha comentado también, puede haber *drivers* de JDBC no para una base de datos, sino para otro conector. En particular, existe un *driver* de JDBC para ODBC que se puede utilizar si, para una base de datos, no existe un *driver* de JDBC, pero sí de ODBC.

Recurso web



El siguiente enlace proporciona acceso a un sitio web de Oracle con información general, una breve introducción y tutoriales de JDBC:

<http://docs.oracle.com/javase/tutorial/jdbc/index.html>

3.6. Operaciones básicas con JDBC

En este apartado se verá en detalle cómo se pueden realizar distintos tipos de operaciones sobre bases de datos relacionales con JDBC. Se trata de ejecutar los principales tipos de sentencias de SQL. Se presupone un conocimiento básico del lenguaje SQL.

En SQL se pueden diferenciar varios sublenguajes, y a cada uno de ellos pertenecen varios tipos de sentencias. Se puede diferenciar entre DML (*data manipulation language* o lenguaje de manipulación de datos) y DDL (*data definition language* o lenguaje de definición de datos). Dentro de DML se pueden diferenciar operaciones de consulta y de modificación de datos. Las sentencias de consulta (**SELECT**) se ejecutan con `executeQuery()`, que devuelve una lista de filas en un `ResultSet`, sobre el que se puede iterar para obtener los resultados uno a uno. El resto de las sentencias de DML (**UPDATE**, **DELETE**, **INSERT**) se ejecutan con `executeUpdate()`, que devuelve el número de filas afectadas por la operación. Las sentencias de DDL se ejecutan con `execute()`. En el siguiente esquema se resume todo lo necesario para ejecutar cualquier sentencia de SQL con JDBC.

<code>Class.forName(nombre del driver); // No necesario desde JDBC 4.0 (Java SE 6)</code>		Cargar driver
<code>Connection c = DriverManager.getConnection(datos de conexión);</code>		Crear conexión
<code>Statement s = c.createStatement();</code>		Crear sentencia
<code>ResultSet rs = s.executeQuery(consulta); while(rs.next()) { ... } rs.close();</code>	<code>int res = s.executeUpdate (sent. DML); (o bien) boolean res = s.execute (sent. DDL);</code>	Ejecutar sentencia Si consulta, obtener resultados fila a fila y cerrar lista de resultados
<code>s.close();</code>		Cerrar sentencia
<code>c.close();</code>		Cerrar conexión

3.6.1. Apertura y cierre de conexiones

Los *drivers* de JDBC están disponibles en ficheros de tipo `jar`. Se puede establecer una conexión mediante la clase `DriverManager`, con el método `getConnection(String URL_conexión)`. La URL de conexión contiene un identificador del tipo de base de datos y los datos

necesarios para establecer una conexión con ella. Para MySQL, por ejemplo, tiene la forma `jdbc:mysql:host:puerto/basedatos`, donde `host` es `localhost` si está en el mismo host y `puerto` suele ser 3306. Este método carga automáticamente en memoria las clases para los *drivers* de JDBC de versión 4.0 (incluida en Java SE 6) o posteriores disponibles. Una vez cargados, selecciona el apropiado para la base de datos indicada en la URL de conexión y le pasa esta para que establezca la conexión. Si el *driver* lo consigue, devuelve como resultado una `Connection` y desde entonces se encargará de todas las operaciones realizadas con ella.

El *driver* para MySQL 8.0, por ejemplo, lo proporciona la clase `com.mysql.cj.jdbc.Driver`.

Para añadir un *driver* de JDBC en un IDE como NetBeans o Eclipse, se puede añadir el fichero `jar` al proyecto. Con el *driver* para MySQL 8.0 quedaría como en la figura 3.4.

Si el *driver* no se carga automáticamente en memoria, debe cargarlo el programa a la manera antigua:

```
Class.forName(nombre de la clase);
```

La clase que implementa el *driver* estará en el fichero `jar`. En caso de duda, hay que consultar la documentación del *driver*, que explicará también el formato de la cadena de conexión.

El siguiente programa abre una conexión a una base de datos de MySQL y luego la cierra. El servidor de base de datos es un proceso que escucha en un puerto TCP de un *host*. Para la conexión hace falta indicar el servidor (*host* y *puerto*), el nombre de la base de datos y los datos de autenticación (usuario y contraseña). Se pueden proporcionar en la URL parámetros de conexión adicionales. En el siguiente programa se utilizan variables para todos los datos de conexión, y con ellos se compone la URL de conexión. Para algunas no se indica un valor, o se indica el habitual. Los valores exactos dependen del entorno donde se ejecute el programa.

Para abrir y cerrar los recursos necesarios y poder realizar operaciones con JDBC se utiliza un bloque `try` con recursos. Esto evita tener que cerrar explícitamente recursos con `close()`, a la vez que asegura que se cierran de manera apropiada aunque ocurra alguna excepción.

Las excepciones que se puedan producir en las operaciones con bases de datos utilizando JDBC serán generalmente de la clase `SQLException`. El método `muestraErrorSQL` muestra toda la información relativa a una `SQLException`, y se usará sin cambios en los siguientes programas de ejemplo, por lo que su código se omitirá en ellos.

```
// Apertura y cierre de conexión con JDBC
package jdbc_connection;

import java.sql.DriverManager;
import java.sql.Connection;
import java.sql.SQLException;
```

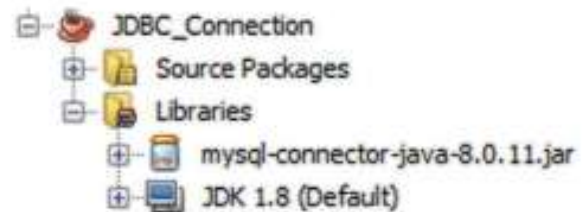


Figura 3.4
Driver de MySQL en proyecto de NetBeans

```

public class JDBC_Connection {
    public static void muestraErrorSQL(SQLException e) {
        System.err.println("SQL ERROR mensaje: " + e.getMessage());
        System.err.println("SQL Estado: " + e.getSQLState());
        System.err.println("SQL código específico: " + e.getErrorCode());
    }

    public static void main(String[] args) {
        String basedatos = "...";
        String host = "localhost";
        String port = "3306";
        String parAdic = "...";
        String urlConnection = "jdbc:mysql://" + host + ":" + port + "/" + basedatos
            + parAdic;
        String user = "...";
        String pwd = "...";

        //Class.forName("com.mysql.jdbc.Driver"); // No necesario desde SE 6.0
        //Class.forName("com.mysql.cj.jdbc.Driver"); // para MySQL 8.0, no necesario

        try (Connection c=DriverManager.getConnection(urlConnection, user,
            pwd)) {
            System.out.println("Conexión realizada.");
        } catch (SQLException e) {
            muestraErrorSQL(e);
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}

```

Recurso digital



En el anexo web 3.1, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás algunas recomendaciones.

3.6.2. La interfaz Statement

La interfaz **Statement** se utiliza para ejecutar cualquier tipo de sentencia SQL. Se puede obtener un **Statement** mediante el método `getStatement()` de **Connection**. En los siguientes apartados se explica cómo ejecutar distintos tipos de sentencias de SQL utilizando esta interfaz y, si se trata de una consulta, cómo recuperar los resultados en un **ResultSet**.

CUADRO 3.1
Métodos de Statement

Método	Funcionalidad
<code>ResultSet executeQuery(String sql)</code>	Ejecuta una consulta (sentencia SELECT de SQL), y devuelve un ResultSet que permite acceder a sus resultados.

[.../...]

CUADRO 3.1 (CONT.)

<code>ResultSet getResultSet()</code>	Obtiene el conjunto de resultados de una consulta <code>SELECT</code> y de otros tipos de sentencias que se verán más adelante, como procedimientos almacenados.
<code>int executeUpdate(String sql)</code>	Se utiliza para realizar operaciones que modifican los contenidos de la base de datos. A saber, sentencias <code>INSERT</code> , <code>UPDATE</code> y <code>DELETE</code> . Devuelve el número de filas afectadas.
<code>boolean execute(String sql)</code>	Se puede utilizar para ejecutar cualquier tipo de consulta. Es el método que hay que utilizar preferentemente para sentencias de DDL (sublenguaje de SQL para definición de datos), tales como <code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> . El valor devuelto depende de la sentencia ejecutada y de sus resultados. Si se trata de una sentencia de DDL, devuelve <code>false</code> . Para ejecutar sentencias de DDL se podría utilizar también el método <code>executeUpdate</code> , y devolvería cero.
<code>void close()</code>	Cierra el <code>Statement</code> .

3.6.3. Ejecución de sentencias de DDL

DDL es el lenguaje de definición de datos. Incluye sentencias para crear, modificar y borrar tablas, vistas y el resto de los objetos que pueden existir en una base de datos relacional.

Las sentencias de DDL se pueden ejecutar con el método `execute()`.

Como ejemplo, el siguiente programa crea, utilizando SQL, una tabla para almacenar datos de clientes. Por supuesto, si se ejecuta una segunda vez, se producirá una excepción. No es necesario llamar al método `close()` ni de `Statement` ni de `Collection`, porque se crean en la parte de inicialización de recursos del bloque `try`.

```
// Ejecución de sentencias de DDL con execute()
package JDBC_create_table;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;

public class JDBC_create_table {

    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión

        try {
            Connection c = DriverManager.getConnection(urlConnection, user,
                pwd);
            Statement s = c.createStatement(); {
                s.execute("CREATE TABLE CLIENTES (DNI CHAR(9) NOT NULL, APELLIDOS
                    VARCHAR(32) NOT NULL, CP CHAR(5), PRIMARY KEY(DNI))");    [...]
```

```

    } catch (SQLException e) {
        muestraErrorSQL(e);
    } catch (Exception e) {
        e.printStackTrace(System.err);
    }
}
}

```

3.6.4. Ejecución de sentencias para modificar contenidos de la base de datos

Estas sentencias se pueden ejecutar con el método `executeUpdate()`, que devolverá el número de filas afectadas por la operación, ya se trate de una sentencia INSERT, UPDATE o DELETE.

Como ejemplo, el siguiente programa añade varias filas con datos de clientes con una sentencia INSERT.

```

// Ejecución de sentencias de modificación de datos con executeUpdate()
package JDBC_insert;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;
public class JDBC_insert {
    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión
        try {
            Connection c = DriverManager.getConnection(urlConnection, user,
                pwd);
            Statement s = c.createStatement();
            int nFil = s.executeUpdate(
                "INSERT INTO CLIENTES (DNI,APELLIDOS,CP) VALUES "
                + "('78901234X','NADALES','44126'),"
                + "('89012345E','HOJAS', null),"
                + "('56789012B','SAMPER','29730'),"
                + "('09876543K','LAMIQUIE', null);");
            System.out.println(nFil + " Filas insertadas.");
        } catch (SQLException e) {
            muestraErrorSQL(e);
            System.err.println("SQL código específico: " + e.getErrorCode());
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}

```



Actividad propuesta 3.1

Haz un programa que haga los cambios necesarios para que los contenidos de la tabla CLIENTES sean los siguientes: ('78901234X', 'NADALES', '44126'), ('89012345E', 'ROJAS', null), ('56789012B', 'SAMPER', '29730'), partiendo de los contenidos de la tabla resultantes de la ejecución del programa anterior. El programa debe utilizar sentencias UPDATE y DELETE.

3.6.5. Ejecución de consultas y manejo de ResultSet

Las consultas son sentencias SELECT. Se pueden ejecutar con `executeQuery()`, que devolverá un `ResultSet` con sus resultados. Un `ResultSet` contiene los resultados de una consulta como un conjunto de filas, y mantiene internamente un cursor o puntero a la fila actual. Hay métodos de `ResultSet` para obtener los datos de las distintas columnas de la fila actual, tanto por posición (por ejemplo, tercera columna) como por nombre de la columna.

`Sentence` tiene un constructor sin parámetros. Los `ResultSet` que se obtienen de las `Sentence` creadas con este constructor solo se pueden recorrer empezando por el primer resultado y avanzando al siguiente con el método `next()`. El siguiente cuadro muestra los métodos disponibles para `ResultSet` de este tipo:

CUADRO 3.2
Métodos de `ResultSet` para consultar contenidos

Método	Funcionalidad
<code>boolean next()</code>	El cursor del <code>ResultSet</code> puede apuntar a cualquier fila suya. Puede, además, apuntar también a una posición especial justo antes de la primera fila y a otra posición especial justo después de la última fila. El cursor apunta inicialmente a la posición especial de antes de la primera fila. Este método mueve el cursor a la siguiente posición y devuelve <code>true</code> , a menos que el cursor esté en la última fila o en la posición especial tras ella, en cuyo caso devuelve <code>false</code> .
<code>getXXX(int)</code> <code>getXXX(String)</code>	Obtiene el contenido de la columna especificada de la fila actual. Se puede especificar una columna por su posición o por su nombre. Hay distintas funciones para distintos tipos: <code>getInt()</code> , <code>getString()</code> , <code>getDate()</code> , etc.
<code>close()</code>	Cierra el <code>ResultSet</code> . Debería hacerse siempre.

Como ejemplo, el siguiente programa muestra los datos de todos los clientes en la tabla. No hace falta utilizar el método `close()` ni del `Statement` ni del `ResultSet`, porque se han creado en la parte de inicialización de recursos del bloque `try`.

```
// Ejecución de una consulta con executeQuery()
package JDBC_select;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class JDBC_select {

    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión
        try {
            Connection c = DriverManager.getConnection(urlConnection, user,
                pwd);
            Statement s = c.createStatement();
```

```

ResultSet rs = s.executeQuery("SELECT * FROM CLIENTES") {
    int i=1;
    while (rs.next()) {
        System.out.println("[ " + (i++) + " ]");
        System.out.println("DNI: " + rs.getString("DNI"));
        System.out.println("Apellidos: " + rs.getString("APELLIDOS"));
        System.out.println("CP: " + rs.getString("CP"));
    }
} catch (SQLException e) {
    muestraErrorSQL(e);
} catch (Exception e) {
    e.printStackTrace(System.err);
}
}
}

```



Actividad propuesta 3.2

El código postal (columna CP) está definido con tipo CHAR(5) en la tabla CUENTES, pero es siempre un número entero. ¿Se podría utilizar `getInt()` en lugar de `getString()` para recuperar su valor? Cambia el programa y verifica tu hipótesis, o justifica los resultados si no son los que esperabas.

Es posible obtener `ResultSet` que permitan una mayor libertad para navegar por sus contenidos. Este tipo de `ResultSet` se llama *scrollable* (que en inglés significa *enrollable* o *desplazable*). Para ello hay que utilizar constructores para `Statement` con parámetros adicionales. Todo lo que se explica a continuación es aplicable igualmente a `PreparedStatement`, que se verá en el siguiente apartado.

CUADRO 3.3
Métodos de `Connection` para obtener `Statement` de tipo *scrollable*

```

Statement createStatement(int tipo, int concurrencia)
PreparedStatement prepareStatement(String sql, int tipo, int
concurrencia).

```

Para el parámetro `tipo` se pueden especificar los siguientes valores:

- `ResultSet.TYPE_FORWARD_ONLY`: el tipo por defecto, utilizado hasta ahora.
- `ResultSet.TYPE_SCROLL_INSENSITIVE`: crea un `ResultSet` de tipo *scrollable* o desplazable, y en el que no se reflejan los cambios realizados por otros procesos en la base de datos.
- `ResultSet.TYPE_SCROLL_SENSITIVE`: crea un `ResultSet` de tipo *scrollable* o desplazable, y sensible a cambios realizados por otros procesos en la base de datos. Quiere esto decir que si algún otro proceso modifica en la base de datos algunos de los datos recuperados en el `ResultSet`, las modificaciones se reflejarán automáticamente en los contenidos del `ResultSet`.

La cuestión es que, si se quiere un `ResultSet` de tipo *scrollable*, hay que decidir si se necesita que sea sensible a cambios realizados por otros procesos en la base de datos. Si no se necesita, lo mejor es seleccionar `ResultSet.TYPE_SCROLL_INSENSITIVE`. Pero, además, hay que decidir un valor para el parámetro concurrencia. Más adelante se verán sus posibles valores. Entre tanto, se puede usar el valor `ResultSet.CONCUR_READ_ONLY`.

CUADRO 3.4

Métodos disponibles adicionalmente para consultar contenidos de `ResultSet` *scrollable*

Método	Funcionalidad
<code>boolean previous()</code> <code>boolean first()</code> <code>boolean last()</code> <code>void beforeFirst()</code> <code>void afterLast()</code> <code>boolean absolute(int pos)</code> <code>boolean isFirst()</code> <code>boolean isLast()</code> <code>boolean isBeforeFirst()</code> <code>boolean isAfterLast()</code> <code>int getRow()</code>	Movimiento del cursor a distintas posiciones y funciones que proporcionan información acerca de la posición del cursor. <code>previous()</code> es la recíproca de <code>next()</code> , es decir, mueve el cursor a la anterior posición y devuelve <code>true</code> , a menos que esté en la primera fila o en la posición especial anterior a ella. <code>first()</code> y <code>last()</code> mueven el cursor a la primera y última fila, y devuelven <code>true</code> a menos que no haya ninguna fila en el <code>ResultSet</code> . Los dos métodos siguientes mueven el cursor a las posiciones especiales antes de la primera fila y después de la última. <code>absolute()</code> mueve el cursor a la posición indicada. Las funciones cuyo nombre empieza por <code>is</code> indican si el cursor está en determinadas posiciones. <code>getRow()</code> devuelve la posición actual del cursor.
<code>getXXX(int)</code> <code>getXXX(String)</code>	Obtiene el contenido de la columna especificada de la fila actual, especificada por posición (<code>int</code>) o por nombre (<code>String</code>). Hay distintas funciones para distintos tipos: <code>getInt()</code> , <code>getString()</code> , <code>getDate()</code> , etc.
<code>close()</code>	Cierra el <code>ResultSet</code> . Debería hacerse siempre.

Actividades propuestas



- 3.3. Haz un programa que muestre los resultados de la misma consulta de SQL del programa anterior pero en orden inverso, del último al primero. La consulta de SQL debe ser la misma, sin ningún cambio.
- 3.4. ¿Cómo se podría averiguar el número de filas obtenidas por una consulta utilizando los métodos de `ResultSet`, pero sin un recorrer sus contenidos para contarlas? Escribe un programa que lo haga. Puedes emplear cualquier consulta.

Con lo visto hasta ahora se está en condiciones de ejecutar prácticamente cualquier sentencia de SQL y, en el caso de consultas, obtener todos sus resultados. Pero falta alguna cosa, como, por ejemplo, las funciones y procedimientos almacenados, que se verán más adelante.

En otro orden de cosas, faltan funcionalidades fundamentales de JDBC tales como sentencias preparadas, transacciones y alguna cosa más, que se verán a continuación. Por último, se verán algunas características algo más avanzadas de JDBC.

3.7. Sentencias preparadas

En los ejemplos vistos hasta ahora las sentencias de SQL eran fijas, estaban en cadenas de caracteres constantes. En una aplicación real suele ser necesario ejecutar sentencias de SQL en las que intervengan variables, bien porque dependan de valores introducidos desde la interfaz de usuario de una aplicación (por ejemplo, un formulario de consulta para clientes con diversos criterios de búsqueda, tales como DNI, nombre, etc.), bien porque dependan de un dato obtenido desde una fuente de datos (como, por ejemplo, un fichero o una consulta en SQL).

Se podría crear la sentencia en un `String`, y asignarle una expresión en la que intervengan variables. Por ejemplo: `String cons= "SELECT * FROM CLIENTES WHERE DNI= '"+dni+"' "`. Pero este planteamiento plantea graves problemas que obligan a descartarlo:

1. *Seguridad.* Una sentencia de SQL construida en tiempo de ejecución utilizando variables está expuesta a ataques mediante técnicas de inyección de SQL. Estas son técnicas para lograr que código SQL hábilmente introducido como parte del contenido de estas variables (`dni` en el ejemplo anterior) se ejecute. Esto permite al atacante consultar datos e incluso modificarlos y borrarlos. El riesgo es especialmente grande cuando los valores se introducen desde una interfaz de usuario, y mayor cuanto más amplia sea la posible base de usuarios. El peor escenario posible sería el acceso universal desde la web. Estos ataques se podrían evitar verificando el contenido de las variables, pero esto resulta engorroso y complejo, y no elimina completamente el riesgo.
2. *Rendimiento.* Una consulta que se envía al SGBD se compila antes de ejecutarse, es decir, se analiza y se crea un plan de ejecución para ella. Si entre una consulta y otra solo cambia el valor de algunas variables, se compilará cada consulta, para obtener siempre el mismo plan de ejecución.

Las sentencias preparadas permiten evitar estos problemas. Una sentencia preparada permite incluir marcadores (*placeholders* en inglés) en determinados lugares de la sentencia donde van valores que se proporcionarán en el momento de ejecutar la sentencia. Una sentencia preparada se proporciona al servidor de base de datos solo una vez, y este la prepara o precompila. A partir de ahí, solo hay que pasar un valor para cada marcador cada vez que se ejecuta la consulta.

Para hacer consultas con sentencias preparadas se utiliza `PreparedStatement`. Se obtiene un `PreparedStatement` con el método `getPreparedStatement` de `Connection`. A este método se le pasa como parámetro la sentencia SQL, y se utiliza el carácter "?" como marcador (*placeholder*).

En el cuadro 3.5 se incluyen los métodos importantes de `PreparedStatement` que no están en `Statement` o que tienen distintos parámetros.

CUADRO 3.5
Métodos de `PreparedStatement`

Método	Funcionalidad
<code>ResultSet executeQuery()</code> <code>int executeUpdate()</code> <code>boolean execute()</code>	Estos tres métodos no tienen como parámetro la consulta, esta se le pasó al constructor.

[.../...]

CUADRO 3.5 (CONT.)

<code>setXXX(int pos, YYYY valor)</code>	Se utilizan para asignar un valor a un placeholder determinado, dado por su posición, siendo 1 la posición del primero. Los hay con distintos nombres, dependiendo del tipo.
<code>setNull(int pos, int tipoSQL)</code>	Asigna valor NULL a una columna. Debe indicarse el tipo. Los posibles tipos están definidos en la clase <code>java.sql.Types</code> .

El siguiente programa de ejemplo inserta en una tabla `CLIENTES1` los datos de tres clientes utilizando una sentencia preparada. Para la inserción de los últimos registros, se ha utilizado un planteamiento que puede ser útil para hacer frente al problema que supone, para la mantenibilidad del código, el identificar por posición los parámetros cuando hay muchos. Si más adelante hay que añadir uno, esto obliga a cambiar sentencias de Java (y podrían ser muchas) solo para incrementar un número. Se evita esto utilizando una variable que se va incrementando. La operación de posincremento `i++` en la última línea no es necesaria, pero evita el riesgo de que se añada un marcador después y se olvide añadir esta operación. Esto podría pasar, e incluso el programa podría funcionar aparentemente bien, y este error podría pasar desapercibido por un tiempo, durante el cual se introducirían datos incorrectos.

La tabla `CLIENTES1` se puede crear en MySQL con: `CREATE TABLE CLIENTES1 (DNI CHAR(9) NOT NULL, APELLIDOS VARCHAR(32) NOT NULL, CP INTEGER, PRIMARY KEY(DNI));`

```
// Ejecución de varias sentencias INSERT de SQL con una sentencia preparada
package JDBC_prepared_statement;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.sql.Types;

public class JDBC_prepared_statement {

    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión

        try {
            Connection c = DriverManager.getConnection(urlConnection, user,
                pwd);
            PreparedStatement sInsert = c.prepareStatement("INSERT INTO
                CLIENTES1(DNI,APELLIDOS,CP) VALUES (?, ?, ?)") {
                sInsert.setString(1, "78901234X");
                sInsert.setString(2, "NADALES");
                sInsert.setInt(3, 44126);
                sInsert.executeUpdate();
                int i = 1;
                sInsert.setString(i++, "89012345E");
                sInsert.setString(i++, "ROJAS");
                sInsert.setNull(i++, Types.INTEGER);
                sInsert.executeUpdate();
            }
        }
    }
}
```

```

        i = 1; sInsert.setString(i++, "56789012B");
        sInsert.setString(i++, "SAMPER");
        sInsert.setInt(i++, 29730);
        sInsert.executeUpdate();
    } catch (SQLException e) {
        muestraErrorSQL(e);
    } catch (Exception e) {
        e.printStackTrace(System.err);
    }
}
}

```



Actividad propuesta 3.5

Escribe un programa que muestre los datos de varios clientes, o todos, de la tabla **CLIENTES1**. El programa debe utilizar una sentencia preparada para la consulta **SELECT * FROM CLIENTES1 WHERE DNI=?**. Debe realizarse una consulta para cada cliente, especificando su DNI, y obtener los datos del **ResultSet** resultante, que solo tendrá una fila, al ser el acceso por clave primaria.

3.8. Transacciones

En este apartado se explica cómo utilizar JDBC para ejecutar varias sentencias de SQL como una transacción y gestionar los errores que se puedan producir durante su ejecución.

Una transacción es un conjunto de operaciones que se ejecutan conjuntamente como un todo, y de manera aislada de otras operaciones que pudiera realizar en paralelo otro proceso sobre los mismos datos. Las operaciones que componen una transacción se ejecutan todas completamente, con lo que todos sus cambios se confirman en la base de datos, o bien, si por cualquier motivo no se puede completar, la base de datos queda como si nunca se hubiera empezado a realizar la transacción. Las características de una transacción se resumen en inglés con el acrónimo ACID (*atomic, consistent, isolated, durable*). En el primer capítulo se explicaron en detalle todas estas características, porque se pueden aplicar a cualquier tipo de base de datos, no solo relacionales, y de hecho existen para otros tipos de bases de datos.

Las transacciones son muy importantes y se utilizan muy frecuentemente. Todos los SGBD relacionales proporcionan soporte para transacciones, pero cada uno tiene sus particularidades. Algunos, como Oracle, las tienen habilitadas por defecto, de manera que ningún cambio se confirma en la base de datos hasta que no se ejecuta una sentencia que marca el fin de una transacción (normalmente COMMIT). Otros, como MySQL, las tienen deshabilitadas por defecto, de manera que los cambios realizados por cualquier sentencia se confirman automáticamente aunque no se ejecute una sentencia COMMIT. Además, cada uno utiliza distintas sentencias, o con distinta sintaxis, para la gestión de transacciones. JDBC proporciona una interfaz común para todas las bases de datos. Por supuesto, también es posible ejecutar directamente las sentencias de SQL que controlan las transacciones, pero esto da como resultado una aplicación que solo funciona para una base de datos en particular.

Una transacción se realiza en SQL como sigue (se usa una sintaxis que no tiene por qué corresponder con la del SQL de ninguna base de datos en particular):

```

START TRANSACTION
Operación 1
Operación 2
...
COMMIT

```

Se puede abortar una transacción con la sentencia **ROLLBACK**. Con ello, se descartan todos los cambios, y todo queda como si la transacción nunca se hubiera iniciado.

La propia sentencia **COMMIT** podría fallar, aunque esto es muy inusual. Entonces se descartarían también todos los cambios, como si nunca se hubiera iniciado la transacción. También podría fallar la sentencia **ROLLBACK**.

La interfaz **Connection** tiene varios métodos para realizar estas operaciones y algunos más relacionados con transacciones. Aquí se verá lo necesario para una gestión básica de transacciones que, por otra parte, es más que suficiente en la inmensa mayoría de los casos.

CUADRO 3.6

Métodos de **Connection** relacionados con transacciones

Método	Funcionalidad
<code>void setAutoCommit(boolean autoCommit)</code>	Con <code>autoCommit=false</code> inicia una transacción. En este sentido, equivale a START TRANSACTION .
<code>void commit()</code>	Equivalente a una sentencia COMMIT de SQL. Si después de utilizar este método se quiere ejecutar más sentencias de SQL pero no en una transacción, se puede hacer <code>setAutoCommit(true)</code> .
<code>void rollback()</code>	Descarta todos los cambios realizados por la transacción actual. Se hará normalmente en respuesta a cualquier excepción de tipo SQLException . Esto significa que alguna sentencia de SQL no se ha ejecutado correctamente y entonces, normalmente, se querrá estar seguro de que se aborta la transacción, descartando todos los cambios.

Un fallo durante la ejecución de una sentencia de SQL provoca una excepción de tipo **SQLException**. En ese caso, normalmente se querrá abortar la transacción con `rollback()`.

Como ejemplo se muestra un programa que ejecuta varias sentencias de SQL como una transacción. No se puede agrupar, como en ejemplos anteriores, la creación de la conexión con la creación de la sentencia preparada en el mismo bloque de inicialización de recursos de un bloque **try**, porque estos no están disponibles en el bloque **catch** correspondiente, y entonces la conexión no estaría disponible para hacer `c.rollback()`. Además, hay que hacer `c.rollback()` en un bloque **try ... catch**, porque puede lanzar una **SQLException**.

```

// Ejecución de varias sentencias en una transacción
package JDBC_transacciones;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

```

```

import java.sql.SQLException;
public class JDBC_transacciones {
    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión
        try {
            Connection c = DriverManager.getConnection(urlConnection, user,
                pwd);
            try {
                PreparedStatement sInsert = c.prepareStatement("INSERT INTO
                    CLIENTES1(DNI,APELLIDOS,CP) VALUES (?, ?, ?);");
                c.setAutoCommit(false);
                int i = 0;
                sInsert.setString(++i, "54320198V");
                sInsert.setString(++i, "CARVAJAL");
                sInsert.setString(++i, "10109");
                sInsert.executeUpdate();

                sInsert.setString(i = 1, "76543210S");
                sInsert.setString(++i, "MARQUEZ");
                sInsert.setString(++i, "46987");
                sInsert.executeUpdate();

                sInsert.setString(i = 1, "90123456A");
                sInsert.setString(++i, "MOLINA");
                sInsert.setString(++i, "35153");
                sInsert.executeUpdate();

                c.commit();
            } catch (SQLException e) {
                muestraErrorSQL(e);
                try {
                    c.rollback();
                    System.err.println("Se hace ROLLBACK");
                } catch (SQLException er) {
                    System.err.println("ERROR haciendo ROLLBACK");
                    muestraErrorSQL(er);
                }
            }
        } catch (Exception e) {
            System.err.println("ERROR de conexión");
            e.printStackTrace(System.err);
        }
    }
}

```



Actividad propuesta 3.6

Comprueba las diferencias entre el programa anterior, en el que se agrupan todas las inserciones de registros con transacciones, y otro programa igual pero sin transacciones. Primero hay que hacer una copia del programa y eliminar el código que gestiona las transacciones (`setAutoCommit`, `commit` y `rollback`). Después se trata de probar ambos programas con un mismo conjunto de datos inicial en la tabla CLIENTES1, para comprobar la manera distinta en que se comportan.

El conjunto de datos inicial y el programa podrían ser tales que las dos primeras inserciones se realizaran sin problemas, pero la última no, por haber ya en la tabla un cliente con ese DNI. El programa con transacciones no insertaría ningún registro. El programa sin transacciones insertaría registros hasta que se produjera un error. La creación del conjunto de datos inicial debe automatizarse. Se puede hacer mediante una secuencia de sentencias SQL, que se pueden guardar en un fichero de texto, y ejecutar con el intérprete de SQL, o mediante un programa. En esas secuencias podrían borrarse todos los contenidos de la tabla con una sentencia DELETE y añadirse varias filas con sentencias INSERT.

3.9. Valores de claves autogeneradas

Es una práctica muy habitual crear una tabla de manera que la clave primaria sea una columna numérica y se deje al sistema gestor de base de datos que proporcione un nuevo valor para cada nueva fila que se inserta. Un buen ejemplo es una aplicación que trabaja con facturas. Cada factura debe tener un identificador único, que normalmente es un simple número. El número asignado a una factura no tiene ninguna significación especial. Lo importante es que cada factura tenga un número distinto, y se deja al sistema que asigne un nuevo número consecutivo cada vez que crea una nueva factura. Las claves de este tipo se suelen llamar claves autoincrementales o, en JDBC, claves autogeneradas.

El mecanismo es esencialmente igual en casi todas las bases de datos, con pequeños cambios en la sintaxis del SQL utilizado. Oracle ha tenido históricamente una particularidad al respecto, y es el uso de secuencias para la generación de este tipo de identificadores. Las secuencias son objetos especiales que se crean en la base de datos para este fin. Pero en la versión 12 Oracle introdujo un mecanismo similar al habitual en otras bases de datos. A continuación, se muestra la definición de una tabla FACTURAS con clave autogenerada en MySQL y Oracle. Se define, además, una clave foránea sobre la columna DNI_CLIENTE para que solo se puedan crear facturas para clientes que existen en la tabla CLIENTES.

CUADRO 3.7

Definición de tabla FACTURAS con claves autogeneradas en MySQL y Oracle

MySQL	Oracle 12 o posterior con IDENTITY
<pre>CREATE TABLE FACTURAS (NUM_FACTURA INTEGER AUTO_INCREMENT NOT NULL, DNI_CLIENTE CHAR(9) NOT NULL, PRIMARY KEY(NUM_FACTURA), FOREIGN KEY FK_FACT_DNI_CLIENTES (DNI_CLIENTE) REFERENCES CLIENTES(DNI));</pre>	<pre>CREATE TABLE FACTURAS (NUM_FACTURA INTEGER NOT NULL GENERATED BY DEFAULT AS IDENTITY, DNI_CLIENTE CHAR(9) NOT NULL, CONSTRAINT PK_FACTURAS PRIMARY KEY(NUM_FACTURA), CONSTRAINT FK_FACT_DNI_CLIENTES FOREIGN KEY(DNI_CLIENTE) REFERENCES CLIENTES(DNI));</pre>

Aparte de la tabla FACTURAS, se utilizará en el ejemplo otra tabla LINEAS_FACTURA.

CUADRO 3.8

Definición de tabla LINEAS_FACTURA en MySQL y Oracle

MySQL	Oracle 12
<pre>CREATE TABLE LINEAS_FACTURA(NUM_FACTURA INTEGER NOT NULL, LINEA_FACTURA SMALLINT NOT NULL, CONCEPTO VARCHAR(32) NOT NULL, CANTIDAD SMALLINT NOT NULL, PRIMARY KEY(NUM_FACTURA, LINEA_FACTURA), FOREIGN KEY FK_LINEAFACT_ NUM_FACTURA(NUM_FACTURA) REFERENCES FACTURAS (NUM_FACTURA));</pre>	<pre>CREATE TABLE LINEAS_FACTURA(NUM_FACTURA INTEGER NOT NULL, LINEA_FACTURA SHORTINTEGER NOT NULL, CONCEPTO VARCHAR2(32) NOT NULL, CANTIDAD SHORTINTEGER NOT NULL, CONSTRAINT PK_LINEAS_FACTURA PRIMARY KEY(NUM_FACTURA, LINEA_FACTURA), CONSTRAINT FK_LINEAFACT_NUM_FACTURA FOREIGN KEY(NUM_FACTURA) REFERENCES FACTURAS(NUM_FACTURA));</pre>

Ahora falta saber cómo se puede recuperar, con JDBC, el valor asignado a la clave auto-generada una vez que se ha insertado una nueva fila en una tabla que tiene una clave de este tipo. Esto se puede hacer utilizando algunos métodos de `Statement` y de `Connection` (para obtener `PreparedStatement`).

CUADRO 3.9

Métodos de `Statement` (y de `PreparedStatement`) para claves autogeneradas

Método	Funcionalidad
<pre>int executeUpdate(String sql, int autoGeneratedKeys);</pre>	<code>autoGeneratedKeys</code> puede tomar valores: • <code>Statement.RETURN_GENERATED_KEYS</code> • <code>Statement.NO_GENERATED_KEYS</code> Con el primero se pueden recuperar las claves autogeneradas como se explica a continuación. Normalmente será solo una. Este método no se puede utilizar con <code>PreparedStatement</code>. Una sentencia preparada debe crearse utilizando un método de <code>Connection</code>, como se explica más abajo
<pre>ResultSet getGeneratedKeys();</pre>	Devuelve un <code>ResultSet</code> con los valores de las claves autogeneradas. Normalmente será solo uno. Se puede utilizar el mismo método con <code>PreparedStatement</code>.

CUADRO 3.10

Métodos de `Connection` relacionados con claves autogeneradas

Método	Funcionalidad
<pre>PreparedStatement prepareStatement(String sql, int autoGeneratedKeys);</pre>	Prepara una sentencia. Para poder recuperar valores para claves autogeneradas, hay que indicar el valor <code>PreparedStatement.RETURN_GENERATED_KEYS</code> para <code>autoGeneratedKeys</code>.

El siguiente programa de ejemplo crea una factura con varias líneas. Este ejemplo funciona sin cambios en MySQL, en Oracle 12 y en otras bases de datos que dispongan de un mecanismo similar para claves autogeneradas (todas en general). Si hay alguna diferencia, está en la creación de las tablas, y eso es otra cuestión. Una vez insertada una nueva factura en FACTURAS, inserta sus líneas en LINEAS_FACTURAS, e indica como valor para el número de factura el valor para la clave autogenerada que se acaba de crear para el número de factura. La creación de la factura y de todas sus líneas se realiza conjuntamente como una transacción.

```
// Recuperación de valores para claves autogeneradas. Creación de factura.
package JDBC_claves_autogeneradas;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class JDBC_claves_autogeneradas {

    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión

        try {
            Connection c=DriverManager.getConnection(urlConnection, user, pwd) {
                try {
                    PreparedStatement sInsertFact = c.prepareStatement("INSERT INTO
                        FACTURAS(DNI_CLIENTE) VALUES (?)",PreparedStatement.RETURN_
                        GENERATED_KEYS);
                    PreparedStatement sInsertLineaFact = c.prepareStatement("INSERT
                        INTO LINEAS_FACTURA(NUM_FACTURA,LINEA_FACTURA,CONCEPTO,CANTIDAD)
                        VALUES (?, ?, ?, ?);");
                    c.setAutoCommit(false);

                    int i = 1;
                    sInsertFact.setString(i++, "78901234X");
                    sInsertFact.executeUpdate();
                    ResultSet rs=sInsertFact.getGeneratedKeys();
                    rs.next();
                    int numFact=rs.getInt(1);

                    int lineaFact = 1;
                    i = 1;
                    sInsertLineaFact.setInt(i++, numFact);
                    sInsertLineaFact.setInt(i++, lineaFact++);
                    sInsertLineaFact.setString(i++, "TUERCAS");
                    sInsertLineaFact.setInt(i++, 25);
                    sInsertLineaFact.executeUpdate();

                    i = 1;
                    sInsertLineaFact.setInt(i++, numFact);
                    sInsertLineaFact.setInt(i++, lineaFact++);
                    sInsertLineaFact.setString(i++, "TORNILLOS");
                    sInsertLineaFact.setInt(i++, 250);
                    sInsertLineaFact.executeUpdate();

                    c.commit();
                }
            }
        }
    }
}
```

```

    } catch (SQLException e) {
        muestraErrorSQL(e);
        try {
            c.rollback();
            System.err.println("Se hace ROLLBACK");
        } catch (Exception er) {
            System.err.println("ERROR haciendo ROLLBACK");
            er.printStackTrace(System.err);
        }
    }
    } catch (Exception e) {
        System.err.println("ERROR de conexión");
        e.printStackTrace(System.err);
    }
}
}
}

```

3.10. Llamadas a procedimientos y funciones almacenados

El estándar SQL incluye extensiones procedurales del lenguaje SQL, es decir, lenguajes de programación estructurados basados en SQL, pero que incluyen sentencias condicionales (IF), de asignación e iterativas (bucles). Los procedimientos y funciones almacenados son bloques de código escrito en un lenguaje de este tipo que pueden tener parámetros de entrada, de salida y de entrada-salida, y que, en el caso de las funciones, pueden devolver un valor. Son análogos a los procedimientos y funciones de cualquier lenguaje de programación estructurado (como Java). Se pueden invocar desde un intérprete de SQL y, como no, desde JDBC. Cada SGBD tiene su propio lenguaje de este tipo y su propia sintaxis para definir procedimientos y funciones almacenados, pero son muy similares. JDBC proporciona una interfaz única e independiente de la base de datos para utilizarlos: `CallableStatement`. En este apartado se explicará cómo utilizar esta interfaz, y se desarrollará un ejemplo completo para llamar a un procedimiento almacenado de MySQL.

El procedimiento que se va a utilizar se puede crear de la siguiente manera desde el intérprete de SQL. No es necesario entender los detalles de su implementación. Se trata de entender qué hace, cómo se le pasan valores para sus parámetros y cómo se obtienen los resultados.

```

DELIMITER //
CREATE PROCEDURE listado_parcial_clientes
(IN in_dni CHAR(9), INOUT inout_long INT)
BEGIN
    DECLARE apell VARCHAR(32) DEFAULT NULL;
    SELECT APELLIDOS FROM CLIENTES WHERE DNI=in_dni INTO apell;
    SET inout_long = inout_long + LENGTH(apell);
    SELECT DNI, APELLIDOS FROM CLIENTES
        WHERE APELLIDOS<=apell ORDER BY APELLIDOS;
END //
DELIMITER;

```

A este procedimiento se le pasa un DNI en el parámetro de entrada `in_dni`, y devuelve un conjunto de filas con DNI y APELLIDOS de los clientes por orden alfabético de APELLIDOS, hasta el valor de APELLIDOS para el cliente con el DNI proporcionado. Al valor pasado en el parámetro de entrada y salida `inout_long` se le suma la longitud del valor de APELLIDOS para el cliente con el

DNI pasado en `in_dni`. Este ejemplo tiene todo lo que puede tener un procedimiento almacenado: parámetros de entrada y de entrada-salida, y que devuelva una lista de filas. Lo único que podría faltarle sería un parámetro de solo salida, pero esto no supondría ninguna complicación adicional. Se puede utilizar este procedimiento desde el intérprete de SQL de la siguiente manera:

```
SET @long=0;
CALL listado_parcial_clientes('78901234X', @long);
SELECT @long;
```

Primero, se asigna un valor a la variable `@long`, que se pasa como parámetro de entrada y salida `inout_long`. La llamada al procedimiento con `CALL` muestra las filas devueltas por el procedimiento, y la sentencia `SELECT` del final muestra el valor devuelto en `@long`.

La secuencia de pasos para utilizar el procedimiento con JDBC es la misma, como se verá en breve en un programa de ejemplo. Lo primero es obtener un `CallableStatement` con el método `prepareCall(String patron_llamada)` de `Connection`. `patron_llamada` incluye el nombre del procedimiento o de la función almacenada. Su sintaxis tiene variantes según se trate de un procedimiento o de una función, y según tenga o no parámetros.

CUADRO 3.11

Sintaxis para `patron_llamada` en método `prepareCall`
(`String patron_llamada`) de `CallableStatement`

	Con parámetros	Sin parámetros
Procedimiento	{ call procedimiento(?, ?,...) }	{ call procedimiento }
Función	{ ? = call función(?, ?,...) }	{ call función }

Los métodos de `CallableStatement` que permiten realizar la llamada y obtener los resultados se detallan en el cuadro 3.12.

CUADRO 3.12

Métodos de `CallableStatement` para uso de procedimientos y funciones almacenados

Método	Funcionalidad
<code>setXXX(int pos, YYYY valor)</code>	De manera similar a <code>PreparedStatement</code> , se utilizan para asignar un valor de diversos tipos (las hay con distintos nombres dependiendo del tipo) a un parámetro determinado, identificado por su posición, siendo 1 el primero.
<code>void registerOutParameter(int pos, int sqlType)</code>	Registra un parámetro de salida para poder obtener el valor devuelto en él.
<code>getXXX(int pos)</code>	Obtiene el valor de un parámetro de salida.
<code>ResultSet getResultSet()</code>	Obtiene el <code>ResultSet</code> resultado del procedimiento o función.

En este programa de ejemplo se llama al procedimiento almacenado anterior.

```
// Llamada a procedimiento almacenado en base de datos de MySQL
package JDBC_callable_statement;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.CallableStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class callable_statement {

    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión
        try {
            Connection c = DriverManager.getConnection(urlConnection, user,
                pwd);
            CallableStatement s =
                c.prepareCall("{call listado_parcial_clientes(?,?)}");

            s.setString(1, "78901234X");
            s.setInt(2, 0);
            s.registerOutParameter(2, java.sql.Types.INTEGER);

            s.execute();

            ResultSet rs = s.getResultSet();

            int inout_long = s.getInt(2);
            System.out.println("=> inout_long: "+inout_long);
            int nCli=0;
            while (rs.next()) {
                System.out.println("[ " + (++nCli) + " ]");
                System.out.println("DNI: " + rs.getString("DNI"));
                System.out.println("Apellidos: " + rs.getString("APELLIDOS"));
            }
        } catch (SQLException e) {
            muestraErrorSQL(e);
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}
```



Actividad propuesta 3.7

Creas una función almacenada con MySQL (o con Oracle o con otra base de datos) a la que se le pase el DNI de un cliente y que devuelva sus apellidos. Creas un programa en Java que realice una llamada a ella utilizando JDBC y escriba el valor devuelto por la función. Será necesario consultar documentación o investigar cómo crear funciones almacenadas en la base de datos utilizada, pero se hará de manera muy similar a un procedimiento almacenado.

3.11. Actualizaciones sobre los resultados de una consulta

La interfaz `ResultSet` no solo tiene métodos para consultar los contenidos de una base de datos, sino también para modificar esos contenidos. Con ellos se pueden eliminar, insertar y modificar filas en una tabla. Pero para ello es necesario crear `ResultSet` actualizables que permitan estas operaciones, y especificar algunos parámetros adicionales al crearlos.

Los `ResultSet` actualizables no se utilizan con mucha frecuencia. Normalmente, para hacer cambios en las bases de datos, se emplean sentencias de SQL `UPDATE`, `DELETE` e `INSERT`, y con `UPDATE` y `DELETE` las filas se seleccionan con una cláusula `WHERE`. Pueden ser útiles cuando las condiciones para seleccionar las filas para actualizar o borrar, o los cálculos para los nuevos valores que asignar a los atributos son muy complejos, o si no es posible, o es muy difícil, seleccionar las filas con SQL. Pero casi siempre es posible –y recomendable– utilizar sentencias de SQL separadas para modificaciones en bases de datos relacionales.

CUADRO 3.13

Métodos de `Connection` para obtener `Statement` y `PreparedStatement` actualizables

```
Statement createStatement(int tipo, int concurrencia)
PreparedStatement prepareStatement(String sql, int tipo, int concurrencia)
```

El parámetro `tipo` permite obtener un `ResultSet` de tipo *scrollable*, como ya se ha visto en un apartado previo, en el que se dejaron para más adelante las explicaciones sobre el parámetro `concurrencia`. Pues bien, este es el momento. Sus posibles valores son:

- `ResultSet.CONCUR_READ_ONLY`: el `ResultSet` no es actualizable, es decir, no permite que los cambios realizados en él se graben en la base de datos.
- `ResultSet.CONCUR_UPDATABLE`: el `ResultSet` es actualizable, es decir, los cambios realizados en él se pueden grabar en la base de datos.

En el cuadro 3.14 se muestran los métodos disponibles para los `ResultSet` actualizables.

CUADRO 3.14

Métodos disponibles para actualizaciones con `ResultSet` actualizables

Método	Funcionalidad
<code>void updateXXX(int pos, YYY valor)</code> <code>void updateXXX(String nombre, YYY valor)</code>	Asigna el valor indicado a la columna indicada, que se puede identificar, bien por posición, bien por nombre. XXX es un tipo de JDBC, YYY es un tipo de Java.
<code>void updateNull(int pos)</code> <code>void updateNull(String nombre)</code>	Asigna valor NULL a la columna indicada.
<code>void updateRow()</code>	Graba en la base de datos los contenidos de la fila actual del <code>ResultSet</code> .
<code>deleteRow()</code>	Borra de la base de datos la fila actual del <code>ResultSet</code> .

[.../...]

CUADRO 3.14 (CONT.)

<code>moveToInsertRow()</code>	Mueve el cursor a una posición especial, la fila de inserción (<i>insert row</i>), que sirve para guardar los contenidos de una nueva fila que insertar en la base de datos. Una vez ejecutado este método, se asignan valores a las columnas con <code>updateXXX()</code> y finalmente se inserta la fila en la base de datos con <code>insertRow()</code> . Una vez hecho esto, se puede volver a la fila anterior con <code>moveToCurrentRow()</code> .
<code>void insertRow()</code>	Inserta los contenidos de la fila de inserción en el <code>ResultSet</code> y en la base de datos.

El siguiente programa de ejemplo utiliza las funciones anteriores para modificar el código postal del último cliente recuperado por una consulta, borrar el penúltimo cliente e insertar un nuevo cliente, todo dentro de una transacción.

```
// Modificación de contenidos de una tabla con un ResultSet actualizable
package JDBC_ResultSet_actualizable;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBC_ResultSet_actualizable {
    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión

        try {
            Connection c=DriverManager.getConnection(urlConnection, user, pwd)) {
                try {
                    Statement sConsulta = c.createStatement(
                        ResultSet.TYPE_SCROLL_INSENSITIVE,
                        ResultSet.CONCUR_UPDATABLE)) {

                        ResultSet rs = sConsulta.executeQuery(
                            "SELECT * FROM CLIENTES WHERE CP IS NOT NULL");

                        c.setAutoCommit(false);

                        rs.last(); // Modifica último cliente
                        rs.updateString("CP", "02568");
                        rs.updateRow();

                        rs.previous(); // Borra penúltimo cliente
                        rs.deleteRow();

                        rs.moveToInsertRow(); // Inserta nuevo cliente
                        rs.updateString("DNI", "24862486S");
                        rs.updateString("APELLIDOS", "EURITA");
                        rs.updateString("CP", "33983");
                        rs.insertRow();

                        c.commit();
                    }
                }
            }
        }
    }
}
```

```

    } catch (SQLException e) {
        muestraErrorsSQL(e);
        try {
            c.rollback();
            System.err.println("Se hace ROLLBACK");
        } catch (Exception er) {
            System.err.println("ERROR haciendo ROLLBACK");
            er.printStackTrace(System.err);
        }
    }
} catch (Exception e) {
    System.err.println("ERROR de conexión");
    e.printStackTrace(System.err);
}
}
}

```

Actividad propuesta 3.8



¿Existe alguna diferencia entre `updateXXX(1, null)` y `updateNull(1)`?

3.12. Ejecución de *scripts*

Un *script* de SQL es una secuencia de sentencias de SQL separadas por el carácter ";". Para poder ejecutar *scripts* con MySQL debe establecerse la conexión indicando en la URL de conexión la opción `allowMultiQueries=true`. Con ello se puede ejecutar un *script*, utilizando un `Statement`, exactamente igual que una sentencia de SQL individual. Los *scripts* de SQL son muy utilizados para la instalación de aplicaciones, para crear y poner a punto la base de datos con la que trabajan. Normalmente se prepara en un fichero de texto una secuencia de sentencias de SQL que crean una base de datos, y dentro de ella tablas, vistas y en general todos los objetos necesarios, y añaden algunos datos básicos en determinadas tablas.

3.13. Ejecución de sentencias por lotes

Para ejecutar un número muy grande de sentencias de SQL, puede ser conveniente hacerlo por lotes. Un lote es un conjunto de sentencias de SQL que se envían todas juntas al servidor de bases de datos y se ejecutan todas juntas, en lugar de enviar y ejecutar una a una.

No hay métodos para crear un lote, se crean automáticamente al ejecutar sentencias con el método `addBatch(String sql)` de `Statement` y `addBatch()` de `PreparedStatement`. En un lote creado para `Statement` las sentencias pueden ser de distinto tipo. En un lote creado para `PreparedStatement` son del mismo, porque el lote se crea con una sentencia preparada, pero cada ejecución se hace con un conjunto de parámetros distinto. El lote se ejecuta con `executeBatch()`, que devuelve un `array` de tipo `int[]` con el número de filas afectadas por cada sentencia del lote. Existe también un método `executeLargeBatch()` que devuelve un `array` de tipo `long[]`. También `CallableStatement` admite ejecución por lotes.

El siguiente programa crea un lote para una `PreparedStatement` de tipo `INSERT`, añade varias sentencias y lo ejecuta, todo dentro de una transacción. Se obtienen los datos de un *array* para ilustrar el hecho de que, normalmente, los datos se obtendrán de una fuente de datos mediante un bucle, dado que la ejecución por lotes tiene sentido cuando se agrupa un número grande de sentencias en el lote.

```
// Ejecución de un lote de sentencias preparadas

package JDBC_prepared_statement_en_lote;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class JDBC_prepared_statement_en_lote {
    public static void main(String[] args) {
        (...) // Se omite declaración de variables para los datos de conexión

        String[][] datosClientes = {
            {"13579135G", "MOYA", null},
            {"24680246G", "SILVA", "25865"},
            {"96307418R", "TORRES", "19273"}
        };

        try {
            Connection c = DriverManager.getConnection(urlConnection, user,
                pwd);
            try {
                PreparedStatement sInsert = c.prepareStatement("INSERT INTO
                    CLIENTES1(DNI,APELLIDOS,CP) VALUES (?, ?, ?)");
                c.setAutoCommit(false);
                for (int nCli = 0; nCli < datosClientes.length; nCli++) {
                    for (int i = 0; i < datosClientes[nCli].length; i++) {
                        sInsert.setString(i + 1, datosClientes[nCli][i]);
                    }
                    sInsert.addBatch();
                }
                sInsert.executeBatch();
                c.commit();
            } catch (SQLException e) {
                muestraErrorSQL(e);
            }
            try {
                c.rollback();
            } catch (Exception er) {
                System.err.println("ERROR haciendo ROLLBACK");
                er.printStackTrace(System.err);
            }
        } catch (Exception e) {
            System.err.println("ERROR de conexión");
            e.printStackTrace(System.err);
        }
    }
}
```

Actividad propuesta 3.9



No todas las bases de datos proporcionan soporte para lotes. Se puede saber con `soportaLotes = c.getMetaData().supportsBatchUpdates()`, siendo `c` una `Connection`. Crea un programa a partir de una copia del programa anterior que compruebe si la base de datos proporciona soporte para lotes y escriba esta información en pantalla. Si es el caso, agrupará los cambios en un lote, como hace el programa anterior. Si no, ejecutará directamente las sentencias preparadas. Si tu base de datos tiene soporte para lotes, prueba que funciona bien en ambos casos, por ejemplo, añadiendo solo para una prueba: `soportaLotes = false`.



Recurso digital

En el anexo web 3.2, disponible en www.sintesis.com y accesible con el código indicado en la primera página del libro, encontrarás información sobre *pool* de conexiones.

Resumen

- Un conector es una API que permite acceder, desde programas de aplicación, a los datos almacenados en SGBD.
- Cada tipo de base de datos tiene su propio mecanismo para almacenamiento de los datos y su propio lenguaje para trabajar con ella. Los conectores suelen ser para un tipo particular de bases de datos (relacional, de objetos, de XML). Pero tienen características comunes. Para empezar, proporcionan métodos para abrir una conexión a una base de datos. Una vez hecho esto, permiten ejecutar sentencias en el lenguaje propio de la base de datos (SQL en el caso de las bases de datos relacionales). Proporcionan iteradores para poder acceder a los resultados de una consulta. En concreto, para navegar por el conjunto de resultados y para obtener los datos de cada resultado individual. En el caso concreto de las bases de datos relacionales, las consultas devuelven un conjunto de filas, y los iteradores permiten navegar por el conjunto de filas y obtener los datos de cada fila particular. Si la base de datos proporciona soporte para transacciones, también proporcionan métodos para gestionarlas.
- La API JDBC para Java permite trabajar con bases de datos relacionales. Es más que una API, es una arquitectura, dado que proporciona una interfaz común para los programas de aplicaciones y para el acceso a bases de datos, y requiere *drivers* específicos para el acceso a las bases de datos particulares. Un *driver* hace posible la conexión a una base de datos particular utilizando sus propios protocolos de red e interfaces de bajo nivel, y proporciona la implementación para esa base de datos de todas las interfaces recogidas en la especificación JDBC.

- JDBC permite ejecutar cualquier sentencia de SQL mediante `execute`, `executeUpdate` y `executeQuery`. En el caso de consultas, permite obtener los resultados en un `ResultSet`.
- En el caso de que en la consulta intervengan variables, deben utilizarse siempre sentencias preparadas o `PreparedStatement` por seguridad (para evitar ataques de inyección de SQL). Deben utilizarse también, por eficiencia, siempre que se ejecuten muchas sentencias que son iguales salvo que cambien determinados valores de una a otra.
- JDBC permite ejecutar un conjunto de sentencias de SQL en una transacción de manera sencilla e independiente de la base de datos, y utiliza métodos estándares de JDBC en lugar de sentencias de SQL distintas según la base de datos.
- JDBC permite invocar procedimientos y funciones almacenados de manera independiente de la base de datos, utiliza métodos estándares, y hace abstracción de las diferencias entre distintas bases de datos en lo referente a procedimientos y funciones almacenados.
- JDBC permite ejecutar grupos de sentencias como *scripts*, la base de datos lo permite.
- JDBC permite ejecutar por lotes sentencias de SQL y sentencias de SQL preparadas, si la base de datos lo permite.

Ejercicios propuestos



Hay que utilizar transacciones para las modificaciones de datos siempre que sea apropiado. Para la realización de estos ejercicios puede ser necesario, como en los capítulos anteriores, consultar la documentación de Java SE 8 (<https://docs.oracle.com/javase/8/docs/api/>).

1. Haz un programa que permita navegar de forma interactiva por los contenidos de la tabla `CLIENTES` creada con un programa de ejemplo anterior. Primero, el programa debe realizar una consulta para obtener los contenidos de la tabla, y debe mostrar el mensaje "fila 1" y el contenido de la fila, indicando para cada columna el nombre de la columna y su valor. Después, se deben ejecutar los comandos que se vayan introduciendo por teclado. Si el comando es ".", debe terminar, por supuesto liberando todos los recursos. Si es "k", debe ir a la siguiente fila, indicar el número de la fila y mostrar sus contenidos, como al principio para la primera fila. El comando para ir a la fila anterior será "d". Si se introduce un número, se debe mostrar la fila en la posición indicada por el número. El programa debe mostrar mensajes apropiados en caso de que el comando que se ha introducido no se pueda realizar (por ejemplo, estando en la última fila se pide ir a la siguiente, o se introduce el número de una fila que no existe). La clase `Integer` tiene métodos que permiten determinar si un `String` representa un número entero.

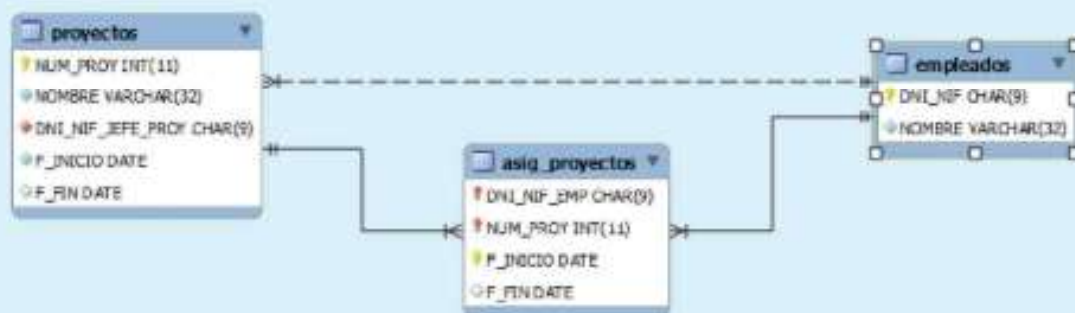
Nota: se puede leer una cadena de caracteres desde el teclado de la siguiente forma: `BufferedReader br=new BufferedReader(new InputStreamReader(System.in)); String comando=br.readLine();`

- Mejora el programa anterior para que se pueda utilizar con cualquier tabla, cuyo nombre se pedirá al iniciarse el programa. Será necesario obtener información acerca de la tabla mediante el método `getMetaData()` de `ResultSet`.

Para algunos ejercicios que siguen, debe utilizarse un conjunto de tablas para representar los proyectos que desarrolla una empresa, sus empleados y las asignaciones de empleados a proyectos. Las tablas se pueden crear en MySQL con las siguientes sentencias (una posibilidad es ejecutarlas con un programa Java con opción de conexión `allowMultiQueries=true`).

```
CREATE TABLE EMPLEADOS(DNI_NIF CHAR(9) NOT NULL, NOMBRE VARCHAR(32) NOT
NULL, PRIMARY KEY(DNI_NIF));
CREATE TABLE PROYECTOS(NUM_PROY INTEGER AUTO_INCREMENT NOT NULL, NOMBRE
VARCHAR(32) NOT NULL, DNI_NIF_JEFE_PROY CHAR(9) NOT NULL, F_INICIO
DATE NOT NULL, F_FIN DATE, PRIMARY KEY(NUM_PROY), FOREIGN KEY
FK_PROY_JEFE(DNI_NIF_JEFE_PROY) REFERENCES EMPLEADOS(DNI_NIF));
CREATE TABLE ASIG_PROYECTOS(DNI_NIF_EMP CHAR(9), NUM_PROY INTEGER NOT
NULL, F_INICIO DATE NOT NULL, F_FIN DATE, PRIMARY KEY(DNI_NIF_EMP,
NUM_PROY, F_INICIO), FOREIGN KEY F_ASIG_EMP(DNI_NIF_EMP) REFERENCES
EMPLEADOS(DNI_NIF), FOREIGN KEY F_ASIG_PROY(NUM_PROY) REFERENCES
PROYECTOS(NUM_PROY));
```

Para mayor claridad, el siguiente diagrama, obtenido con MySQL Workbench, muestra los esquemas de las tablas, incluyendo campos y claves primarias, y las relaciones entre tablas (claves foráneas).



- Crea una clase `GestorProyectos` que contenga métodos para almacenar datos de empleados, proyectos y asignaciones de empleados a proyectos. La clase debe tener métodos para:
 - Crear un nuevo empleado (`nuevoEmpleado`). Debe devolver `true` si el empleado se creó correctamente. A este método se le pueden pasar todos los datos del empleado. Un valor `null` para alguno significa que no se especifica valor. No debe validar los datos que se le pasan. Si el valor indicado para alguno provoca que se lance alguna excepción, este método la propagará. Es decir, su definición debe incluir la opción `throws` con la clase de excepción correspondiente (al menos `SQLException`).

- b) Crear un nuevo proyecto (`nuevoProyecto`). Debe devolver el número de proyecto (`NUM_PROY`). Como el método anterior, tiene parámetros para todos los datos del proyecto (un valor `null` significa que no se especifica valor), no valida los valores proporcionados para ellos, y propaga excepciones. Si se especifica `null` para `F_INICIO`, debe asignarse la fecha actual como fecha de inicio del proyecto, que en MySQL se puede obtener con la función `now()`. Un valor `null` para `F_FIN` significa que no está informada, y debe asignarse un valor `NULL` en la base de datos.
- c) Asignar un empleado a un proyecto (`asignaEmpAProyecto`). Seguir para ello las mismas directrices que para los métodos anteriores, incluyendo las referentes a `F_INICIO` y `F_FIN`.

Debe probarse esta clase mediante un programa de prueba en el método `main()` que cree varios empleados y proyectos, y realice la asignación de algunos empleados a proyectos. No es necesario realizar ninguna verificación de fechas. Por ejemplo: que la fecha de inicio de una asignación de un empleado a un proyecto no es anterior a la fecha de inicio del proyecto, y otras similares. Pero por supuesto se pueden incluir como mejora. Una posibilidad es realizar estas verificaciones en la clase de Java. Otra es realizarlas mediante restricciones de integridad definidas en la propia base de datos o mediante *triggers*.

4. Se trata de hacer algo similar a lo hecho en el anterior ejercicio, pero con un planteamiento algo distinto. Hay que crear clases `Empleado`, `Proyecto` y `AsignacionEmpAProyecto`. Cada una de ellas debe tener un constructor sin parámetros y campos correspondientes a los campos de las correspondientes tablas en la base de datos. Deben tener métodos `getXXX()` y `setXXX()` para cada propiedad. Por ejemplo, para "Empleado" serían `String getDNINIF()`, `void setDNINIF(String DNINIF)`, `String getNombre()` y `void setNombre(String nombre)`. Aparte del constructor sin parámetros, deben tener uno con los parámetros correspondientes a los campos de la clave primaria. Por ejemplo: `Empleado(String DNINIF)`, que lanzará una excepción `SQLException` si no existe en la base de datos una fila para los valores de atributos de la clave primaria proporcionados. Deben tener un método `save()` que guarde el objeto en la base de datos, con una sentencia `INSERT`, si no existe (en el caso de clientes, si no existe ninguno con el DNI), o que modifique los datos, con una sentencia `UPDATE`, si existe. Puedes considerar el uso de `INSERT... ON DUPLICATE KEY UPDATE` en MySQL, o sentencias similares en otras bases de datos.
5. Completa la clase `Proyecto` desarrollada en el ejercicio anterior con un método `getListAsigEmpleados()` que devuelva una lista con los empleados asignados actualmente al proyecto. Un empleado está asignado actualmente a un proyecto si existe para él una fila en `ASIG_PROYECTOS` con `F_INICIO` anterior a la fecha actual (que se puede recuperar en MySQL con `now()`) y con `F_FIN` no informada (es decir, con valor `NULL`) o posterior a la fecha actual.
6. Haz un programa para insertar en una tabla de clientes los datos leídos de un fichero en formato CSV, que debes preparar tú mismo. Los datos de cada cliente deben estar en una línea distinta, y como separador de campos puedes utilizar ";" o "|". Debe crearse una clase `LectorDatosClientes`, con un método `insertaDatosClientes(String nombreFichero, String nombreTabla, String separadorCampos)`.

La tabla debe tener la misma estructura que la tabla de ejemplo `CLIENTES`, y debe estar creada previamente a la ejecución del programa. El parámetro `separadorCampos` indica el separador de campos. Se puede leer el fichero línea a línea utilizando un `BufferedReader`. Se pueden obtener los campos de una línea utilizando la clase `StringTokenizer`, o bien con el método `split()` de `String`. Se debe utilizar una sentencia preparada (`PreparedStatement`) para las operaciones `INSERT`, y deben agruparse todas en una única transacción. Si el carácter separador aparece dos veces seguidas, eso significa que para un campo no se ha especificado un valor, y entonces se le debe asignar el valor `null`. Haz alguna prueba con ficheros cuyos contenidos puedan provocar errores, para asegurarte de que se da el mensaje de error apropiado y no se realiza ningún cambio, al estar todas las operaciones dentro de una transacción. Por ejemplo, especifica un valor nulo para DNI o valores incorrectos para algunos campos.

7. Haz una lista lo más completa posible con condiciones de prueba para el programa anterior. Para cada condición de prueba, indica el resultado esperado. Un ejemplo de condición de prueba sería: "Se especifica valor nulo para tal columna (que no admite valor nulo)", y resultado esperado: "Se muestra un mensaje de error apropiado, se termina la ejecución del programa y no se realiza ningún cambio en la base de datos". Otra sería: "Se especifica un valor alfabético para tal columna (que tiene valores numéricos)", y resultado esperado podría ser el mismo que para la anterior condición de prueba. Verifica cada condición de prueba con un fichero apropiado. No es necesario que el programa proporcione mensajes de error específicos, ni que en las condiciones de prueba se indique un error específico. Normalmente bastará con que el programa gestione las excepciones de tipo `SQLException` y muestre la información que proporciona esta clase de excepciones. Se recomienda redactar las condiciones de prueba sin tener el programa en mente, como lo haría alguien que no sabe cómo está hecho el programa, pero sí qué debe hacer.
8. Si para el ejercicio 6 no has utilizado lotes, modifica el programa para que los utilice. El lote se creará con el método `createBatch()` de `PreparedStatement`. Después se le añadirán todas las sentencias con `addBatch()` y por último se ejecutará el lote con `executeBatch()`.

ACTIVIDADES DE AUTOEVALUACIÓN

1. Un conector es una API que permite a las aplicaciones utilizar bases de datos, pero:
 - ☐ a) Solo utilizando el lenguaje Java.
 - ☐ b) Solo bases de datos relacionales.
 - ☐ c) Solo mediante un *driver* específico para cada base de datos.
 - ☐ d) Ninguna de las respuestas anteriores es correcta.
2. Parte de la problemática del desfase objeto-relacional consiste en que:
 - ☐ a) No siempre es fácil obtener los contenidos de una tabla desde un lenguaje orientado a objetos.

- ☐ b) No siempre es sencillo almacenar los datos contenidos en objetos complejos en un conjunto de tablas y, a la inversa, recuperar esos datos como objetos.
 - ☐ c) Cuando se modifican objetos nunca se utilizan transacciones, pero cuando se modifican los contenidos de una base de datos relacional sí.
 - ☐ d) Todas las respuestas anteriores son correctas.
3. Los *drivers* de JDBC:
- ☐ a) Proporcionan acceso a una base de datos relacional particular, pero a cambio de utilizar SQL estándar, es decir, sin ninguna característica propia de la base de datos.
 - ☐ b) Solo están disponibles para bases de datos relacionales.
 - ☐ c) Proporcionan clases que implementan las interfaces de la especificación JDBC para una base de datos particular.
 - ☐ d) Se pueden comunicar directamente con *drivers* de ODBC para complementar sus funcionalidades.
4. La carga de un *driver* JDBC con `Class.forName(...)`:
- ☐ a) No es necesaria a partir de Java SE 6.
 - ☐ b) Es imprescindible si se quiere que el programa funcione con versiones muy antiguas de Java.
 - ☐ c) Requiere que se indique el nombre de la clase que implementa el *driver*.
 - ☐ d) Todas las respuestas anteriores son correctas.
5. Para ejecutar una sentencia `INSERT` de SQL, lo más aconsejable es utilizar el método:
- ☐ a) `executeQuery()`, porque siempre hay que utilizar este método para cualquier sentencia de SQL que afecte a un conjunto de filas.
 - ☐ b) `execute()`, porque una sentencia `INSERT` solo afecta a una fila, y entonces basta con un boolean para saber si la sentencia insertó una fila o no.
 - ☐ c) `executeUpdate`, que devuelve el número de filas insertadas.
 - ☐ d) `executeQuery`, que devuelve un `ResultSet` con la fila recién insertada.
6. Un `ResultSet`:
- ☐ a) Permite obtener datos de la base de datos, pero no modificarlos.
 - ☐ b) Permite obtener datos de la base de datos, pero no modificarlos, a menos que sea de tipo *scrollable*, en cuyo caso solo se pueden recorrer secuencialmente empezando por la primera fila y avanzando una a una.
 - ☐ c) Permite obtener filas de tablas de la base de datos, modificar y borrar esas filas en la base de datos, y añadir nuevas filas en la base de datos.
 - ☐ d) Proporciona una imagen consistente de los contenidos de la base de datos en el momento de realizar la consulta. Esto significa que los cambios realizados por otros procesos en la base de datos nunca se reflejarán en sus contenidos.
7. A los valores para los campos de la fila actual de un `ResultSet`:
- ☐ a) Se puede acceder tanto por posición como por nombre.
 - ☐ b) Se puede acceder por posición nada más.
 - ☐ c) Se puede acceder por nombre nada más.
 - ☐ d) Se puede acceder por posición, por nombre o tanto por posición como por nombre, según se especifique en los parámetros del método `getResultSet()`.

8. Si una sentencia de SQL solo se va a ejecutar una vez:
- ☐ a) Da igual ejecutarla con un `Statement` o con un `PreparedStatement`, porque solo se ejecutará una vez y el rendimiento es igual.
 - ☐ b) Debe ejecutarse con `PreparedStatement` si para construir la sentencia con `Statement` se necesita utilizar alguna variable de programa. Ejecutar con `Statement` sentencias construidas utilizando variables de programa hace al programa vulnerable ante técnicas de inyección de SQL.
 - ☐ c) Debe ejecutarse siempre utilizando un `PreparedStatement` para evitar ataques por inyección de SQL.
 - ☐ d) Debe ejecutarse siempre utilizando un `PreparedStatement` si se trata de una sentencia que modifique los datos. Las consultas solo leen datos, por lo que no son vulnerables a ataques por inyección de SQL.
9. Un lote creado para un `PreparedStatement`:
- ☐ a) Permite ejecutar sentencias de SQL que sería imposible ejecutar con un lote creado para un `Statement`.
 - ☐ b) Puede tener sentencias SQL de cualquier tipo, siempre que solo tenga sentencias de un tipo (SELECT, INSERT, UPDATE o DELETE).
 - ☐ c) Es para una única sentencia preparada.
 - ☐ d) Puede tener sentencias SQL de cualquier tipo, siempre que sean del mismo tipo y tengan el mismo número de marcadores (*placeholders*).
10. Las claves autogenerated:
- ☐ a) Pueden ser de tipo tanto alfabético como numérico, pero para que puedan ser de tipo alfabético hay que utilizar secuencias.
 - ☐ b) Solo pueden ser de tipo numérico.
 - ☐ c) No existen en Oracle, en su lugar se usan secuencias.
 - ☐ d) Son valores que se generan para un campo clave y que se pueden recuperar con un método de `Connection`.

SOLUCIONES:

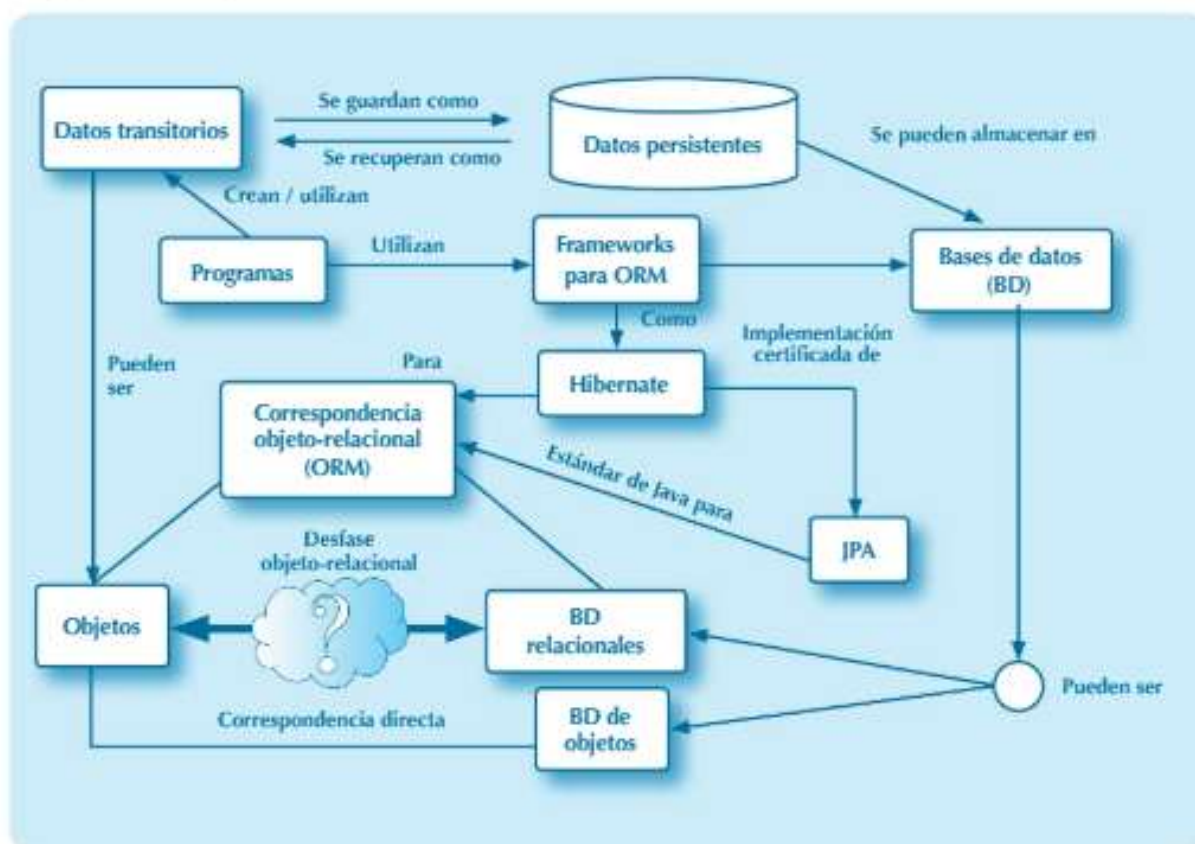
- | | | |
|---|---|--|
| 1. ☐ a ☐ b ☐ c ☒ d | 5. ☐ a ☐ b ☒ c ☐ d | |
| 2. ☐ a ☒ b ☐ c ☐ d | 6. ☐ a ☐ b ☒ c ☐ d | 9. ☐ a ☐ b ☒ c ☐ d |
| 3. ☐ a ☐ b ☒ c ☐ d | 7. ☒ a ☐ b ☐ c ☐ d | 10. ☐ a ☒ b ☐ c ☐ d |
| 4. ☐ a ☐ b ☐ c ☒ d | 8. ☐ a ☒ b ☐ c ☐ d | |

Correspondencia objeto-relacional

Objetivos

- ✓ Comprender el propósito y los fundamentos de la correspondencia objeto-relacional (ORM).
- ✓ Utilizar Hibernate para persistencia de objetos en un esquema relacional mediante ficheros de correspondencia hbm, incluyendo relaciones de uno a uno, de uno a muchos y herencia.
- ✓ Crear POJO (ficheros de clases de Java) apropiados para ORM con Hibernate, crear un esquema relacional para almacenamiento de objetos persistentes, y crear ficheros de correspondencia para establecer la correspondencia entre POJO y tablas del esquema relacional.
- ✓ Analizar el ciclo de vida de un objeto persistente en Hibernate y crear programas en Java que utilicen las principales operaciones disponibles en cada estado.
- ✓ Conocer los fundamentos de los lenguajes HQL y JPQL y crear programas que utilicen HQL.

Mapa conceptual



Glosario

Contexto de persistencia. Lleva el control de los cambios realizados sobre objetos persistentes asociados a él durante una sesión mediante un gestor de entidades, para reflejarlos en la base de datos.

Desfase objeto-relacional. Conjunto de dificultades conceptuales y técnicas para llevar a cabo la persistencia de objetos sobre bases de datos relacionales.

Ficheros de correspondencia o ficheros hbm. Ficheros propios de Hibernate en los que se especifica la correspondencia entre una clase y un esquema relacional (es decir, un conjunto de tablas relacionadas). Es decir, en un fichero de correspondencia se especifica cómo la información contenida en un objeto de una clase determinada se almacena en un esquema relacional.

HQL (Hibernate Query Language), JPQL (Java Persistence Query Language). Lenguajes similares a SQL pero que manejan conjuntos de objetos y sus atributos en lugar de conjuntos de filas y columnas. JPQL es un subconjunto de HQL y es parte de la especificación JPA.

JPA (Java Persistence Architecture). API estándar de Java para ORM.

Objeto persistente. Objeto que tiene una representación persistente en la base de datos. Un objeto persistente está asociado a un contexto de persistencia que detecta los cambios que se realizan sobre él para, en su momento, reflejarlos en la base de datos.

ORM (Object-Relational Mapping). Correspondencia objeto-relacional. Técnicas y herramientas que hacen posible la persistencia de objetos en un esquema relacional, basada en una correspondencia entre clases creadas con un lenguaje de programación orientado a objetos y tablas creadas en una base de datos relacional. La traducción correcta de *mapping* en español es *correspondencia*, y no *mapeo*, aunque esta última es, con mucho, la más habitual.

4.1. Correspondencia objeto-relacional

Los lenguajes orientados a objetos se popularizaron enormemente en los años noventa. C++ estaba disponible desde mediados de los ochenta como una extensión orientada a objetos del lenguaje C. Java se popularizó a mediados de los noventa, especialmente para aplicaciones de gestión empresarial con uso intensivo de bases de datos relacionales. Para entonces las bases de datos relacionales ya tenían un dominio absoluto como medio de almacenamiento de datos, que continúa en la actualidad. El lenguaje Java se ha seguido utilizando para este tipo de aplicaciones y ha sido dominante en el campo de las aplicaciones empresariales de gran envergadura, normalmente desplegadas sobre servidores de aplicaciones basados en Java EE (Java Enterprise Edition).

Bien es cierto que utilizar lenguajes orientados a objetos no significa necesariamente utilizar sus características orientadas a objetos. De hecho, gran parte de estas aplicaciones no lo hacen. Es decir, no utilizan objetos para representar los datos persistentes con los que trabajan, o bien no sacan partido de la herencia y otras posibilidades de estos lenguajes.

En cualquier caso, cada vez más se planteó la conveniencia o incluso la necesidad de almacenar objetos en bases de datos relacionales, incluso objetos complejos con referencias a otros objetos, o a colecciones de objetos, y objetos de clases sobre las que se definen otras subclases. La persistencia de objetos en bases de datos relacionales planteaba una serie de problemas, conocidos conjuntamente como *desfase objeto-relacional* (en inglés *object-relational impedance mismatch*) o *desajuste de impedancia objeto-relacional* (véase la figura 1.9).

Para resolver o para evitar estos problemas surgieron varios planteamientos:

1. *Bases de datos de objetos.* Almacenan directamente objetos. Esta parece en principio la solución ideal o la más natural, y esta idea surgió con fuerza en los noventa, durante el *boom* de la programación orientada a objetos. Pero el desarrollo de estas bases de datos planteaba problemas conceptuales y prácticos. Para empezar, la falta de un modelo formal ampliamente aceptado en el que basarse, al contrario que las bases de datos relacionales, basadas en el modelo relacional. También la falta de estándares ampliamente adoptados, como SQL para las bases de datos relacionales, a pesar del trabajo inicial de ODMG (Object Data Management Group). Este grupo se creó en 1991 y se disolvió en 2001. Hoy en día las bases de datos de objetos siguen teniendo un uso muy reducido.
2. *Bases de datos objeto-relacionales.* Son bases de datos relacionales con capacidades para gestionar objetos. En SQL:99 se introdujeron tipos estructurados definidos por el usuario,

entre ellos los tipos objetos (clases). Son una solución de compromiso y resuelven solo parcialmente el problema.

3. *ORM o correspondencia objeto-relacional*. Consiste en el establecimiento de una correspondencia entre clases definidas en un lenguaje de programación orientado a objetos, como Java, y tablas de una base de datos relacional, y en el uso de mecanismos para que las modificaciones sobre los objetos se registren en la base de datos y, a la inversa, para que se pueda recuperar en objetos la información registrada en la base de datos. Esto hace posible la persistencia de objetos en bases de datos puramente relacionales. La primera herramienta ORM importante que surgió fue Hibernate para el lenguaje Java en 2001, como alternativa a la persistencia de objetos proporcionada de Java EE, basada en EJB (Enterprise Java Beans). Después de Hibernate surgieron otras herramientas y *frameworks* ORM para Java, como Apache Cayenne, Apache OpenJPA, Oracle TopLink, etc. Utilizan en general JDBC para la interacción con la base de datos y funcionan con diferentes bases de datos. También existen soluciones ORM para otros lenguajes orientados a objetos como C++, PHP, Python, etc.

4.2. Hibernate

Hibernate es un *framework* de ORM para Java, distribuido bajo licencia LGPL 2.1 de GNU. Al estar distribuido bajo licencia LGPL, Hibernate se puede utilizar en aplicaciones comerciales sin necesidad de hacer público su código fuente.

www

Recurso web

En el sitio web de Hibernate ORM se puede encontrar el *software* y abundante documentación: <http://hibernate.org/orm/>.

Hibernate se desarrolló en 2001 como una alternativa a la persistencia de objetos proporcionada en Java EE mediante EJB. Java EE es una plataforma muy potente pero compleja y que necesita muchos recursos, pensada para aplicaciones empresariales de envergadura. Como alternativa, se desarrolló Hibernate, un *framework* que proporciona persistencia de objetos basada en ORM, pero sin necesidad de un servidor Java EE. Tiene soporte para transacciones, y utiliza técnicas para mejorar el rendimiento, como por ejemplo *pool* de conexiones, técnicas de *caching* y también *batching* (agrupación en lotes) para las operaciones sobre las bases de datos.

En 2010, Hibernate 3 (versión 3.5.0 y posteriores) se convirtió en implementación certificada de JPA 2.0. JPA es la API estándar de Java para persistencia de objetos, y parte de la especificación Java EE. A partir de entonces, Hibernate siempre ha proporcionado dos API, tanto la suya propia como la de JPA.

Hibernate tiene su propio lenguaje para manejar objetos persistentes, HQL, inspirado en SQL. Las sentencias de SQL operan sobre filas y columnas de tablas, mientras que las de HQL lo hacen sobre conjuntos de objetos persistentes y sus atributos. El lenguaje estándar de JPA para manejo de objetos persistentes, JPQL, es un subconjunto de HQL.

La siguiente ilustración muestra la arquitectura de Hibernate y sus principales componentes. Como ya se ha comentado, Hibernate proporciona tanto una API propia como una implementación de la API de JPA 2.0. En esta ilustración se muestran las clases y componentes de la API propia de Hibernate, pero son muy similares en concepto a los de la API de JPA.

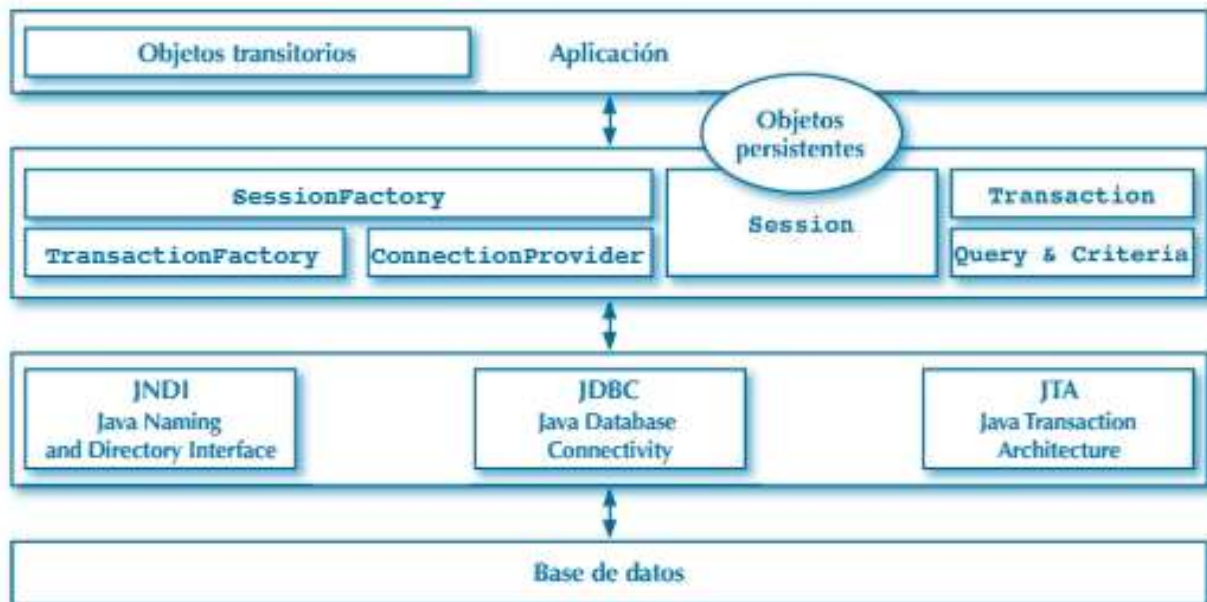


Figura 4.1
Arquitectura de Hibernate

Una aplicación que utiliza Hibernate trabaja con objetos tanto transitorios como persistentes. Los objetos persistentes están siempre asociados a una sesión (**Session**) creada por una **SessionFactory**. Una aplicación puede crear objetos transitorios y después convertirlos en persistentes para que después se almacenen en la base de datos. Puede también recuperar un objeto persistente desde la base de datos y realizar cambios sobre él, para que después se reflejen en la base de datos. Puede agrupar operaciones sobre los objetos persistentes dentro de transacciones (**Transaction**) que están siempre asociadas a una sesión. Puede también utilizar sentencias de HQL mediante la clase **Query**. Hibernate interactúa con la base de datos mediante JDBC. Hibernate puede utilizar internamente API de Java tales como JNDI (Java Naming and Directory Interface) para fuentes de datos (**DataSource**) y JTA (Java Transaction Architecture) para transacciones.

4.3. Iniciación a la correspondencia objeto-relacional con Hibernate

La correspondencia entre objetos de Java y datos en un esquema relacional (es decir, un conjunto de tablas) es compleja si se tienen en cuenta todos los posibles casos. Pero para empezar con lo más sencillo, en general se puede decir que para cada clase existe una tabla en el esquema relacional que permite almacenar los objetos de esa clase.



Figura 4.2

Correspondencia entre una clase de Java sencilla y una tabla de la base de datos

La correspondencia de otras características de las clases es menos directa. A saber:

- Colecciones de objetos, que representan relaciones de uno a muchos o de muchos a muchos entre objetos.
- Referencias a objetos, que representan relaciones de uno a uno o de muchos a uno entre objetos.
- Herencia.

Hibernate permite establecer una correspondencia objeto-relacional en la que basar la persistencia de objetos sobre bases de datos relacionales no solo cuando se parte de cero, sino también cuando se parte de una base de datos ya existente o cuando se quiere añadir persistencia para clases ya creadas. Esto es muy importante porque no siempre se desarrolla una aplicación desde cero, sino que a menudo debe construirse sobre algo ya existente.

Hibernate impone, eso sí, algunas condiciones tanto a las tablas como a las clases. Si no se crean de cero, sino que vienen dadas, y no las cumplen, puede ser necesario un trabajo previo de adecuación.

- Las tablas deben tener clave primaria. En una base de datos relacional no es obligatorio que todas las tablas la tengan, pero en la práctica casi siempre es así. Cuando no existe, se puede crear una nueva clave primaria autogenerada (se habló de ellas en el capítulo previo dedicado a bases de datos relacionales). Esto es sencillo y normalmente no obliga a hacer ningún cambio en las aplicaciones que utilizan la tabla. En correspondencia, las clases deben tener un atributo para almacenar un identificador único.

RECUERDA

- ✓ Un clave primaria autogenerada para una tabla se define sobre una columna numérica, de manera que el sistema gestor de base de datos asigna un nuevo valor para esa columna para cada nueva fila insertada en la tabla. En MySQL se crean con la opción **AUTO_INCREMENT** en la definición de la columna.

- Los valores de la columna o columnas que forman la clave primaria no pueden cambiar. Esto significa, por ejemplo, que si la clave primaria es el DNI, este no se puede cambiar, ni siquiera para corregir errores o para rellenar con ceros por la izquierda, no al menos con una aplicación basada en Hibernate. En la práctica, lo más habitual es tener para

cada tabla una clave primaria autogenerada, lo que evita este tipo de inconvenientes. Si en este caso particular se quisieran evitar duplicidades de DNI, que podrían darse si no es clave primaria, se podría definir un índice único para ese campo, por ejemplo: `CREATE UNIQUE INDEX i_cliente_dni ON cliente(dni).`

- Las clases deben implementar la interfaz `Serializable`. Para ello basta con incluir `implements Serializable` en la definición de la clase.
- Las clases deben cumplir una serie de convenciones tales como la existencia de método `getX` y `setX` (getters y setters) para obtener y asignar, respectivamente, el valor de cada atributo `X`.

TOMA NOTA



Hay que tener muy presentes los problemas que pueden surgir debido a la nomenclatura de las tablas y los campos cuando se usan algunos sistemas operativos. En Windows y MacOS pueden surgir problemas si se usan mayúsculas para los nombres de tablas o de campos. El motivo, en última instancia, es que los datos se almacenan en sistemas de ficheros y su nombre se puede convertir a minúsculas. Para evitar problemas, lo mejor es utilizar nombres sin mayúsculas para las tablas y sus campos y, en general, para cualquier objeto de la base de datos.

La correspondencia se puede establecer mediante ficheros de correspondencia de tipo hbm (*Hibernate mapping*). Estos son ficheros de XML en los que se especifica la correspondencia entre, por una parte, clases y sus atributos, y por otra parte, tablas y sus columnas. También, alternativamente, se puede establecer mediante JPA, por medio de anotaciones en las clases. Se puede incluso utilizar un mecanismo para unas clases y otro para otras.

4.4. Correspondencia objeto-relacional a partir de las tablas

En esta sección se explicará cómo con Hibernate se puede establecer la correspondencia para un conjunto de tablas relacionadas. Algunos IDE o entornos de desarrollo incluyen asistentes para establecer esta correspondencia, si bien puede ser necesario o conveniente ajustar posteriormente algunas cosas manualmente. Se explica-

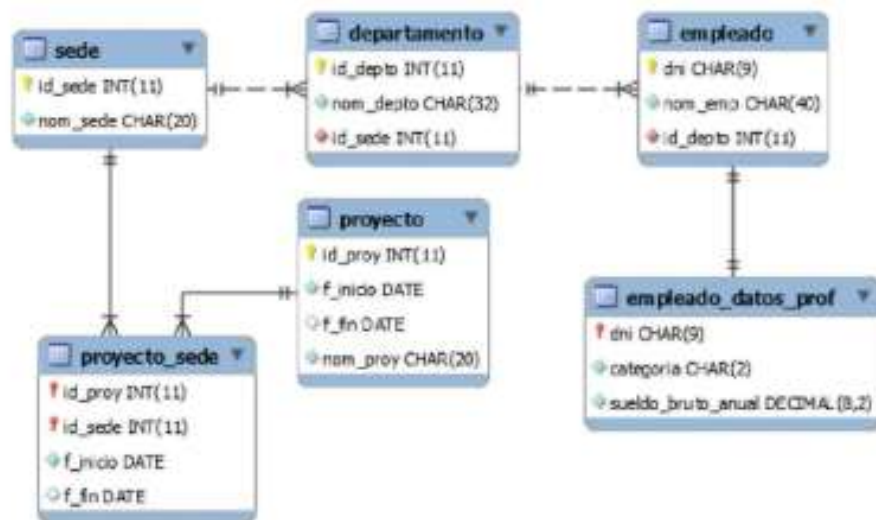


Figura 4.3

Esquema relacional para proyecto de ORM con Hibernate

rá cómo hacerlo en el IDE Netbeans. Se acabará con un pequeño programa de ejemplo que se basa en esta correspondencia para crear varios objetos relacionados y almacenarlos en la base de datos. Los nombres de las tablas se ponen en singular (**departamento** y no **departamentos**). Esto no es estrictamente necesario, pero hace que los nombres generados por Hibernate para la definición de las clases sean claros y descriptivos.

Con las siguientes sentencias de SQL se puede crear, en MySQL, el esquema relacional anterior y un usuario **libro_ad** con los permisos necesarios sobre él. Para ello se puede entrar con el usuario **root**.

```
create database proyecto_orm;
use proyecto_orm;
create table sede(
    id_sede integer auto_increment not null,
    nom_sede char(20) not null,
    primary key(id_sede)
);
create table departamento(
    id_depto integer auto_increment not null,
    nom_depto char(32) not null,
    id_sede integer not null,
    primary key(id_depto),
    foreign key fk_depto_sede(id_sede) references sede(id_sede)
);
create table empleado(
    dni char(9) not null,
    nom_emp char(40) not null,
    id_depto integer not null,
    primary key(dni),
    foreign key fk_empleado_depto(id_depto) references departamento(id_depto)
);
create table empleado_datos_prof(
    dni char(9) not null,
    categoria char(2) not null,
    sueldo_bruto_anual decimal(8,2),
    primary key(dni),
    foreign key fk_empleado_datosprof_empl(dni) references empleado(dni)
);
create table proyecto (
    id_proy integer auto_increment not null,
    f_inicio date not null,
    f_fin date,
    nom_proy char(20) not null,
    primary key(id_proy)
);
create table proyecto_sede (
    id_proy integer not null,
    id_sede integer not null,
    f_inicio date not null,
    f_fin date,
    primary key(id_proy, id_sede),
    foreign key fk_proysede_proy (id_proy) references proyecto(id_proy),
    foreign key fk_proysede_sede (id_sede) references sede(id_sede)
);
CREATE USER 'libro_ad'@'localhost' IDENTIFIED BY '(password)';
GRANT SELECT,INSERT,UPDATE,DELETE,CREATE,DROP,EXECUTE
ON proyecto_orm.* TO 'libro_ad'@'localhost';
```

4.4.1. Creación de la conexión con la base de datos

Lo primero es crear en NetBeans una conexión con la base de datos. Habrá que seleccionar uno de los *drivers* disponibles en la pestaña “Services”, o si ninguno de ellos permite la conexión, habrá que instalar antes un *driver* que la permita. En este caso se quiere establecer una conexión con un servidor MySQL 8.0. Se pueden consultar las propiedades de un *driver* pulsando sobre él con el botón derecho y seleccionando la opción “Customize”. En este ejemplo, se puede ver que el nombre del fichero *jar* para el *driver* con nombre “MySQL (Connector/J driver)” incluye el número de versión 5.1.23.

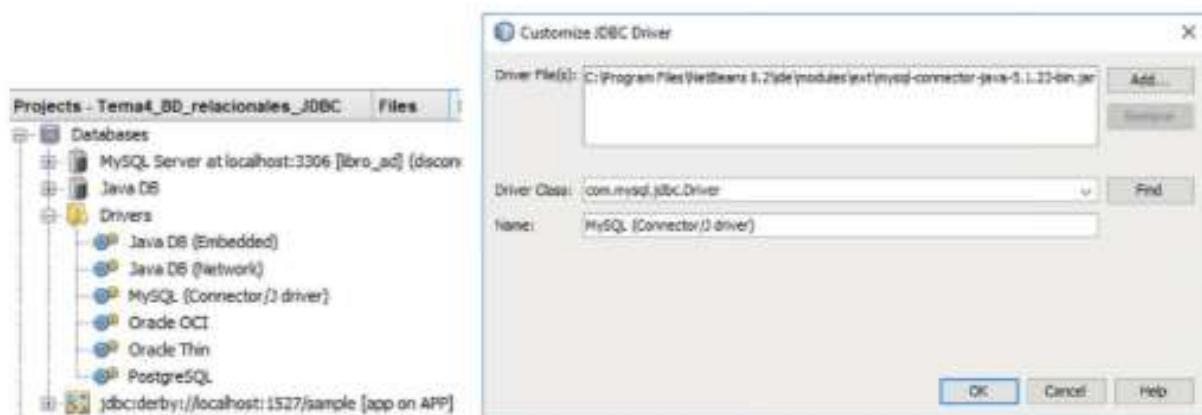


Figura 4.4

Consulta de los *drivers* instalados y de sus propiedades en NetBeans

Ni este *driver* ni ningún otro disponible permite la conexión con MySQL 8.0. Hay que añadir el *driver* apropiado, que viene empaquetado en un fichero *jar* disponible en la sección de descargas del sitio web de MySQL. Para ello, se pulsa con el botón derecho sobre “Drivers” y se selecciona la opción “New driver...”. Por último, se selecciona el fichero *jar*. Hay que pulsar el botón “Find” para localizar la clase que implementa el *driver*. Conviene cambiarle el nombre para diferenciarlo del *driver* para versiones anteriores de MySQL. Se ha elegido “MySQL (Connector/J driver) [8.0]”.

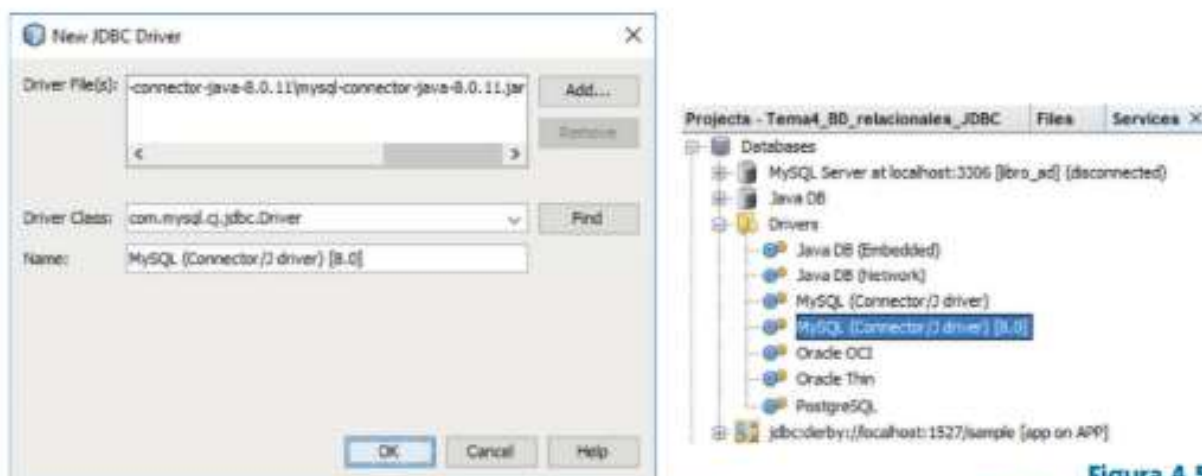


Figura 4.5

Instalación de un nuevo *driver* JDBC en NetBeans

Para crear la conexión a la base de datos, se pulsa con el botón derecho sobre "Databases" y se selecciona la opción "New connection...". Después se selecciona el *driver* y se especifican las opciones de conexión. Para MySQL 8.0 es necesario añadir algunas opciones en la URL de conexión y esta quedaría así:

```
jdbc:mysql://localhost:3306/proyecto_ORM?useUnicode=true&useJDBCCompliantTimezoneShift=true&useLegacyDatetimeCode=false&serverTimezone=UTC
```

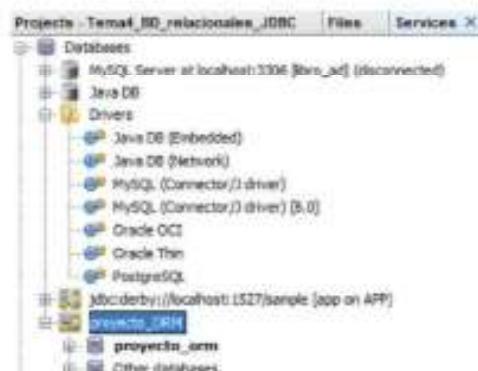
Conviene verificar que la conexión se puede establecer pulsando el botón "Test Connection". Por último, se especifica el nombre de la conexión. En este caso se le pone el nombre **proyecto_ORM**.



Figura 4.6
Datos de conexión para nueva conexión



Figura 4.7
Nombre para nueva conexión



4.4.2. Creación del proyecto

Se trata de crear un proyecto en NetBeans como se ha venido haciendo hasta ahora, con nombre **ORM_conexion**.

Una vez creado el proyecto, se pueden utilizar distintos *wizards* o asistentes para automatizar la creación de todo lo necesario para que el proyecto pueda hacer uso de la persistencia de objetos con Hibernate.

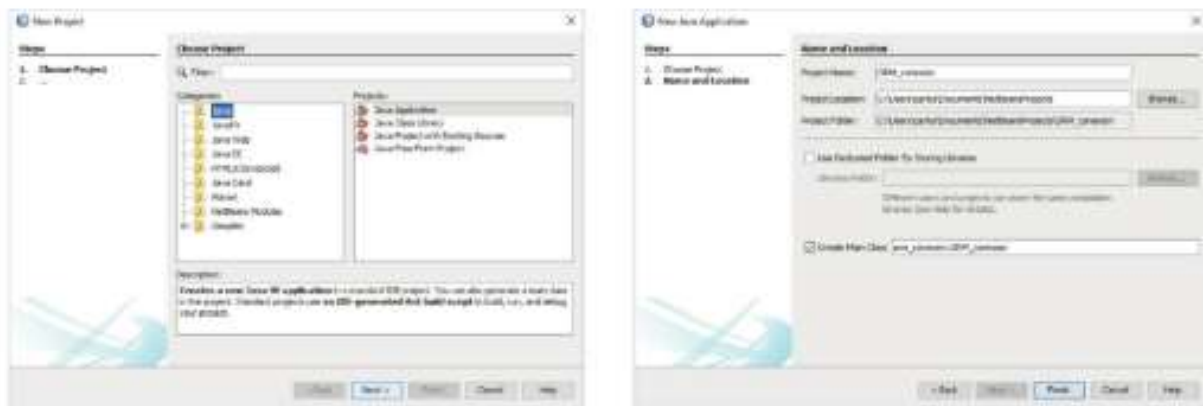


Figura 4.8
Nuevo proyecto en NetBeans

4.4.3. Fichero de configuración de Hibernate: `hibernate.cfg.xml`

Este fichero contiene todo lo necesario para establecer una conexión con la base de datos.

Para crear el fichero de configuración con Netbeans 8.2 se pulsa sobre el proyecto con el botón derecho del ratón y se selecciona la opción “New...”, después “Other...”, y después “Hibernate”. Entonces se muestra una lista con todos los asistentes para Hibernate (figura 4.9).

Se selecciona “Hibernate Configuration Wizard”. En los siguientes diálogos se indica `src` como directorio (*Folder*) para situar el fichero y como conexión la que se acaba de crear en el apartado anterior `proyecto ORM` (figura 4.10). Eso es todo.



Figura 4.9
Asistentes para Hibernate en NetBeans 8.2



Figura 4.10
Asistente para creación de fichero de configuración `hibernate.cfg.xml`

El contenido del fichero de configuración recién creado es este. Como se puede ver, contiene los datos necesarios para establecer una conexión a la base de datos con JDBC.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration
    DTD 3.0//EN" "http://hibernate.sourceforge.net/hibernate-configuration-
    3.0.dtd">
<hibernate-configuration>
```

```

<session-factory> <property name="hibernate.connection.driver_class">
    com.mysql.cj.jdbc.Driver</property>
<property name="hibernate.connection.driver_class">com.mysql.jdbc.
    Driver</property>
<property name="hibernate.connection.url">jdbc:mysql://localhost:3306/
    proyecto_orm?(... URL de conexión, conteniendo opciones
    adicionales)</property>
<property name="hibernate.connection.username">libro_ad</property>
<property name="hibernate.connection.password">(password)</property>
</session-factory>
</hibernate-configuration>

```

4.4.4. Fichero de Ingeniería Inversa **hibernate.reveng.xml**

En este fichero de configuración se especifica para qué clases y tablas se va a establecer la correspondencia, y suele tener por nombre **hibernate.reveng.xml** (figura 4.11).

Se puede crear este fichero con otro asistente para Hibernate. Es necesario especificar el fichero de configuración antes creado y las tablas para las que se va a establecer correspondencia. Como directorio para situar el fichero ("Folder") se indica **src**. Hay que elegir las tablas (figura 4.12).



Figura 4.11
Asistente para creación del fichero de
Ingeniería Inversa **hibernate.reveng.xml**

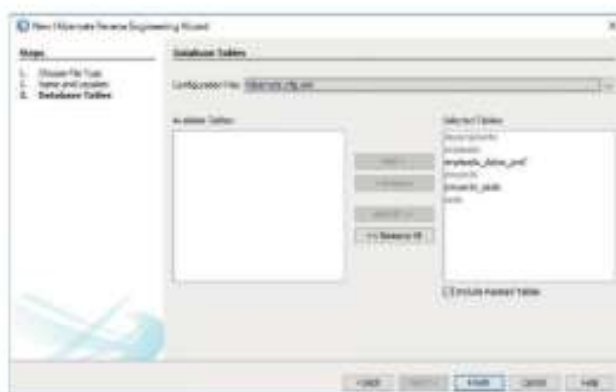


Figura 4.12
Selección de tablas para el fichero de Ingeniería
Inversa

Si se muestra un mensaje de error indicando que los *drivers* no se han añadido al proyecto, hay que añadirlos como se explicó en el capítulo anterior. Si no se ven las tablas que cabría esperar ver, o bien si aparecen junto con el texto (*no primary key*), probablemente se está trabajando en Windows y no se ha tenido la precaución de no utilizar mayúsculas para nombres de tablas y atributos.

A continuación, se muestra el contenido del fichero de ingeniería inversa recién creado. En él se indica la base de datos y las tablas que se van a hacer corresponder con objetos.

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-reverse-engineering PUBLIC "-//Hibernate/Hibernate
    Reverse Engineering DTD 3.0//EN" "http://hibernate.sourceforge.net/
    hibernate-reverse-engineering-3.0.dtd">
<hibernate-reverse-engineering>
    <schema-selection match-catalog="proyecto_orm"/>
    <table-filter match-name="proyecto"/>

```

```

<table-filter match-name="empleado" />
<table-filter match-name="empleado_datos_prof" />
<table-filter match-name="proyecto_sede" />
<table-filter match-name="departamento" />
<table-filter match-name="sede" />
</hibernate-reverse-engineering>

```

4.4.5. POJO (clases) y ficheros de correspondencia

Esta es la parte interesante. Finalmente se generan las clases y los ficheros de correspondencia con el asistente "Hibernate Mapping Files and POJOs from Database". POJO es el acrónimo de *plain old Java file*, algo así como "un fichero de Java de toda la vida, sin más". Son clases que incluyen un atributo por cada columna de la tabla y un método *getter* y *setter* para cada uno, es decir, un método para obtener su valor y otro para asignarlo.

Hay que indicar el fichero de configuración antes creado, el de ingeniería inversa, y el nombre del paquete para incluir los POJO y los ficheros de correspondencia. En este ejemplo se opta por crear un nuevo paquete llamado **ORM**. La opción "EJB 3 Annotations" se puede utilizar para generar los ficheros con anotaciones de JPA. No supone ningún problema generarlos con esas anotaciones aunque no se vaya a utilizar JPA para la correspondencia. Como se verá en breve, en el fichero de configuración **hibernate.cfg.xml** se indicarán los ficheros hbm para la correspondencia de cada clase, con lo que se ignorarán estas anotaciones.

Además de crear las clases (POJO) y los ficheros de correspondencia, este asistente añade las siguientes líneas al fichero de configuración **hibernate.cfg.xml**, para establecer la correspondencia de las clases mediante los correspondientes ficheros hbm. Si para alguna clase se quisiera utilizar anotaciones en lugar de ficheros hbm, habría que eliminar del fichero de configuración la línea correspondiente a su fichero de correspondencia. Más adelante se verá en detalle el contenido de estos ficheros de correspondencia.

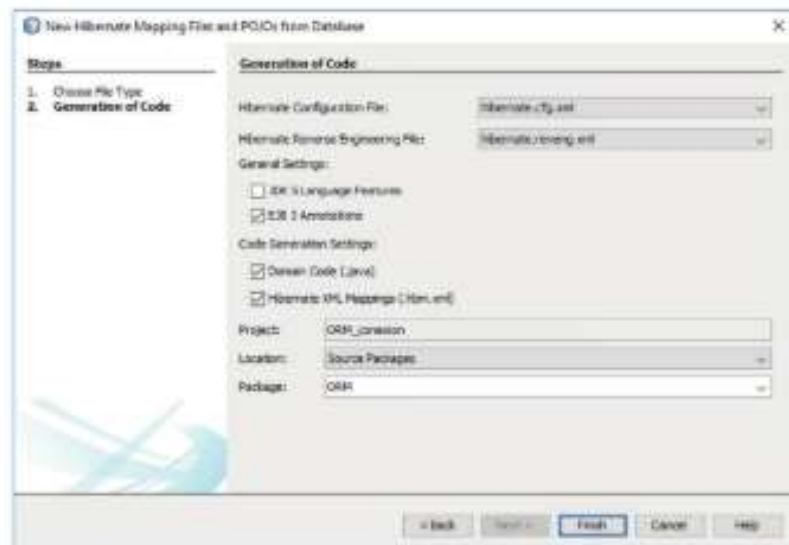


Figura 4.13

Asistente para la creación de clases (POJO) y ficheros de correspondencia

```

<mapping resource="ORM/Departamento.hbm.xml" />
<mapping resource="ORM/Empleado.hbm.xml" />
<mapping resource="ORM/EmpleadoDatosProf.hbm.xml" />
<mapping resource="ORM/ProyectoSede.hbm.xml" />
<mapping resource="ORM/Proyecto.hbm.xml" />
<mapping resource="ORM/Sede.hbm.xml" />

```

4.4.6. HibernateUtil.java

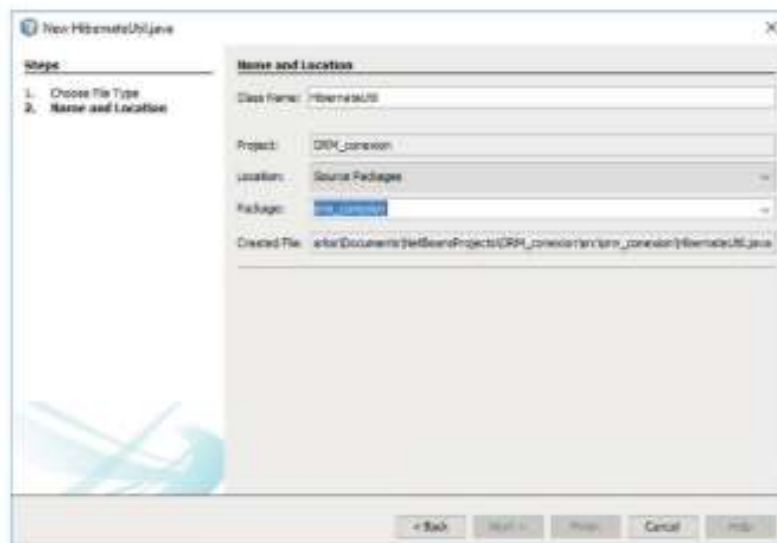


Figura 4.14

Asistente para la creación de `HibernateUtil.java`

Este es un fichero en el que se crea una clase que facilita enormemente la inicialización de Hibernate. Solo hay que indicar el nombre del fichero y el paquete en que se quiere crear.

Con esto se ha terminado de crear todo lo que necesita un programa que utilice Hibernate para persistencia de objetos. Solo falta escribir este programa en el fichero `ORM_conexion.java`. Pero antes se explicará cómo descargar y utilizar una versión reciente de Hibernate.

4.5. Descarga y uso de una versión reciente de Hibernate

Se puede comprobar que los asistentes han añadido los ficheros jar necesarios para Hibernate, pero que son de la versión 4.3. Se puede descargar la última versión estable de Hibernate de su página web. Es muy importante verificar sus prerequisites, especialmente en lo relativo a la versión de Java. La versión 5.3, que se va a emplear aquí, tiene como prerequisite Java SE 8.

Una vez descargado el fichero zip y descomprimido, aparece un directorio `lib`, y dentro de él los directorios `required`, `optional` y algunos más.

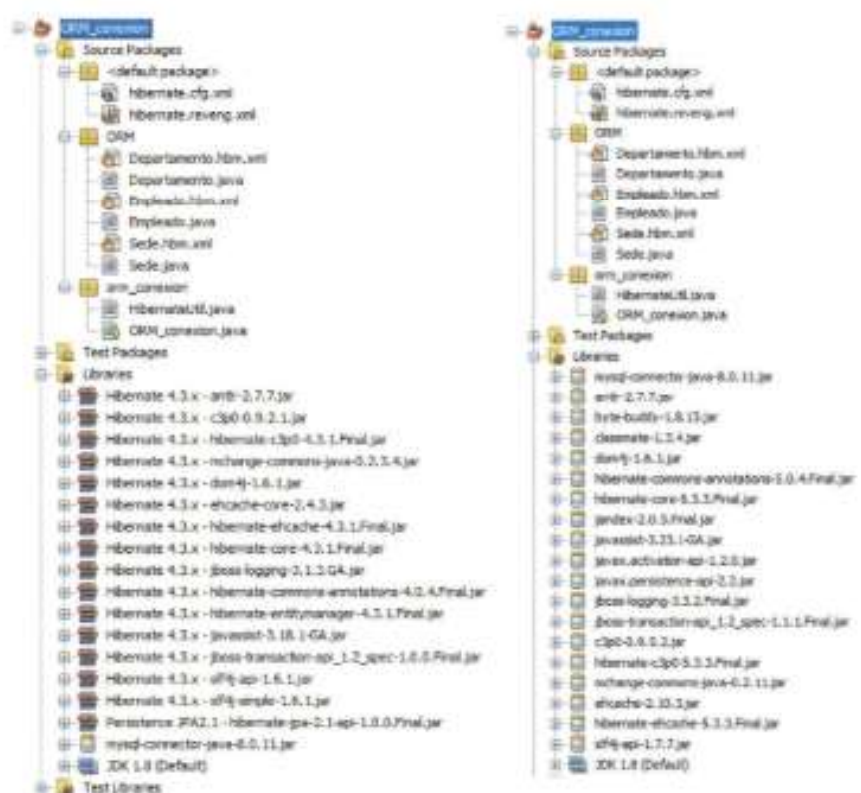


Figura 4.15

Cambios en el proyecto para usar una versión más reciente de Hibernate

Hay que reemplazar los ficheros `jar` de Hibernate añadidos por el asistente al proyecto por los de la versión más reciente. En el proyecto deben estar al menos los que vienen en el directorio `required`. Aparte de eso, se incluyen todos los equivalentes en la nueva versión a alguno de los que están en el proyecto. También se ha eliminado el `driver` para versiones anteriores de MySQL, `mysql-connector-java-5.1.23-bin.jar`, porque se usa el nuevo para la versión 8.0. En la figura 4.15 se muestra el proyecto antes y después de hacer estos cambios.

Ahora se plantea un pequeño problema, y es que no compila `HibernateUtil.java`. No se admite `AnnotationConfiguration`. Como con cualquier API, cuando se trabaja con Hibernate conviene tener a mano sus Javadocs.

Recurso web

www

Los Javadocs de Hibernate están disponibles en la descarga y también en Internet en las direcciones que se muestran a continuación:

<http://docs.jboss.org/hibernate/orm/5.3/javadocs/>
<http://docs.jboss.org/hibernate/orm/4.3/javadocs/>

En los Javadocs de la versión más reciente de Hibernate no aparece `AnnotationConfiguration`. Pero este código se creó para la versión 4.3 de Hibernate. En los Javadocs de esta versión dice que está obsoleta (*deprecated*), y que su funcionalidad se ha pasado a `Configuration`. El problema se resuelve cambiando `AnnotationConfiguration` por `Configuration`.

Los programas de ejemplo que se hagan a partir de ahora se harán en un proyecto como este, en el fichero `ORM_conexion.java` o en otro en la misma ubicación.

4.6. Programa de ejemplo para persistencia de objetos con Hibernate

Finalmente, un programa de ejemplo que crea una sede, un departamento de esa sede y un empleado de ese departamento. Todas las operaciones se realizan en una sesión y, dentro de ella, en una transacción. La clase `HibernateUtil`, definida en `HibernateUtil.java`, permite abrir una `Session` con suma facilidad. Las clases para objetos persistentes (POJO) están en el paquete con nombre `ORM`, por lo que se prefija con `ORM` su nombre. Para crear objetos se utilizan sus creadores sin parámetros. Para asignar valores a sus atributos se utilizan métodos `setter`, y también para relacionarlos con otros objetos (por ejemplo, para asociar un empleado a su departamento). Para guardar objetos en la base de datos se utiliza el método `save()` de la sesión. Baste esto para una primera aproximación. En las secciones siguientes se explicarán los detalles de la correspondencia objeto-relacional y otros aspectos de Hibernate.

```
// Programa sencillo para ORM con Hibernate
package orm_conexion;

import org.hibernate.Session;
import org.hibernate.Transaction;
```

```

public class ORM_conexion {
    public static void main(String[] args) {
        Transaction t = null;
        try (Session s = HibernateUtil.getSessionFactory().openSession()) {
            t = s.beginTransaction();

            ORM.Sede sede = new ORM.Sede();
            sede.setNomSede("MÁLAGA");
            s.save(sede);

            ORM.Departamento depto = new ORM.Departamento();
            depto.setNomDepto("INVESTIGACIÓN Y DESARROLLO");
            depto.setSede(sede);
            s.save(depto);

            ORM.Empleado emp = new ORM.Empleado();
            emp.setDni("56789012B");
            emp.setNomEmp("SAMPER");
            emp.setDepartamento(depto);
            s.save(emp);

            t.commit();
        } catch (Exception e) {
            e.printStackTrace(System.err);
            if (t != null) {
                t.rollback();
            }
        }
    }
}

```

Como resultado de la ejecución del programa, este será el contenido de las tablas. Para todos los valores definidos como `auto_increment` se ha asignado valor 1, al ser la primera fila que se añade a la tabla.

sede		departamento			empleado		
id_sede	nom_emp	id_depto	nom_depto	id_sede	dni	nom_emp	id_depto
1	MÁLAGA	1	I+D	1	56789012B	SAMPER	1

Actividades propuestas



- 4.1. Haz un programa que cree una nueva sede, dos departamentos para esta nueva sede y dos empleados para cada uno de estos departamentos. Verifica que los datos se crean correctamente comprobando el contenido de las tablas.
- 4.2. Ejecuta otra vez el programa de ejemplo, y verifica si se produce alguna excepción. Localiza el tipo de excepción. Cambia el programa para que este tipo de excepción se gestione de manera separada y se proporcione una información más concisa pero suficiente, en lugar de la muy prolija proporcionada por `printStackTrace()`. No se puede capturar directamente una excepción del tipo `ConstraintViolationException`. Hay que utilizar repetidamente el método

`getCause()` de `Exception` y verificar el tipo de excepción con `instanceof ConstraintViolationException`.

Haz que no puedan existir dos sedes distintas con idéntico nombre, y que no puedan existir dos departamentos distintos con idéntico nombre en una misma sede. La manera más sencilla es con índices únicos (sentencia `CREATE UNIQUE INDEX` de SQL). Verifica tu solución utilizando el programa de ejemplo inicial o pequeñas variaciones de él. Hay que introducir esta restricción en la propia base de datos, y hay que verificar que (en el caso en que se intente crear una nueva sede con el mismo nombre que una ya existente, y en el caso en que se intente crear un nuevo departamento en una sede con el mismo nombre que un departamento ya existente en esa sede) se produce una excepción y el programa la gestiona adecuadamente.

4.7. Ficheros hbm o de correspondencia de Hibernate

En este apartado se verá en detalle el contenido de los ficheros hbm (*Hibernate mapping files* o ficheros de correspondencia de Hibernate), que especifican la correspondencia entre clases de Java y tablas de una base de datos relacional. Se pondrá como ejemplo el fichero de correspondencia `departamento.hbm.xml`, que especifica la correspondencia entre la clase de Java `Departamento` y la tabla `departamento`. Pero la explicación no se limitará a lo que aparece en este fichero, sino que se pondrá todo en un contexto más amplio para explicar otras opciones y características de Hibernate.

La tabla `departamento` se definió de la siguiente manera en MySQL:

```
create table departamento(
  id_depto integer auto_increment not null,
  nom_depto char(32) not null,
  id_sede integer not null,
  primary key(id_depto),
  foreign key fk_depto_sede(id_sede) references sede(id_sede)
);
```

El POJO generado para ella, `Departamento.java`, es el siguiente:

```
/**
 * Departamento generated by hbm2java
 */
@Entity
@Table(name="departamento", catalog="proyecto_orm")
public class Departamento implements java.io.Serializable {

  private Integer idDepto;
  private Sede sede;
  private String nomDepto;
  private Set empleados = new HashSet(0);

  public Departamento() {
  }
}
```

```

public Departamento(Sede sede, String nomDeppto) {
    this.sede = sede;
    this.nomDeppto = nomDeppto;
}
public Departamento(Sede sede, String nomDeppto, Set empleados) {
    this.sede = sede;
    this.nomDeppto = nomDeppto;
    this.empleados = empleados;
}

@Id @GeneratedValue(strategy=IDENTITY)
@Column(name="id_depto", unique=true, nullable=false)
public Integer getIdDeppto() {
    return this.idDeppto;
}

public void setIdDeppto(Integer idDeppto) {
    this.idDeppto = idDeppto;
}

@ManyToOne(fetch=FetchType.LAZY)
@JoinColumn(name="id_sede", nullable=false)
@OneToMany(fetch=FetchType.LAZY, mappedBy="departamento")
public Sede getSede() {
    return this.sede;
}

public void setSede(Sede sede) {
    this.sede = sede;
}

@Column(name="nom_depto", nullable=false, length=32)
public String getNomDeppto() {
    return this.nomDeppto;
}

public void setNomDeppto(String nomDeppto) {
    this.nomDeppto = nomDeppto;
}

@OneToMany(fetch=FetchType.LAZY, mappedBy="departamento")
public Set getEmpleados() {
    return this.empleados;
}

public void setEmpleados(Set empleados) {
    this.empleados = empleados;
}
}

```

Hibernate requiere la opción `implements java.io.Serializable`. Los atributos se declaran como privados y tienen métodos *getter* y *setter*.

Los nombres de los atributos de la clase se obtienen a partir de los nombres de los atributos de las tablas siguiendo sencillas reglas, de manera que al atributo `nom_depto` de la tabla `departamento`, por ejemplo, le corresponde el atributo `nomDeppto` de la clase `Departamento`.

En la clase `Departamento` se reflejan las relaciones de uno a muchos en las que participa la tabla `departamento`, definidas mediante claves foráneas (`FOREIGN KEY` en SQL). La relación

de uno a muchos de un departamento con sus empleados se refleja en `Set empleados`, para los empleados del departamento. El nombre `empleados` es el resultado de añadir `s` al final del nombre de la tabla `empleado` para formar su plural.



PARA SABER MÁS

Reglas de nomenclatura de Hibernate

Hibernate utiliza reglas para la correspondencia entre nombres de tablas y nombres de columnas de tablas, por una parte, y nombres de clases y nombres de atributos de clases, por otra. Esto incluye la formación de los plurales de los nombres de clases para las colecciones, añadiendo `s` o `es`. El motivo para poner los nombres de las tablas en singular, si se tiene esa posibilidad, es que los plurales resultantes sean correctos. Que no lo sean, por supuesto, no impide que las clases funcionen como deben. Las reglas de nomenclatura se pueden cambiar, pero no es sencillo.

La relación de uno a muchos de una sede con sus departamentos se refleja en el atributo `Sede sede` en la clase `Departamento`, para la sede a la que pertenece el departamento. Este es el contenido del fichero de correspondencia `departamento.hbm.xml`.

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//
EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<!-- Generated by Hibernate Tools 4.3.1 -->
<hibernate-mapping>
  <class name="ORM.Departamento" table="departamento" catalog="proyecto_orm"
    optimistic-lock="version">
    <id name="idDepto" type="java.lang.Integer">
      <column name="id_depto" />
      <generator class="identity" />
    </id>
    <many-to-one name="sede" class="ORM.Sede" fetch="select">
      <column name="id_sede" not-null="true" />
    </many-to-one>
    <property name="nomDepto" type="string">
      <column name="nom_depto" length="32" not-null="true" />
    </property>
    <set name="empleados" table="empleado" inverse="true" lazy="true"
      fetch="select">
      <key>
        <column name="id_depto" not-null="true" />
      </key>
      <one-to-many class="ORM.Empleado" />
    </set>
  </class>
</hibernate-mapping>
```

4.7.1. Correspondencia para las clases y atributos de clase

En el elemento `<class>` se indica la clase (`ORM.Departamento`), la tabla con la que se corresponde (`departamento`) y la base de datos en que está dicha tabla (`proyecto ORM`).

A continuación, vienen una serie de elementos que establecen la correspondencia de los distintos atributos de la clase. Tienen algunos atributos comunes, tales como:

name: el nombre del atributo en la clase.

type: el tipo del atributo en la clase de Java. Hibernate hace corresponder a cada posible tipo para un atributo de una clase un tipo estándar de SQL.

El atributo de la tabla de la base de datos se indica con el elemento `<column>`. Este puede tener algún atributo con información adicional para algunos tipos. Para un `string`, por ejemplo, `length` indica la longitud del campo en la base de datos.

Los tipos de elementos que aparecen son:

a) `<id>` hace corresponder un atributo de la clase con uno que es clave primaria de la tabla:

Departamento.java	CREATE TABLE departamento
private Integer idDepto;	id_depto integer auto_increment not null, primary key(id_depto)
<pre><id name="idDepto" type="java.lang.Integer"> <column name="id_depto" /> <generator class="identity" /> </id></pre>	
Departamento.hbm.xml	

Empleado.java	CREATE TABLE empleado
private String dni;	dni char(9) not null, primary key(dni)
<pre><id name="dni" type="string"> <column name="dni" length="9"/> <generator class="assigned"/> </id></pre>	
Empleado.hbm.xml	

El valor del atributo `generator` hace referencia a una clase de Hibernate utilizada para generar valores para los campos de la clave primaria. Hay unas cuantas disponibles que deberían ser suficientes para cualquier situación. Pero si fuera necesario, se pueden crear nuevas a medida. Algunos de los valores disponibles son:

- **identity:** la columna se define en la base de datos como clave autogenerada, en el caso de MySQL con la opción `auto_increment`. La base de datos genera un valor para esta (`id_depto` de la tabla `departamento`), que se asigna al atributo correspondiente (`idDepto` de la clase `Departamento`).

- **assigned:** la aplicación debe proporcionar un valor para el campo. Este es el caso, por ejemplo, para el atributo **DNI** de la clase **Empleado**, que se corresponde con el atributo **DNI** de la tabla **empleado**.
- **sequence:** se puede utilizar con Oracle para generar un identificador numérico utilizando una secuencia de la base de datos.
- **native:** puede comportarse como **identity**, **sequence** o **hilo** (esta última opción no se verá aquí), dependiendo de la base de datos con la que se trabaje.
- **foreign:** usa el identificador de otro objeto asociado. Usado para relaciones de uno a uno, que se verán en breve.

b) **<property>** hace corresponder un atributo de la clase con uno de la tabla, sin más:

Departamento.java	CREATE TABLE departamento
private String nomDeppto;	nom_depto char(32) not null
<pre><property name="nomDeppto" type="string"> <column name="nom_depto" length="32" not-null="true" /> </property></pre>	
Departamento.hbm.xml	

El atributo **not-null** se utiliza para indicar que no puede tomar valor nulo, lo que se corresponde con la opción **not null** en SQL.

4.7.2. Correspondencia para las relaciones

También debe establecerse una correspondencia para las relaciones. Es decir, también es necesario especificar cómo las relaciones entre distintos objetos se reflejan en la base de datos. Existen relaciones de uno a muchos, de uno a uno y de muchos a muchos. Hibernate permite especificar la correspondencia para todos estos tipos.

Para el ejemplo desarrollado se ha partido de un esquema relacional. Para ilustrar las relaciones se recurre a un diagrama entidad-relación (o diagrama E-R).

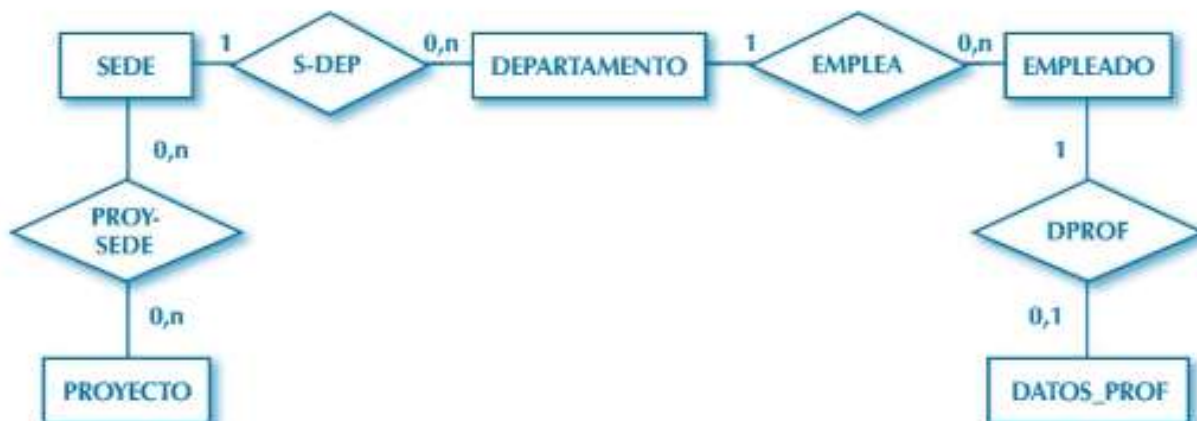


Figura 4.16
Diagrama E-R para el esquema relacional de ejemplo

RECUERDA

- ✓ En el módulo "Bases de Datos" de primer curso se estudian los diagramas E-R, su uso para el modelado conceptual de datos, y la manera de obtener un esquema relacional a partir de un diagrama E-R. En un diagrama E-R se pueden representar relaciones de muchos a muchos. No es así en un esquema relacional, por lo que, para poder representarlas en un esquema relacional, hay que descomponerlas previamente en dos relaciones de uno a muchos.

El diagrama E-R que se muestra en la figura 4.16 tiene relaciones de todos los tipos: de uno a muchos (entre **SEDE** y **DEPARTAMENTO** y entre **DEPARTAMENTO** y **EMPLEADO**), de uno a uno (entre **EMPLEADO** y **DATOS_PROF**) y de muchos a muchos (entre **SEDE** y **PROYECTO**).

Las relaciones entre dos clases deben definirse en Hibernate en los ficheros de correspondencia de ambas clases. Por este motivo, si a un objeto de la clase A le pueden corresponder varios de la clase B, la relación entre ambas clases se define de uno a muchos desde la clase A hacia la clase B en el fichero de correspondencia para la clase A, y de muchos a uno desde la clase B hacia la clase A en el fichero de correspondencia para la clase B.

A continuación, se explica la manera en que las relaciones de distintos tipos se han representado en los ficheros de correspondencia generados por Hibernate.

A) Relaciones de uno a muchos

`<many-to-one>` y `<one-to-many>` se usan para especificar dos relaciones recíprocas, la segunda de uno a muchos y la primera de muchos a uno. A una sede pueden corresponder varios departamentos (relación uno a muchos o *one-to-many*). Recíprocamente, varios departamentos pueden corresponder a una sede (relación muchos a uno o *many-to-one*). En última instancia, estas relaciones de uno a muchos, sus recíprocas de muchos a uno, sea entre instancias de objetos o entre filas de tablas, son una misma cosa, especificada en la tabla **departamento** con la clave foránea **fk_depto_sede(id_sede)** **references** **sede(id_sede)**, que establece que a cada fila de la tabla **departamento** le corresponde una en la tabla **sede**, determinada por el valor del atributo **id_sede**. Esto se refleja en la clase **ORM.Departamento** mediante un atributo **sede**, que indica la sede a la que pertenece el departamento, y en la clase **ORM.Sede** mediante una colección **departamentos**, que contiene los departamentos pertenecientes a la sede.

Departamento.java	CREATE TABLE departamento
private Sede sede;	id_sede integer not null, foreign key fk_depto_sede(id_sede) references sede(id_sede)
<code><many-to-one name="sede" class="ORM.Sede" fetch="select"></code> <code><column name="id_sede" not-null="true" /></code> <code></many-to-one></code>	
Departamento.hbm.xml	

Sede.java	CREATE TABLE sede
private Set departamentos = new HashSet(0);	id_sede integer auto_increment not null, primary key(id_sede)
<set name="departamentos" table="departamento" inverse="true" lazy="true" fetch="select"> <key> <column name="id_sede" not-null="true" /> </key> <one-to-many class="ORM.Departamento" /> </set>	
Sede.hbm.xml	

El elemento `<set>` especifica el tipo de colección que se utilizará para almacenar los departamentos de una sede (`HashSet`) y cómo se pueden obtener en la base de datos (consultando la tabla `departamento` por `id_sede`, como se indica en `<key>`). `<one-to-many>` establece el tipo de objetos que se almacenarán en la colección (`ORM.Departamento`).

El atributo `lazy` toma por defecto valor `true`. Eso significa que, cuando se consulta la base de datos para obtener un objeto de tipo `ORM.Departamento`, no se obtienen los empleados del departamento en la colección `empleados`. En lugar de ello, Hibernate proporciona un proxy, que obtendrá los empleados de la base de datos solo cuando se consulte el contenido de la colección `empleados`.

El atributo `fetch` puede tomar los valores `select` y `join`. El valor elegido solo supone una diferencia cuando se consulta la base de datos para obtener varios departamentos. `select` significa que para cada departamento se hará una consulta de SQL para obtener sus empleados. `join` significa que en algunos casos (no siempre, depende del mecanismo utilizado para obtener los datos de los departamentos) se hará una única consulta de SQL con `left outer join`, lo que puede mejorar el rendimiento.

B) Relaciones uno a uno

Se especifica en el elemento `<one-to-one>` en los ficheros de correspondencia `Empleado.hbm.xml` y `EmpleadoDatosProf.hbm.xml`. Es una relación de uno a uno y no de uno a muchos porque la clave foránea se define desde la clave primaria de una tabla (`empleado_datos_prof`). No es completamente simétrica: a un objeto de la clase `EmpleadoDatosProf` le corresponde exactamente uno de la clase `Empleado` (cardinalidad 1), pero a uno de la clase `Empleado` puede no corresponderle ninguno de la clase `EmpleadoDatosProf` (cardinalidad 0,1). Esta asimetría se refleja en los ficheros de correspondencia, cuyos contenidos más relevantes se muestran a continuación:

Empleado.java	CREATE TABLE empleado
private EmpleadoDatosProf empleadoDatosProf;	dni char(9) not null, primary key(dni)
<id name="dni" type="string"> <column name="dni" length="9" /> <generator class="assigned" /> </id> <one-to-one name="empleadoDatosProf" class="ORM.EmpleadoDatosProf"> </one-to-one>	
Empleado.hbm.xml	

EmpleadoDatosProf.java	CREATE TABLE empleado_datos_prof
private Empleado empleado;	dni char(9) not null, primary key(dni), foreign key fk_empleado_datosprof_ empl(dni) references empleado(dni)
<pre> <id name="dni" type="string"> <column name="dni" length="9" /> <generator class="foreign"> <param name="property">empleado</param> </generator> </id> <one-to-one name="empleado" class="ORM.Empleado" constrained="true"> </one-to-one> </pre>	
Empleado.hbm.xml	

Las asimetrías se dan en `EmpleadoDatosProf.hbm.xml`, porque es en la tabla `Empleado_datos_prof` donde se define, desde su clave primaria, la clave foránea a la otra tabla `Empleado`. Esto significa que no puede existir un objeto de clase `EmpleadoDatosProf` sin un objeto correspondiente de clase `Empleado`, y se refleja con `constrained=true`. Además, el objeto de clase `EmpleadoDatosProf` debe tener el mismo identificador que el correspondiente de clase `Empleado`, y esto se refleja con `<generator class="foreign">`, lo que significa que el identificador (`dni`) se tomará del objeto de esta otra clase.

C) Relaciones de muchos a muchos

En los ficheros de correspondencia de Hibernate se pueden definir relaciones de este tipo mediante el elemento `<many-to-many>`. No se explica aquí cómo especificar la correspondencia para relaciones de muchos a muchos con Hibernate. Cuando se generan los ficheros de correspondencia a partir de un esquema relacional, no aparecen este tipo de relaciones porque, como se ha comentado, en un esquema relacional no se pueden representar directamente.

Cuando se obtiene un esquema relacional a partir de un diagrama E-R, una relación de muchos a muchos se descompone en dos relaciones de uno a muchos con una nueva entidad. En este caso, la relación de muchos a muchos `PROY-SEDE` entre `SEDE` y `PROYECTO` se descompone en dos relaciones de uno a muchos de `SEDE` y de `PROYECTO` a una nueva entidad, y en el esquema relacional se introducen tablas para `SEDE` (tabla `sede`), para `PROYECTO` (tabla `proyecto`) y para la nueva entidad (tabla `proyecto_sede`), y una restricción de clave foránea (`FOREIGN KEY`) para cada relación de uno a muchos.

Como recapitulación, se incluye un diagrama E-R similar al anterior, solo que con la relación de muchos a muchos entre `SEDE` y `PROYECTO` transformada en dos relaciones de uno a muchos con una nueva entidad (a la que corresponde la clase `ProyectoSede`). Se indican nombres de clases en lugar de entidades. Se indican junto a cada clase los atributos que se emplean para representar las relaciones con otras clases, los que son colecciones con letra en cursiva. Estos atributos son privados y no se puede acceder directamente a ellos, pero existe un método `getter` para recuperar cada uno (`getSede()` para `sede`, `getEmpleados()` para `empleados`, etc), y un método `setter` para modificarlos.

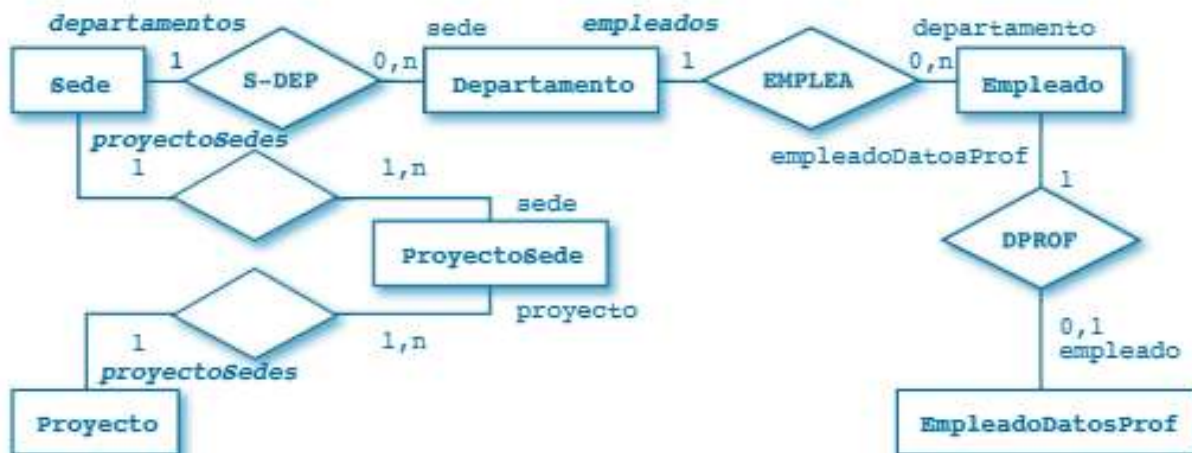


Figura 4.17

Atributos de clase para representación de las relaciones entre clases

Una vez recuperado un objeto persistente, estos atributos permiten acceder fácilmente a otros objetos relacionados, y crear nuevas asociaciones con otros, sin necesidad de operar con la base de datos, solo trabajando con clases de Java. Por ejemplo: a partir del DNI de un empleado se puede obtener su departamento con `getDepartamento()`, y a partir de él todos sus empleados con `getEmpleados()`, y para cada uno de ellos sus datos profesionales con `getEmpleadoDatosProf()`. También se podrían establecer nuevas relaciones entre objetos o modificar las que ya existen utilizando los correspondientes métodos *setter*.

TOMA NOTA



No es recomendable utilizar los métodos *setter* para asignar directamente un valor a las colecciones. Es mejor obtener la colección con el método *getter* y manejarla con los métodos propios de las colecciones tales como `add()` para añadir, etc. En breve se pondrán algunos ejemplos.

Si no se trabaja con Hibernate o una herramienta similar, habría que utilizar consultas SQL para obtener los departamentos de una sede, o los empleados de un departamento, y también para modificar estos datos. Para ello es necesario tener presente la estructura de la base de datos. Esta operativa no se presta a un desarrollo orientado a objetos. Los programas de este tipo tienden a utilizar muchas y a menudo complejas sentencias de SQL. La desventaja que puede tener utilizar Hibernate o *frameworks* similares es el menor rendimiento de las operaciones de consulta o modificación de datos. Para aplicaciones de envergadura que manejan bases de datos complejas y con grandes volúmenes de datos, es necesario optimizar las operaciones con la base de datos. Pero el margen de maniobra es menor, y para optimizar bien el rendimiento se necesita un conocimiento avanzado del *framework*. Lo que se gana en claridad y mantenibilidad, y las ventajas de utilizar la programación orientada a objetos, puede ir en detrimento del grado de control sobre la base de datos, de la flexibilidad y de la eficiencia en las operaciones con la base de datos. De todas formas, como se verá más adelante, Hibernate permite también realizar consultas directamente con SQL.

En los siguientes apartados se explica la manera de gestionar las relaciones de uno a muchos y de uno a uno.

4.8. Manejo de relaciones de uno a muchos entre objetos persistentes

Para reflejar este tipo de relaciones entre dos clases se utilizan atributos en ambas clases. En una de ellas el atributo es sencillo y en la otra es una colección.

El siguiente programa crea una nueva sede y tres departamentos de ella, y por último muestra los datos de la sede y de sus departamentos, e itera sobre la colección que en un objeto de tipo `Sede` permite acceder a sus departamentos. El programa no muestra los departamentos creados para la sede si no se utiliza `refresh()`, aunque estos se graban correctamente en la base de datos. La asignación de la sede a los departamentos en objetos de tipo `ORM.Departamento` no se refleja automáticamente en la colección de los departamentos para la sede en el objeto de tipo `ORM.Sede`. Con `s.refresh(sede)` se actualiza `sede` con los contenidos actuales de la base de datos. Es posible que los departamentos no se listen en el mismo orden en el que se han creado. Con Hibernate se puede establecer un orden para los objetos de una colección, pero una vez establecido, no se pueden recuperar en otro. Para el segundo y tercer departamento, se usan constructores que permiten especificar directamente sus datos, en lugar de usar el constructor sin parámetros y después métodos `setter`, como para el primero. Se ha hecho de las dos formas para ilustrar ambas formas de hacer lo mismo.

```
// Programa que crea una sede y luego varios departamentos asociados
package orm_colecciones_e_iteradores;

import java.util.Iterator;
import org.hibernate.Session;
import org.hibernate.Transaction;

public class ORM_colecciones_e_iteradores {
    public static void main(String[] args) {
        Transaction t = null;
        try (Session s = HibernateUtil.getSessionFactory().openSession()) {
            t = s.beginTransaction();
            ORM.Sede sede = new ORM.Sede("TERUEL");
            s.save(sede);

            ORM.Departamento depto = new ORM.Departamento();
            depto.setNomDeppto("I+D");
            depto.setSede(sede);
            s.save(depto);

            depto = new ORM.Departamento(sede, "MARKETING");
            s.save(depto);

            depto = new ORM.Departamento(sede, "QA");
            s.save(depto);

            s.refresh(sede);

            Iterator itDeptos = sede.getDepartamentos().iterator();
            System.out.println("Nueva sede [" + sede.getIdSede() + "] (" +
                sede.getNomSede() + ")");
            System.out.println("-----");

            while(itDeptos.hasNext()) {
                ORM.Departamento unDeppto = (ORM.Departamento) itDeptos.
                    next();
            }
        }
    }
}
```

```

        System.out.println("Deppto: [" + unDeppto.getIdDeppto() + "] (" +
            +
            unDeppto.getNomDeppto()+")");
    }
    t.commit();
} catch (Exception e) {
    e.printStackTrace(System.err);
    if (t != null) {
        t.rollback();
    }
}
}
}
}

```

El siguiente programa hace lo mismo, pero de distinta manera. Primero crea una nueva sede. Después crea uno a uno los nuevos departamentos y los añade a la colección que almacena los departamentos para la nueva sede. Para este programa no hace falta usar `refresh()` para que se muestre correctamente toda la información.

```

// Programa que crea una sede y varios departamentos que añade a su lista
package orm_colecciones_e_iteradores_2;

import java.util.Iterator;
import org.hibernate.Session;
import org.hibernate.Transaction;
public class ORM_colecciones_e_iteradores_2 {

    public static void main(String[] args) {
        Transaction t = null;
        try(Session s = HibernateUtil.getSessionFactory().openSession()) {
            t = s.beginTransaction();

            ORM.Sede sede = new ORM.Sede("VALENCIA");
            s.save(sede);

            ORM.Departamento depto1=new ORM.Departamento(sede, "I+D");
            s.save(depto1);
            sede.getDepartamentos().add(depto1);

            ORM.Departamento depto2=new ORM.Departamento(sede, "MARKETING");
            s.save(depto2);
            sede.getDepartamentos().add(depto2);

            ORM.Departamento depto3=new ORM.Departamento(sede, "QA");
            s.save(depto3);
            sede.getDepartamentos().add(depto3);

            Iterator<ORM.Departamento> itDeptos =
                sede.getDepartamentos().iterator();

            System.out.println("Nueva sede [" + sede.getIdSede() + "] (" +
                sede.getNomSede() + ")");
            System.out.println("-----");

            while(itDeptos.hasNext()) {
                ORM.Departamento unDeppto = (ORM.Departamento) itDeptos.next();
                System.out.println("Deppto: [" + unDeppto.getIdDeppto() + "] (" +

```



```

ORM.EmpleadoDatosProf datosProf = new ORM.EmpleadoDatosProf();
datosProf.setEmpleado(emp);
datosProf.setCategoria("A");
datosProf.setSueldoBrutoAnual(new BigDecimal(52000.0));
s.save(datosProf);
t.commit();

s.refresh(emp);
System.out.println("Categoría del nuevo empleado: " +
    emp.getEmpleadoDatosProf().getCategoria() +
    ", DNI: " + emp.getEmpleadoDatosProf().getDni()
);
} catch (Exception e) {
e.printStackTrace(System.err);
if (t != null) {
    t.rollback();
}
}
}
}

```

4.10. Sesiones y estados de los objetos persistentes

Ya se ha aprendido cómo se puede establecer la correspondencia objeto-relacional para clases de Java, y también para relaciones entre clases. Se han creado objetos transitorios y se han almacenado en la base de datos como objetos persistentes con Hibernate. Ahora es el momento de explicar en detalle el ciclo de vida de los objetos persistentes, los distintos estados en los que pueden estar, y las operaciones que permiten recuperar objetos persistentes de la base de datos, modificarlos y grabar estas modificaciones de nuevo en la base de datos.

Una clase para la que se establecen correspondencias mediante Hibernate se denomina *clase persistente*, y una instancia de una clase persistente, *objeto persistente*.

La sesión (interfaz `org.hibernate.Session`) es un componente esencial en Hibernate. Una sesión se construye sobre una conexión a la base de datos. Las transacciones creadas por los programas anteriores son transacciones de sesión, y pueden incluir varias transacciones de base de datos. Una sesión puede mejorar el rendimiento de las consultas en objetos persistentes utilizando una caché. La apertura de sesiones es muy rápida utilizando un *pool* de conexiones.

Una sesión, junto con un gestor de entidades asociado, constituye un *contexto de persistencia*. Un gestor de entidades (interfaz `javax.persistence.EntityManager`) lleva el control de los cambios que se realizan sobre objetos persistentes. La interfaz `Session` pertenece a la API propia de Hibernate, mientras que la interfaz `EntityManager` pertenece a la de JPA. Se puede obtener un `EntityManager` a partir de una `Session` y viceversa; existe, pues, un puente entre ambas API. Casi todo lo que se puede hacer utilizando una de estas API se puede hacer utilizando la otra, si bien es cierto que la interfaz de Hibernate ofrece más posibilidades, o al menos una manera más directa de hacer muchas cosas. Aquí, por brevedad y simplicidad, se explicará solo la interfaz `Session` de la API de Hibernate.

Los cambios que se realizan sobre objetos persistentes se reflejan, en última instancia, en la base de datos en tanto en cuanto están asociados a un contexto de persistencia.

Los objetos persistentes pueden estar en diferentes estados:

1. *Transitorio (transient)*. El objeto se acaba de crear y no está asociado con ningún contexto de persistencia. No está grabado en la base de datos. No tiene por qué tener un identificador único asignado, pero podría tenerlo si se ha especificado para su identificador el generador `assigned` (valor "`assigned`" para el atributo `generator` del elemento `id`), y se le ha asignado un valor.
2. *Gestionado (managed) o persistente (persistent)*. El objeto tiene un identificador asociado y está asociado con un contexto de persistencia, y está grabado en la base de datos. Cualquier cambio que se realice en el objeto se reflejará en la base de datos. No inmediatamente, sino en respuesta a determinadas operaciones sobre la sesión, típicamente cuando se cierra con `close()`.
3. *Separado (detached)*. El objeto tiene un identificador asociado, pero ya no está asociado con un contexto de persistencia, bien porque el contexto se ha cerrado, bien porque se desvinculó el objeto del contexto.
4. *Eliminado (removed)*. El objeto tiene un identificador asociado y está asociado con un contexto de persistencia, pero está pendiente de ser borrado de la base de datos.

Un objeto persistente puede cambiar de estado merced a distintas operaciones realizadas sobre él dentro de una sesión. Algunos de estos cambios de estado conllevan operaciones en la base de datos para grabar, modificar o borrar la representación del objeto en ella.

Los objetos en estados transitorio y separado, una vez que ya no están asociados a un contexto de persistencia, pueden ser eliminados por el recolector de basura de Java (*garbage collector*), desde el mismo momento en que no exista ninguna referencia a ellos.



Figura 4.18
Diagrama de estados de un objeto persistente

En el resto de este apartado se ofrece una breve explicación de las operaciones que permiten crear objetos persistentes, cambiar su estado y realizar diversas operaciones con ellos, limitadas, como ya se ha comentado, a la API de Hibernate. Esta es una explicación general para proporcionar una primera introducción a las posibilidades de Hibernate. Por tanto, se omiten muchos detalles, muchas opciones de algunas operaciones, muchos casos particulares, muchas implicaciones que pueden ser relevantes en situaciones concretas, y muchas posibilidades de una herramienta tan potente y compleja como es Hibernate.

4.10.1. De transitorio a persistente con `save()`, `saveOrUpdate()` y `persist()`

Un objeto se puede pasar de estado transitorio a persistente con los métodos `save()`, `saveOrUpdate()` y `persist()` de `Session`. Si el identificador de la clase se genera automáticamente (es el caso con un valor `identity`, `sequence` o `native`, y algunos otros, para el atributo `generator`), no hay que asignar un valor al identificador único para la clase. En ese caso, `save()` y `saveOrUpdate()` devuelven el identificador una vez que se ha grabado el objeto en la base de datos, lo que se hace inmediatamente, mientras que `persist()` no devuelve este identificador, y en algunos casos se puede grabar el objeto posteriormente, lo que contribuye a mejorar el rendimiento. `persist()` solo puede utilizarse dentro de una transacción, al contrario que `save()` y `saveOrUpdate()`, que pueden utilizarse tanto dentro como fuera de una transacción.

```
ORM.Sede sede = new ORM.Sede();
sede.setNomSede("CUENCA");
Integer idSede = (Integer) s.save(sede);
```

En caso contrario, como es el caso con el valor `assigned` para el atributo `generator`, el programa debe asignar un valor al identificador, y la grabación en la base de datos no se realizará inmediatamente.

```
ORM.Empleado emp = new ORM.Empleado();
emp.setDni("78901234X");
emp.setNomEmp("NADALES");
emp.setDepartamento(depto);
s.save(emp);
```

Si se intenta hacer persistente un objeto transitorio creado con `new()` y ya existe un objeto persistente de la misma clase y con idéntico identificador, `save()` generará una excepción, dado que no puede haber dos objetos persistentes con el mismo identificador. De hecho, en alguna de las actividades propuestas anteriormente, se ha pedido verificar esto mismo, y gestionar la excepción que se producía.

El método `saveOrUpdate()` funciona igual que `save()` para un objeto transitorio creado con `new` si no hay ningún objeto persistente con el mismo identificador. Si lo hay, entonces modifica dicho objeto. Dicho de otra forma, `saveOrUpdate()` es a `save()` en Hibernate lo que `INSERT INTO ... ON DUPLICATE KEY UPDATE` es a `INSERT INTO` en SQL.

```
ORM.Empleado emp = new ORM.Empleado();
emp.setDni("78901234X");
emp.setNomEmp("NADALES");
emp.setDepartamento(depto);
s.saveOrUpdate(emp);
```



Actividad propuesta 4.3

Modifica el primer programa de ejemplo para que, si ya existe un empleado con el DNI, le asigne un nombre determinado, y si no existe, cree un empleado con dicho nombre. Utiliza `saveOrUpdate()`.

Ya se ha comentado que `persist()` no devuelve un identificador y que, en el caso de que el identificador de la clase se genere automáticamente, en algunos casos se podría postergar la grabación del objeto en la base de datos. Esto puede mejorar el rendimiento si esta operación se realiza dentro de una transacción con muchas operaciones o que se prolongue mucho en el tiempo. Si no es el caso, no hay ninguna diferencia en la práctica entre utilizar `save()` y `persist()`, aparte de que con la primera se podrá disponer siempre inmediatamente de los identificadores generados automáticamente.

```
ORM.Sede sede = new ORM.Sede();
sede.setNomSede("PALENCIA");
s.persist(sede);
// sede.getIdSede() podría devolver null porque todavía no se haya grabado
// la sede en la base de datos. En cambio, con s.save() la sede se habría
// grabado ya en la base de datos y getIdSede() siempre devolvería un valor.
```

```
ORM.Empleado emp = new ORM.Empleado();
emp.setDni("78901234X");
emp.setNomEmp("NADALES");
emp.setDepartamento(depto);
s.persist(emp);
// emp.getDni() devolvería el DNI: "78901234X"
```

4.10.2. Obtención de un objeto persistente con `get()` y `load()`

`get()` y `load()` permiten obtener un objeto persistente e indicar su clase y el valor para su identificador único. Se diferencian en que, si no existe un objeto persistente de la clase indicada y con el identificador indicado, `get()` devuelve un valor `null`, mientras que `load()` lanza una excepción de tipo `ObjectNotFoundException`.

Hasta ahora no se había visto cómo recuperar un objeto persistente grabado en la base de datos. Más adelante se verá cómo obtener objetos persistentes sin necesidad de conocer su identificador único, sino especificando criterios de selección.

Existen versiones de ambos métodos con parámetros adicionales para especificar diversas opciones, tales como tipo de bloqueo sobre el objeto persistente, tiempo de espera máximo antes de desistir de conseguir el bloqueo, y otras más, pero no se explican aquí.

```
ORM.Empleado emp = s.get(ORM.Empleado.class, "78901234X");
if(emp==null) {
    System.out.println("No existe empleado");
}
else {
    System.out.println("El nombre del empleado es "+emp.getNomEmp());
}
```

Se suele utilizar `load()` cuando se sabe con seguridad que el objeto existe, o solo puede no existir en una situación anómala. El siguiente fragmento de código muestra un escenario de uso típico, en el que no se gestionan excepciones de tipo `ObjectNotFoundException`.

```
ORM.Sede sede = s.load(ORM.Sede.class, 12);  
System.out.println("El nombre de la sede es " + sede.getNomSede());
```

4.10.3. De persistente a eliminado con `delete()`

Se puede borrar un objeto persistente con `delete()`.

```
ORM.Sede sede = s.get(ORM.Sede.class, 15);  
if(sede != null) {  
    s.delete(sede);  
}
```

4.10.4. De persistente a separado con `evict()`, `close()` y `clear()`

Estos métodos permiten separar objetos de sesiones, es decir, pasarlos a estado separado o *detached*. Con `evict(Object obj)` se puede separar un objeto. Con `close()` y `clear()` se separan todos los objetos asociados a la sesión. Con `close()` se grabarán los cambios realizados sobre ellos en la base de datos, mientras que con `clear()` no se grabarán. Se pueden seguir consultando y modificando los datos de un objeto en estado separado, pero las modificaciones no se reflejarán en la base de datos, a menos que se vuelva a asociar el objeto a una sesión, que se puede hacer.

```
s.evict(sede); // Separa objeto sede de sesión
```

```
s.close(); // Graba todos los objetos asociados a la sesión y los separa
```

```
s.clear(); // Separa todos los objetos asociados a la sesión, sin grabarlos
```

4.10.5. De separado a persistente con `update()`, `saveOrUpdate()`, `lock()` y `merge()`

Un objeto separado puede volver a hacerse persistente con `lock()`. Al llamar a este método hay que indicar un modo de bloqueo. Hay unos cuantos modos de bloqueo.

Recurso web

[www](http://www.jboss.org/hibernate/stable/orm/userguide/html_single/Hibernate_User_Guide.html#locking-LockMode)

Modos de bloqueo con Hibernate:

https://docs.jboss.org/hibernate/stable/orm/userguide/html_single/Hibernate_User_Guide.html#locking-LockMode

Se puede también volver a hacer persistente un objeto separado con `update()` y `saveOrUpdate()`. También con `merge()`, que no hace persistente el objeto que se le pasa, sino que actualiza los contenidos de otro objeto persistente con el mismo identificador en la sesión si existe, o crea uno nuevo si no existe. El resultado, en cualquier caso, es que los datos del objeto que se le pasa se incorporan a un objeto persistente asociado a la sesión, y se devuelve este objeto. El objeto que se le pasa a `merge()` sigue siendo un objeto separado y se puede descartar.

4.11. Lenguajes de consulta HQL y JPQL

HQL son las siglas de Hibernate Query Language. Es un lenguaje inspirado en SQL. Tiene sentencias `SELECT`, `INSERT`, `UPDATE` y `DELETE`, similares a las del lenguaje SQL.

JPQL es un subconjunto de HQL incluido en la especificación de JPA.

www

Recurso web

El lenguaje HQL es similar al lenguaje SQL. Para más información sobre la sintaxis de los lenguajes HQL y JPQL se puede consultar el siguiente enlace, con numerosos ejemplos, y muy asequible con unos conocimientos básicos de SQL (sección de la guía de usuario de Hibernate).

https://docs.jboss.org/hibernate/stable/orm/userguide/html_single/Hibernate_User_Guide.html#hql-statement-types

Los IDE NetBeans y Eclipse tienen intérpretes de HQL que permiten ejecutar directamente sentencias de HQL. En Netbeans, se accede a él pulsando con el botón derecho del ratón sobre el fichero de configuración `hibernate.cfg.xml` y seleccionando la opción "Run HQL Query". No es muy intuitivo, pero tiene su lógica que el intérprete de HQL se lance desde ahí, ya que en este fichero se define la conexión con la base de datos. En la imagen se puede ver el resultado de una sencilla consulta

Session: hibernate.cfg			
FROM Departamento			
Result: SQL			
Set Max. Row Count: 100			
0 row(s) updated; 3 row(s) selected.			
Sede	NomDepto	IdDepto	Empleados
ORM.Sede@65d1621e	INVESTIGACIÓN Y DESARROLLO	11	[ORM.Empleado@3817317f, ORM.Empleado@5b61ff9f]
ORM.Sede@65d1621e	MARKETING	13	[]
ORM.Sede@65d1621e	RECURSOS HUMANOS	14	[]

Figura 4.19

Resultado de una consulta en el intérprete de HQL de Netbeans

Tiene un aspecto muy similar al resultado de una consulta con SQL, pero con algunas particularidades. La columna `Sede` muestra objetos en lugar de datos de un tipo elemental. Cada objeto viene identificado por el nombre de su clase (`ORM.Sede`) y un identificador único. La columna `Empleados` muestra listas de empleados, vacías todas excepto para un departamento con dos empleados, que se muestran de la misma manera: con el nombre de su clase (`ORM.Empleado`) y un identificador único.

4.11.1. La interfaz `Query`

La API de Hibernate permite utilizar sentencias de HQL mediante la interfaz `Query`. Esto abre muchas posibilidades. Con lo visto hasta ahora solo se podía acceder a objetos persistentes mediante sus identificadores únicos. Pero hay que tener cuidado con las operaciones de actualización y borrado masivo.

TOMA NOTA



Conviene tener muy en cuenta una advertencia al principio de la sección de la guía de usuario de Hibernate dedicada a HQL y a JPQL, cuyo enlace se ha proporcionado antes (se traduce del inglés).

Hay que tener cuidado cuando se ejecuta una sentencia UPDATE o DELETE. "Debe actuarse con precaución cuando se ejecutan operaciones masivas de actualización o borrado porque pueden provocar inconsistencias entre la base de datos y las entidades dentro del contexto de persistencia activo. En general, las operaciones masivas de actualización y borrado solo deberían realizarse dentro de una transacción en un nuevo contexto de persistencia, o antes de recuperar o acceder a entidades cuyo estado pueda verse afectado por dichas operaciones".

— Sección 4.10 de la especificación JPA 2.0

La especificación JPA (http://download.oracle.com/otn-pub/jcp/persistence-2.0-fr-eval-oth-JSpec/persistence-2_0-final-spec.pdf) se refiere solo a JPQL. Pero se entiende que este aviso es de aplicación para HQL, dado que JPQL es un subconjunto suyo, y que está incluido en la documentación de Hibernate y en una sección dedicada conjuntamente a HQL y JPQL.

Existe una interfaz `Query` tanto en la API de Hibernate como en la de JPA. En este apartado se hablará solo de la API de Hibernate. Se puede obtener una instancia de `Query` a partir de una instancia de `Session`, mediante el método `createQuery(String sentenciaHQL)`.

RECUERDA

- ✓ `createQuery` devuelve una instancia de la interfaz `org.hibernate.query.Query`. No hay que confundirla con la interfaz `org.hibernate.Query`, que está obsoleta (*deprecated*) y que está previsto eliminar para la versión 6 de Hibernate. Hay que tener esto en cuenta cuando se consulte la documentación para `Query`, para hacerlo en el lugar correcto.

El funcionamiento de la interfaz `Query` es análogo al de `PreparedStatement` de JDBC. Una `Query` puede tener parámetros que se pueden identificar por nombre además de por posición. Por ejemplo: `SELECT nomDepto FROM Departamento WHERE idDepto=:dep`.

La interfaz `Query`, de la que se ha venido hablando hasta ahora, es, hablando con propiedad, `Query<R>`, es decir, una interfaz genérica con un parámetro de tipo `R`, que es el tipo de los objetos con los que trabaja la `Query`.

En el siguiente cuadro se incluyen algunos de sus principales métodos. Muchos devuelven la propia `Query`, de manera que se pueden encadenar varias llamadas al mismo método, lo que da como resultado un código muy compacto. Por ejemplo:

```
Query q = s.createQuery(
    "FROM Departamento WHERE idSede=:idSede AND nomDepto LIKE ':nomDepto'"
).
setParameter("idSede", 12).
setParameter("nomDepto", "%ADMIN%").
setReadOnly().
setFirstResult(10).setMaxResults(5);
List<ORM.Departamento> listaDep = q.getResultList();
```

CUADRO 4.1
Métodos de `Query <R>`

Método	Funcionalidad
<code>List<R> getResultList()</code>	Devuelve la lista de resultados de una consulta HQL.
<code>R getSingleResult()</code>	Devuelve un único resultado para una consulta. Este método puede lanzar varias excepciones, entre ellas <code>NoResultException</code> cuando la consulta no devuelve ningún resultado y <code>NonUniqueResultException</code> cuando devuelve más de uno.
<code>int executeUpdate()</code>	Ejecuta la sentencia de tipo <code>UPDATE</code> o <code>DELETE</code> y devuelve el número de objetos afectados.
<code>Query<R> setParameter(...)</code>	Asigna un valor a un parámetro de la sentencia. Este método tiene muchas variantes que se pueden consultar en los Javadocs de la API de Hibernate. En general, tienen un parámetro de tipo <code>Object</code> para especificar el valor.
<code><P> Query <R> setParameterList(...)</code>	Este método tiene diversas variantes que permiten asignar a un parámetro una lista de valores. Es útil cuando se especifica en una sentencia de HQL una restricción del tipo <code>valor IN :listaValores</code> . La lista de valores se puede proporcionar en un <code>array</code> o en diversos tipos de colecciones.
<code>Query<R> setReadOnly(boolean readOnly)</code>	Especifica que los resultados recuperados serán solo para lectura, y no para modificación. Conviene usar este método si ese es el caso.

[.../...]

CUADRO 4.1 (CONT.)

<code>Query<R> setFirstResult(int posInicial)</code>	Especifican el primer resultado que recuperar de entre los obtenidos por la consulta, siendo cero el primero,
<code>Query<R> setMaxResults(int maxResult)</code>	y el número máximo de resultados que recuperar.

En el resto de esta sección se verán varios programas de ejemplo que utilizan distintos tipos de sentencias de HQL.

A) Sentencias SELECT

El siguiente programa muestra los datos de los departamentos cuyo nombre contenga una cadena de caracteres introducida por teclado. La consulta tiene un parámetro `patNombre` al que se le asigna valor con una llamada a `setParameter()` justo antes de lanzar la consulta. Como no se van a modificar los objetos persistentes recuperados, se utiliza `setReadOnly()`. La consulta se realiza con `getResultList()`, que devuelve la lista de resultados.

```
// Programa que recupera varios objetos persistentes con una consulta en HQL
package orm_query_consulta;

import java.util.List;
import java.io.InputStreamReader;
import java.io.BufferedReader;
import org.hibernate.Session;
import org.hibernate.query.Query;

public class ORM_query_consulta {
    public static void main(String[] args) {
        try (Session s = HibernateUtil.getSessionFactory().openSession()) {
            System.out.println("Introducir texto para buscar por nombre: ");
            BufferedReader br =
                new BufferedReader(new InputStreamReader(System.in));
            String nomDeppto = br.readLine();

            Query q = s.createQuery(
                "FROM Departamento WHERE nomDeppto LIKE :patNombre"
            ).setParameter("patNombre", "%" + nomDeppto + "%")
            .setReadOnly(true);

            List<ORM.Departamento> listaDep =
                (List<ORM.Departamento>) q.getResultList();
            for (ORM.Departamento unDeppto: listaDep) {
                System.out.println("Deppto: [" + unDeppto.getIdDeppto() + "] (" +
                    unDeppto.getNomDeppto() + ")");
            }
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}
```

Actividad propuesta 4.4



Escribe un programa que utilice el método `setParameterList()` para mostrar los departamentos de varias sedes, y que muestre para cada uno el identificador y el nombre de la sede y del departamento. Es necesario consultar documentación acerca de la interfaz `Query` de la API de Hibernate.

Por supuesto, se podrían modificar los objetos obtenidos mediante una consulta de HQL y grabar los cambios realizados, lo que debería hacerse dentro de una transacción.

El siguiente programa muestra los datos del empleado con mayor sueldo, que recupera mediante una consulta con `getSingleResult()`. Si hubiera más de uno, o si no existieran datos profesionales para ningún empleado, se muestra un mensaje de error específico.

```
// Programa que recupera un único objeto persistente con getSingleResult()
package ORM_Query_consulta_getSingleResult;
import org.hibernate.Session;
import org.hibernate.query.Query;
import javax.persistence.NoResultException;
import org.hibernate.NonUniqueResultException;
public class ORM_Query_consulta_getSingleResult {
    public static void main(String[] args) {
        try(Session s = HibernateUtil.getSessionFactory().openSession()) {
            Query q = s.createQuery("FROM EmpleadoDatosProf dp WHERE
                dp.sueldoBrutoAnual>=ALL(SELECT sueldoBrutoAnual FROM
                EmpleadoDatosProf)").setReadOnly(true);
            ORM.EmpleadoDatosProf datosProf =
                (ORM.EmpleadoDatosProf) q.getSingleResult();
            System.out.println("Empleado [" + datosProf.getDni() + "] "
                + "(" + datosProf.getEmpleado().getNomEmp() + ") "
                + " de departamento: "
                + datosProf.getEmpleado().getDepartamento().getNomDepto()
                + " de sede: "
                + datosProf.getEmpleado().getDepartamento().getSede().
                    getNomSede()
                + ", con sueldo: " + datosProf.getSueldoBrutoAnual()
            );
        } catch (NoResultException e) {
            System.err.println("ERROR: No hay datos profesionales para
                ningún empleado");
        } catch (NonUniqueResultException e) {
            System.err.println("ERROR: Hay más de un empleado con el salario
                máximo");
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}
```

El siguiente programa de ejemplo obtiene objetos de tipo `Empleado` y los modifica. Si para un `Empleado` no existen datos profesionales, se crea un nuevo objeto de clase `EmpleadoDatosProf` y se asocian mutuamente ambos objetos, porque entre ellos hay una relación de uno a uno. Si existen, hay que modificar el objeto de clase `EmpleadoDatosProf` que ya existe. Como sueldo, se asigna una cantidad aleatoria entre 20.000€ y 80.000€. El objeto de clase `EmpleadoDatosProf` se graba con `saveOrUpdate()` porque podría existir o no en la base de datos, y el de clase `Empleado`, con `update()`. Siempre que las modificaciones sobre los objetos se puedan realizar con una sentencia `UPDATE` de HQL, es preferible hacerlo así. Si no, habrá que recuperar los objetos con una sentencia `SELECT` de HQL y hacer las modificaciones en un bucle, como se hace en este programa.

```
// Programa que recupera objetos mediante una consulta de HQL y los modifica
package ORM_query_mas_modificacion;
import java.util.List;
import java.util.Random;
import java.math.BigDecimal;
import org.hibernate.Session;
import org.hibernate.Transaction;
import org.hibernate.query.Query;

public class ORM_rel_uno_a_uno {
    public static void main(String[] args) {
        Transaction t = null;
        try (Session s = HibernateUtil.getSessionFactory().openSession()) {
            t = s.beginTransaction();
            Query q = s.createQuery("FROM Empleado");
            List<ORM.Empleado> listaEmp = (List<ORM.Empleado>)
                q.getResultList();
            Random rand = new Random();
            for (ORM.Empleado unEmp: listaEmp) {
                ORM.EmpleadoDatosProf datosProf = unEmp.getEmpleadoDatosProf();
                if (datosProf == null) {
                    datosProf = new ORM.EmpleadoDatosProf();
                    datosProf.setEmpleado(unEmp);
                    unEmp.setEmpleadoDatosProf(datosProf);
                }
                datosProf.setCategoria("A");
                datosProf.setSueldoBrutoAnual(
                    BigDecimal.valueOf(2000000 +
                        rand.nextInt(6000000)).movePointLeft(2)
                );
                s.saveOrUpdate(datosProf);
                s.update(unEmp);
            }
            t.commit();
        } catch (Exception e) {
            e.printStackTrace(System.err);
            if (t != null) {
                t.rollback();
            }
        }
    }
}
```

B) Sentencias UPDATE y DELETE

Se pueden ejecutar sentencias de tipo UPDATE de HQL con `executeUpdate()`. El siguiente programa cambia el nombre del departamento llamado Investigación y Desarrollo por I+D. Después, borra el departamento con nombre Actividades Paranormales. Además, escribe el número de objetos afectados por cada una de estas operaciones. Una característica de HQL es que el operador `LIKE` no distingue entre mayúsculas y minúsculas. Si hubiera algún empleado en uno de los departamentos que se quiere borrar, no se realizaría la operación. Por último, hay que insistir en que, como se indicó al principio, operaciones de este tipo que pueden realizar cambios con muchos objetos podrían dar como resultado inconsistencias si hubiera objetos afectados por ellas en el contexto de persistencia en el que se realizan. Y al igual que cuando se utiliza SQL, hay que extremar la precaución con este tipo de sentencias, y especificar la cláusula `WHERE` de manera apropiada para no modificar o borrar objetos que realmente no se quiere modificar o borrar, porque los resultados podrían ser catastróficos.

```
// Ejecución de sentencias UPDATE y DELETE de HQL con executeQuery()
package orm_executeUpdate;
import org.hibernate.Session;
import org.hibernate.Transaction;
import org.hibernate.query.Query;
public class orm_executeUpdate {
    public static void main(String[] args) {
        Transaction t = null;
        try(Session s = HibernateUtil.getSessionFactory().openSession()) {
            t = s.beginTransaction();

            Query q = s.createQuery("UPDATE Departamento SET nomDepto='I+D'
                                   WHERE nomDepto='Investigación y Desarrollo'");
            int numObj = q.executeUpdate();
            System.out.println(numObj + " objetos cambiados.");

            q = s.createQuery(
                "DELETE Departamento WHERE nomDepto='ACTIVIDADES"
                " PARANORMALES'");
            numObj = q.executeUpdate();
            System.out.println(numObj + " objetos borrados.");

            t.commit();
        } catch (Exception e) {
            e.printStackTrace(System.err);
            if (t != null) {
                t.rollback();
            }
        }
    }
}
```

4.12. Correspondencia de la herencia

Una característica fundamental de los lenguajes orientados a objetos es la relación de jerarquía entre clases. Cuando una clase se define como subclase de otra, hereda sus atributos y métodos.

Por esta razón, a este tipo de relación se le denomina también relación de herencia. En Java, una clase se declara como subclase de otra mediante la palabra clave `extends`.

La principal dificultad que plantea la herencia para la persistencia es la vinculación dinámica (*late binding*). Esta consiste en que, si un objeto se define como perteneciente a una clase de la que existen subclases, solo se puede saber si ese objeto pertenece a la propia clase o a una de sus subclases en tiempo de ejecución, cuando se ejecuta el programa, y no antes, en tiempo de compilación. Cuando se quiera hacer persistente ese objeto, podría ser de esa misma clase o de una subclase suya. Por ejemplo, para esta sección se utiliza una superclase `Publicacion` y dos subclases suyas `Libro` y `Revista`. Si se declara un objeto de clase `Publicacion`, en el momento de almacenarlo en base de datos, podría ser un objeto de clase `Libro` o `Revista`, además de `Publicacion`, y en cada caso tendría un conjunto de atributos diferente. Tendrá los de `Publicación`, pero podría, además, tener los propios de `Libro` o de `Revista`.

A continuación, se muestra la definición de estas clases, que han sido escritas y no generadas automáticamente, y cumplen los requisitos para los POJO que impone Hibernate. Como es práctica habitual, los constructores de las subclases invocan el constructor de su superclase con `super()`.

```
// Fichero Publicacion.java
package ORM;

public class Publicacion implements java.io.Serializable {
    private Integer idPub;
    private String nomPub;

    public Publicacion() {
    }

    public Publicacion(String nomPub) {
        this.nomPub = nomPub;
    }

    public Integer getIdPub() {
        return this.idPub;
    }

    public void setIdPub(Integer idPub) {
        this.idPub = idPub;
    }

    public String getNomPub() {
        return this.nomPub;
    }

    public void setNomPub(String nomPub) {
        this.nomPub = nomPub;
    }
}

// Fichero Libro.java
package ORM;

public class Libro extends Publicacion implements java.io.Serializable {
    private String isbn;
```

```

private String autor;
public Libro() {
}

public Libro(String titulo, String isbn, String autor) {
    super(titulo);
    this.isbn = isbn;
    this.autor = autor;
}

public String getIsbn() {
    return this.isbn;
}
public void setIsbn(String isbn) {
    this.isbn = isbn;
}

public String getAutor() {
    return this.autor;
}
public void setAutor(String autor) {
    this.autor = autor;
}
}

// Fichero Revista.java
package ORM;
public class Revista extends Publicacion implements java.io.Serializable {
    private String issn;
    public Revista () {
    }
    public Revista(String nombre, String issn) {
        super(nombre);
        this.issn = issn;
    }
    public String getIssn() {
        return this.issn;
    }
    public void setIssn(String issn) {
        this.issn = issn;
    }
}

```

Al igual que sucede con las relaciones de muchos a muchos, las relaciones jerárquicas:

- No se pueden representar directamente en un esquema relacional.
- Se pueden representar en un diagrama E-R.
- Se pueden hacer corresponder con un esquema relacional con Hibernate.

Por tanto, se ilustrará un ejemplo de correspondencia de relaciones jerárquicas entre clases mediante un diagrama E-R.

En el módulo Bases de Datos de primer curso se estudia la obtención de esquemas relacionales a partir de diagramas E-R. Las relaciones jerárquicas no se pueden representar en esquemas relacionales, pero existe un conjunto de posibles transformaciones que se pueden aplicar a los diagramas E-R para eliminarlas. No siempre se pueden aplicar todas. Cada una tiene ventajas e inconvenientes que hay que sopesar para tomar la decisión más adecuada para cada caso particular. No se entrará en más detalles aquí. Solo se explicarán dos posibles transformaciones y cómo se puede establecer la correspondencia mediante ficheros de correspondencia de Hibernate.



Figura 4.20

Diagrama E-R con herencia



Figura 4.21

Distintas formas de eliminar una relación de herencia en un diagrama E-R

En esencia es lo mismo hablar de entidades relacionadas por una relación jerárquica de generalización/especialización en un diagrama E-R que de una jerarquía de clases. Hibernate permite implementar fácilmente los planteamientos anteriores para la correspondencia entre una jerarquía de clases y un esquema relacional, que se puede obtener directamente del diagrama E-R que resulta de eliminar la jerarquía.

En los subapartados siguientes se mostrará cómo se puede establecer la correspondencia con Hibernate de cada una de las formas anteriores, y se indicará la definición de las tablas y los ficheros de correspondencia hbm.

El programa de ejemplo que se muestra a continuación funcionará sin cambios con cualquiera de los planteamientos anteriores para la correspondencia. Crea un objeto de cada una de las clases **Publicacion**, **Libro** y **Revista** que graba en la base de datos como persistentes. Para cada proyecto hay que indicar el nombre del paquete y de la clase apropiados, de acuerdo al nombre del proyecto.

```
// Programa que crea un objeto persistente de cada clase de una jerarquía
package (...);
import org.hibernate.Session;
import org.hibernate.Transaction;
public class (...) {
    public static void main(String[] args) {
        Transaction t = null;
        try(Session s = HibernateUtil.getSessionFactory().openSession()) {
            t = s.beginTransaction();

            ORM.Publicacion pub = new ORM.Publicacion("FOLLETO INDEFINIDO");
            s.save(pub);
            Integer idPub = pub.getIdPub();

            ORM.Libro libro = new ORM.Libro("CORRESPONDENCIA OBJETO-
                RELACIONAL", "9789901234567", "LOMAS, ANTONIO");
            s.save(libro);
            Integer idLibro = libro.getIdPub();

            ORM.Revista rev = new ORM.Revista("PERSISTENCIA DE OBJETOS",
                "6789012X05");
            s.save(rev);
            Integer idRev = rev.getIdPub();

            pub = (ORM.Publicacion) s.load(ORM.Publicacion.class, idPub);
            libro = (ORM.Libro) s.load(ORM.Libro.class, idLibro);
            rev = (ORM.Revista) s.load(ORM.Revista.class, idRev);

            System.out.println("Publicación [" + pub.getIdPub() + "] " + pub.
                getNomPub());
            System.out.println("Libro [" + libro.getIdPub() + "] (" + libro.
                getNomPub() + "; " + libro.getIsbn() + "; " + libro.getAutor() +
                ")");
            System.out.println("Revista [" + rev.getIdPub() + "] (" + rev.
                getNomPub() + "; " + rev.getIsbn() + ")");

            t.commit();
        } catch (Exception e) {
            e.printStackTrace(System.err);
            if (t != null) {
                t.rollback();
            }
        }
    }
}
```

4.12.1. Eliminación de subtipos (una tabla para la jerarquía)

Se utiliza una sola tabla en la que se añaden todos los atributos del tipo y los subtipos y un campo discriminante llamado **tipo**. Todos los atributos de los subtipos deben admitir valores nulos

en la base de datos, pero, dependiendo del valor del discriminante, determinados atributos no deberían tenerlos. Para mayor seguridad, se añade una restricción adicional para controlar que **tipo** tiene un valor válido y que, para cada posible valor, no se admitan valores nulos en los atributos obligatorios para el subtipo correspondiente.

```
create table publicacion(
    id_pub integer auto_increment not null,
    nom_pub varchar(50) not null,
    tipo char(3) not null,
    isbn char(13),
    autor varchar(40) not null,
    issn char(10),
    primary key(id_pub),
    constraint check_subtipos check(tipo='pub' or (tipo='lib' and not
        isnull(isbn) and not isnull(autor)) or (tipo='rev' and not
        isnull(issn)))
);
```

El fichero de correspondencia **Publicacion.hbm.xml** sería el siguiente. Se especifica el campo discriminante con **<discriminator>**, y se especifica su valor para la superclase además de para las subclases. Las subclases se definen con **<subclass>**. Hibernate se encarga de asignar el valor apropiado a este campo, y de interpretarlo correctamente.

```
<?xml version="1.0" encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//
    EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="ORM.Publicacion" table="publicacion" catalog="proyecto_orm"
        discriminator-value="pub">
        <id name="idPub" type="java.lang.Integer">
            <column name="id_pub"/>
            <generator class="identity"/>
        </id>
        <discriminator column="tipo" type="string"/>
        <property name="nomPub" type="string">
            <column name="nom_pub" length="50"/>
        </property>
        <subclass name="ORM.Libro" discriminator-value="lib">
            <property name="isbn" type="string">
                <column name="isbn" length="13"/>
            </property>
            <property name="autor" type="string">
                <column name="autor" length="40"/>
            </property>
        </subclass>
        <subclass name="ORM.Revista" discriminator-value="rev">
            <property name="issn" type="string">
                <column name="issn" length="10"/>
            </property>
        </subclass>
    </class>
</hibernate-mapping>
```

Debe añadirse la siguiente línea en el fichero de configuración `hibernate.cfg.xml`:

```
<mapping resource="ORM/Publicacion.hbm.xml" />
```

El proyecto se puede crear en Netbeans de la siguiente forma:

1. Antes de nada, crear la tabla en la base de datos `proyecto_orm`.
2. Crear un proyecto para una aplicación de Java estándar (Java application).
3. Crear el fichero de configuración de Hibernate, `hibernate.cfg.xml`, utilizando el wizard o asistente, para la conexión `conexion_ORM`.
4. Crear un paquete (Java package) con nombre `ORM`, e incluir en él (New Java Class...) los POJO `Publicacion.java`, `Libro.java` y `Revista.java`.
5. Crear el fichero de correspondencia `Publicacion.hbm.xml` en el paquete `ORM`, junto a `Publicacion.java`. Validarlo, pulsando con el botón derecho del ratón y seleccionando la opción `Validate XML`.
6. Modificar el fichero de configuración `hibernate.cfg.xml` para añadir una línea `<mapping resource="ORM/Publicacion.hbm.xml" />`. NetBeans no deja editarlo directamente, pero se puede añadir en la sección "Mappings", con "Add...", y especificando el valor `ORM/Publicacion.hbm.xml` para `resource`.
7. Crear un fichero `HibernateUtil.java` en el paquete que contiene el fichero creado por NetBeans para la clase principal (la que contiene el método `main()`). Este fichero debe ser igual que los que se han usado, con el mismo nombre, para todos los programas hasta ahora. Se puede copiar de un proyecto anterior.
8. Modificar el fichero para la clase principal, creado automáticamente por NetBeans, para incluir los contenidos del programa de ejemplo anterior.
9. Por último, eliminar del proyecto los ficheros JAR incluidos automáticamente para una versión antigua de Hibernate e incluir los de la nueva versión de Hibernate, y que se han venido incluyendo hasta ahora en todos los proyectos.

Una vez ejecutado el programa de ejemplo, asumiendo que la tabla `publicacion` estaba vacía inicialmente, sus contenidos serán:

id_pub	nom_pub	tipo	isbn	autor	issn
1	FOLLETO INDEFINIDO	pub	NULL	NULL	NULL
2	CORRESPONDENCIA OBJETO-RELACIONAL	lib	9789901234567	LOMAS, ANTONIO	NULL
3	PERSISTENCIA DE OBJETOS	rev	NULL	NULL	6789012X05

Debería, además, garantizarse la unicidad del ISBN para los libros y del ISSN para las revistas. Esto se podría hacer creando índices únicos para la tabla por estos campos:

```
CREATE UNIQUE INDEX i_publicaciones_isbn ON publicaciones(isbn);
CREATE UNIQUE INDEX i_publicaciones_issn ON publicaciones(issn);
```

4.12.2. Una tabla por subclase (eliminación de la jerarquía)

Se utiliza una tabla por clase, y las tablas para cada subclase tienen la misma clave primaria que la de la superclase, e incluyen una clave foránea hacia la tabla para la superclase. Es una solución muy flexible y elegante. Su único inconveniente, si acaso, es que se necesita una clave foránea en cada subclase, y para obtener los datos de un objeto de una subclase se necesita hacer una consulta en SQL relacionando las tablas para la subclase y la superclase.

```
create table publicacion(
    id_pub integer auto_increment not null,
    nom_pub varchar(50) not null,
    primary key(id_pub)
);

create table libro(
    id_pub integer not null,
    isbn char(13) not null,
    autor varchar(40),
    primary key(id_pub),
    constraint fk_libro_publicacion foreign key(id_pub) references
        publicacion(id_pub)
);

create table revista(
    id_pub integer not null,
    issn char(10) not null,
    primary key(id_pub),
    constraint fk_revista_publicacion foreign key(id_pub) references
        publicacion(id_pub)
);
```

En cuanto al fichero de correspondencia `Publicacion.hbm.xml`, las subclases se definen con `<joined-subclass>` en lugar de con `<subclass>`, y no hay campo discriminante. Por lo demás es prácticamente igual.

```
<?xml version="1.0" encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="ORM.Publicacion" table="publicacion" catalog="proyecto_orm">
        <id name="idPub" type="java.lang.Integer">
            <column name="id_pub"/>
            <generator class="identity"/>
        </id>
        <property name="nomPub" type="string">
            <column name="nom_pub" length="50"/>
        </property>
        <joined-subclass name="ORM.Libro">
            <key column="id_pub"/>
            <property name="isbn" type="string">
                <column name="isbn" length="13"/>
            </property>
            <property name="autor" type="string">
                <column name="autor" length="40"/>
            </property>
        </joined-subclass>
    </class>
</hibernate-mapping>
```

```

</joined-subclass>
<joined-subclass name="ORM.Revista">
  <key column="id_pub"/>
  <property name="issn" type="string">
    <column name="issn" length="10"/>
  </property>
</joined-subclass>
</class>
</hibernate-mapping>

```

Una vez ejecutado el programa de ejemplo, asumiendo que las tablas estaban vacías inicialmente, sus contenidos serán:

publicacion

id_pub	nom_pub
1	FOLLETO INDEFINIDO
2	CORRESPONDENCIA OBJETO-RELACIONAL
3	PERSISTENCIA DE OBJETOS

libro

id_pub	isbn	autor
2	9789901234567	LOMAS, ANTONIO

revista

id_pub	issn
3	6789012X05

De nuevo, debería garantizarse la unicidad del ISBN para los libros y del ISSN para las revistas. Esto se podría hacer creando índices únicos:

```

CREATE UNIQUE INDEX i_libros_isbn ON libros(isbn);
CREATE UNIQUE INDEX i_revistas_issn ON revistas(issn);

```

4.13. Consultas con SQL

Con Hibernate también se pueden realizar consultas directamente en SQL. Conviene, de todas formas, no abusar de esta posibilidad y recurrir a ella solo en casos muy justificados. Por ejemplo, cuando supone un aumento del rendimiento muy importante, cuando se requiere una ordenación particular de los resultados, o para consultas muy complejas que no es posible realizar en HQL, o si ya se dispone de la consulta en SQL y utilizarla ahorra tiempo.

El siguiente programa realiza una consulta en SQL y obtiene los resultados como una lista de *arrays* de objetos, que acto seguido muestra en pantalla. Hibernate ofrece muchas más posibilidades, algunas muy interesantes, pero no vale la pena entretenerse aquí en ellas.

```

// Programa que realiza una consulta en SQL y muestra los resultados
package ORM_Query_SQL;
import java.util.List;
import org.hibernate.Session;

```